

2024 - 2025
PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

**SUJET : Développement d'une plateforme
intelligente de mise en relation entre marques
et créateurs de contenu**

Réalisé par: BENALI Wiem

Encadrant ESPRIT: Madame BELHAJDAOUD Sinda

Encadrant Entreprise: Monsieur NEFFATI Ahmed



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : **BENALI Wiem**

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : **Naffeti Ahmed**

Cachet & Signature / Stamp & Signature

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : **BELHAJDAOUD Sinda**

Signature / Signature

Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and
scanned.

Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover
page.

DÉDICACE

Je dédie ce travail à l'ensemble de ceux qui ont contribué, de près ou de loin, à
l'accomplissement de ce parcours.

À ma famille, pour leur soutien indéfectible, leurs encouragements constants et les valeurs
qu'ils m'ont transmises. Leur confiance et leur présence ont été un moteur essentiel tout au
long de mon cheminement académique et personnel.

À mon époux, dont la patience, la compréhension et l'appui quotidien ont constitué une
source de motivation inestimable. Merci pour ton soutien constant et ta bienveillance, qui ont
été précieux dans les moments clés de ce projet.

À la famille de mon époux, pour leur accueil chaleureux, leur respect et leur soutien sincère.
Leur bienveillance a contribué à créer un environnement propice à la réussite et à l'équilibre.

BENALI Wiem

REMERCIEMENTS

Je tiens à exprimer ma profonde reconnaissance à Madame **Sinda BELHAJDAOUD**, mon encadrante académique, pour son suivi attentif, sa disponibilité et la qualité de ses orientations tout au long de ce projet. Ses conseils pertinents et son exigence bienveillante ont largement contribué à la réalisation et à la réussite de ce travail.

J'adresse également mes sincères remerciements à Monsieur **Ahmed NAFFETI**, mon encadrant professionnel au sein de l'entreprise **Bee Coders**, pour la confiance qu'il m'a accordée, la clarté de ses explications et la richesse de ses retours constructifs. Son accompagnement m'a permis de développer aussi bien mes compétences techniques que mon sens professionnel.

Ma gratitude s'étend aussi à l'ensemble de l'équipe **Bee Coders**, pour leur accueil chaleureux, leur disponibilité et leur esprit collaboratif. Leur soutien et la qualité de nos échanges ont fait de ce stage une expérience à la fois formatrice, enrichissante et profondément humaine.

Wiem Ben Ali

TABLE DES MATIÈRES

INTRODUCTION GENERALE	1
CHAPITRE 1 : CONTEXTE GENERAL DU PROJET	4
INTRODUCTION	4
1.1 CADRE GENERAL DU PROJET	4
1.2 PRESENTATION DE L'ORGANISME D'ACCUEIL	4
1.2.1 Bee Coders	4
1.2.2 Services proposés par Bee Coders	5
1.5 METHODOLOGIE DE TRAVAIL	9
1.5.1 Choix du cadre Scrum	10
1.5.2 Rôles dans la méthode Scrum	11
CONCLUSION	11
CHAPITRE 2 : ANALYSE ET SPECIFICATION DES BESOINS	12
INTRODUCTION	12
2.1 IDENTIFICATION DES ACTEURS DE L'APPLICATION	12
2.1.1 Créateur de contenu	12
2.1.2 Marque	12
2.1.3 Super Administrateur	13
2.2 ÉTUDE DES BESOINS	13
2.2.1 Besoins fonctionnels	13
❖ Marque	13
❖ Créateur	14
❖ Super Administrateur	14
2.2.2 Besoins non fonctionnels	15
2.3 GESTION DU PROJET AVEC SCRUM	15
2.3.1 Backlog du produit	15
2.4 DIAGRAMME DE CAS D'UTILISATION GLOBAL	18
2.5 DIAGRAMME DE CLASSES	19
2.6 ENVIRONNEMENT DE TRAVAIL DANS CE STAGE	19
2.6.1 Frameworks et langages utilisés	19
2.6.2 Environnement logiciel	21
2.6.3 Environnement matériel	23
2.7 ARCHITECTURE GLOBALE	23
2.7.1 Architecture physique	23
2.7.2 Architecture logique	25
2.8 SPRINT BACKLOG GENERAL DE LA PLATEFORME MARKET BRIDGE	26
CONCLUSION	27
CHAPITRE 3 : ÉTUDE ET RÉALISATION DU PREMIER SPRINT	29
INTRODUCTION	29
3.1 SPRINT BACKLOG	29
3.2 SPECIFICATION FONCTIONNELLE DU PREMIER SPRINT	30
3.2.1 Diagramme de cas d'utilisation du premier sprint	30
3.2.2 Description textuelle des cas d'utilisation du premier sprint	31
3.3 CONCEPTION DU PREMIER SPRINT	35
3.3.1 Diagrammes de séquence	35
3.4 RÉALISATION DU PREMIER SPRINT	39
3.4.1 Interfaces réalisées	40
3.5 REVUE DU PREMIER SPRINT	44
3.6 RETROSPECTIVE DU PREMIER SPRINT	45
CONCLUSION	45

CHAPITRE 4 : ÉTUDE ET RÉALISATION DU DEUXIÈME SPRINT	46
INTRODUCTION	46
4.1 SPRINT BACKLOG	46
4.2 SPECIFICATION FONCTIONNELLE DU DEUXIEME SPRINT.....	47
4.2.1 <i>Diagramme de cas d'utilisation du deuxième sprint</i>	47
4.2.2 <i>Description textuelle des cas d'utilisation</i>	48
4.3 CONCEPTION DU DEUXIEME SPRINT	51
4.3.1 <i>Diagrammes de séquence</i>	51
4.4 REALISATION DU SPRINT	54
4.4.1 <i>Interfaces réalisées</i>	55
4.5 REVUE DU DEUXIEME SPRINT	59
4.6 RETROSPECTIVE DU DEUXIEME SPRINT	59
CONCLUSION	60
CHAPITRE 5 : ÉTUDE ET RÉALISATION DU TROISIÈME SPRINT.....	61
INTRODUCTION	61
5.1 SPRINT BACKLOG	61
5.2 SPECIFICATION FONCTIONNELLE DU TROISIEME SPRINT.....	62
5.2.1 <i>Diagramme de cas d'utilisation du troisième sprint</i>	63
5.2.2 <i>Description textuelle des cas d'utilisation</i>	63
5.3 CONCEPTION DU TROISIEME SPRINT	68
5.3.1 <i>Diagrammes de séquence</i>	68
5.4 REALISATION DU SPRINT	70
5.4.1 <i>Interfaces réalisées</i>	71
5.5 REVUE DU TROISIEME SPRINT	74
5.6 RETROSPECTIVE DU TROISIEME SPRINT	75
CONCLUSION	76
CHAPITRE 6 : ÉTUDE ET RÉALISATION DU QUATRIÈME SPRINT.....	77
INTRODUCTION	77
6.1 SPRINT BACKLOG	77
6.2 SPECIFICATION FONCTIONNELLE DU QUATRIEME SPRINT	78
6.2.1 <i>Diagramme de cas d'utilisation du quatrième sprint</i>	78
6.2.2 <i>Description textuelle du cas d'utilisation</i>	79
6.3 REALISATION DU SPRINT	79
6.3.1 <i>Interfaces réalisées</i>	80
6.4 REVUE DU QUATRIEME SPRINT.....	84
6.5 RETROSPECTIVE DU QUATRIEME SPRINT.....	84
CONCLUSION	85
CHAPITRE 7: ÉTUDE ET RÉALISATION DU CINQUIÈME SPRINT	86
INTRODUCTION	86
7.1 SPRINT BACKLOG	86
7.2 SPECIFICATION FONCTIONNELLE DU CINQUIEME SPRINT.....	87
7.2.1 <i>Diagramme de cas d'utilisation du cinquième sprint</i>	87
7.2.2 <i>Description textuelle des cas d'utilisation</i>	88
7.3 REALISATION DU SPRINT	88
7.3.1 <i>Interfaces réalisées</i>	90
7.3.2 <i>Implémentation du microservice Chatbot IA (FastAPI)</i>	91
7.4 REVUE DU CINQUIEME SPRINT.....	93
7.5 RETROSPECTIVE DU CINQUIEME SPRINT.....	93
CONCLUSION	94
CHAPITRE 8 : ÉTUDE ET RÉALISATION DU SIXIÈME SPRINT	95
INTRODUCTION	95

8.1	SPRINT BACKLOG	95
8.2	SPECIFICATION FONCTIONNELLE DU SIXIEME SPRINT	96
8.2.1	<i>Diagramme de cas d'utilisation du sixième sprint</i>	96
8.2.2	<i>Description textuelle des cas d'utilisation</i>	97
8.3	CONCEPTION DU SIXIEME SPRINT	98
8.3.1	<i>Diagrammes de séquence</i>	98
8.4	REALISATION DU SPRINT	99
8.4.1	<i>Interfaces réalisées</i>	100
8.5	REVUE DU SIXIEME SPRINT	102
8.6	RETROSPECTIVE DU SIXIEME SPRINT	103
	CONCLUSION	103
CHAPITRE 9 : ÉTUDE ET RÉALISATION DU SEPTIÈME SPRINT.....		104
	INTRODUCTION	104
9.1	SPRINT BACKLOG	104
9.2	SPECIFICATION FONCTIONNELLE DU SEPTIEME SPRINT.....	105
9.2.1	<i>Diagramme de cas d'utilisation du septième sprint</i>	105
9.2.2	<i>Description textuelle des cas d'utilisation</i>	106
9.3	REALISATION DU SPRINT	108
9.3.1	<i>Interfaces réalisées</i>	108
9.4	REVUE DU SEPTIEME SPRINT	110
9.5	RETROSPECTIVE DU SEPTIEME SPRINT	111
	CONCLUSION	111
CHAPITRE 10 : PHASE FINALE ET INDUSTRIALISATION (SPRINT DEVOPS).....		112
	INTRODUCTION	112
10	METHODOLOGIE DEVOPS APPLIQUEE A MARKET BRIDGE	113
10.1.1	<i>Intégration, livraison et déploiement continu</i>	113
10.1.2	<i>Intégration continue (CI)</i>	113
10.1.3	<i>Livraison et déploiement continu (CD)</i>	114
10.1.4	<i>DevOps</i>	114
10.1.5	<i>Virtualisation vs Conteneurisation</i>	115
10.1.6	<i>La Virtualisation</i>	115
10.1.7	<i>La Conteneurisation</i>	116
10.1.8	<i>Architecture et fonctionnement du pipeline CI/CD</i>	116
10.1	INTRODUCTION	117
10.2	LES OUTILS DEVOPS DU PIPELINE MARKET BRIDGE	117
10.2.1	<i>GitHub</i>	117
10.2.2	<i>Jenkins</i>	118
10.3	INTEGRATION ET DEPLOIEMENT CONTINU DU BACKEND	120
10.3.1	<i>Pipelines CI & CD</i>	120
10.3.2	<i>Analyse de code avec SonarQube</i>	121
10.3.3	<i>Gestion des artefacts avec Nexus</i>	122
10.3.4	<i>Conteneurisation et déploiement Docker</i>	122
10.3.5	<i>Supervision et monitoring</i>	122
10.3.6	<i>Alerting automatique (Backend)</i>	123
10.3.7	<i>Résultat du pipeline Backend</i>	124
10.4	INTEGRATION ET DEPLOIEMENT DU FRONTEND.....	125
10.4.1	<i>Étapes principales</i>	125
10.4.2	<i>Alerting automatique (Frontend)</i>	126
10.4.3	<i>Résultat du pipeline Frontend</i>	127
	CONCLUSION	128
CONCLUSION GENERALE		129

LISTE DES FIGURES

FIGURE 1: LOGO DE BEE CODERS	5
FIGURE 2: LOGO CONNECT CONTENT.....	7
FIGURE 3: LOGO KOLSQUARE	7
FIGURE 4: LOGO UPFLUENCE.....	7
FIGURE 5: SCHEMA DU SCRUM.....	10
FIGURE 6: EXPLICATION METHODE SCRUM.....	10
FIGURE 7: DIAGRAMME DE CAS D'UTILISATION GLOBAL DE LA PLATEFORME MARKET BRIDGE.....	18
FIGURE 8: DIAGRAMME DE CLASSES GLOBAL DE LA PLATEFORME MARKET BRIDGE	19
FIGURE 9: LOGO SPRING BOOT.....	20
FIGURE 10: LOGO ANGULAR	20
FIGURE 11: LOGO FLASK	20
FIGURE 12: LOGO FASTAPI	20
FIGURE 13: LOGO MYSQL	20
FIGURE 14: LOGO DOCKER	21
FIGURE 15: LOGO POSTMAN.....	21
FIGURE 16: LOGO GITHUB	21
FIGURE 17: LOGO JIRA.....	21
FIGURE 18: LOGO VISUAL PARADIGM	21
FIGURE 19: LOGO JENKINS.....	22
FIGURE 20: LOGO SONARQUBE.....	22
FIGURE 21: LOGO NEXUS	22
FIGURE 22: LOGO PROMETHEUS & GRAFANA	22
FIGURE 23: LOGO INTELLIJ IDEA	22
FIGURE 24: LOGO VISUAL STUDIO CODE	22
FIGURE 25: ARCHITECTURE PHYSIQUE DE LA PLATEFORME MARKET BRIDGE	24
FIGURE 26: ARCHITECTURE LOGIQUE DE LA PLATEFORME MARKET BRIDGE.....	26
FIGURE 27: DIAGRAMME DE CAS D'UTILISATION DU PREMIER SPRINT (MARKET BRIDGE).....	31
FIGURE 28: DIAGRAMME DE SEQUENCE : CREER UN COMPTE	36
FIGURE 29: DIAGRAMME DE SEQUENCE : SE CONNECTER.....	37
FIGURE 30: DIAGRAMME DE SEQUENCE : GERER LE PROFIL	38
FIGURE 31: DIAGRAMME DE SEQUENCE : GERER LES UTILISATEURS.....	39
FIGURE 32: INTERFACE D'INSCRIPTION MARQUE.....	40
FIGURE 33: INTERFACE D'INSCRIPTION CREATEUR	41
FIGURE 34: INTERFACE D'INSCRIPTION AVEC QR CODE 2FA.....	41
FIGURE 35: INTERFACE DE CONNEXION SECURISEE (JWT + 2FA).....	42
FIGURE 36: INTEFACE DE CONNEXION SECURISEE SANS 2FA	42
FIGURE 37: INTERFACE DE PROFIL UTILISATEUR (CREATEUR).....	43
FIGURE 38: INTERFACE DE PROFIL UTILISATEUR (MARQUE)	43
FIGURE 39: INTERFACE DE GESTION DES UTILISATEURS – CREATEURS.....	43
FIGURE 40: INTERFACE DE GESTION DES UTILISATEURS – MARQUES.....	44
FIGURE 41: DIAGRAMME DE CAS D'UTILISATION DU DEUXIEME SPRINT (MARKET BRIDGE):.....	48
FIGURE 42: DIAGRAMME DE SEQUENCE : POSTULATION D'UN CREATEUR ET NOTIFICATION EN TEMPS REEL.....	52
FIGURE 43: DIAGRAMME DE SEQUENCE : POSTULATION D'UNE MARQUE ET NOTIFICATION EN TEMPS REEL.....	53
FIGURE 44: DIAGRAMME DE SEQUENCE : GESTION DES DEMANDES (MARQUES ET CREATEURS)	54
FIGURE 45: INTERFACE DES PACKS DE FORMATION.....	55
FIGURE 46: INTERFACE DE POSTULATION D'UN CREATEUR	56
FIGURE 47: INTERFACE DES PACKS DE SERVICE	56
FIGURE 48: INTERFACE DE POSTULATION D'UNE MARQUE	57
FIGURE 49: INTERFACE DES DEMANDES DES MARQUES	57
FIGURE 50: INTERFACE DES DEMANDES DES CREATEURS.....	58
FIGURE 51: INTERFACE DE GESTION DES PACKS DE FORMATIONS.....	58
FIGURE 52: INTERFACE DE GESTION DES PACKS DE SERVICES	58
FIGURE 53: INTERFACE DE SUIVI DES DEMANDES	59

FIGURE 54: DIAGRAMME DE CAS D'UTILISATION DU TROISIEME SPRINT (MARKET BRIDGE)	63
FIGURE 55: DIAGRAMME DE SEQUENCE : ÉVALUATION ET BOUCLE D'AMELIORATION.....	69
FIGURE 56: DIAGRAMME DE SEQUENCE : NOUVELLE PROPOSITION	70
FIGURE 57: INTERFACE DU CREATEUR : RECEPTION D'UNE NOUVELLE PROPOSITION	71
FIGURE 58: INTERFACE DU SUPER ADMINISTRATEUR : MODERATION DES CONTENUS	71
FIGURE 59: INTERFACE DE LA MARQUE : ÉVALUATION DU CONTENU (CAS D'ACCEPTATION)	72
FIGURE 60: INTERFACE DE LA MARQUE : ÉVALUATION DU CONTENU (CAS DE REFUS)	72
FIGURE 61: INTERFACE DU SUPER ADMINISTRATEUR : FEEDBACKS ET VALIDATIONS.....	73
FIGURE 62: INTERFACE DU CREATEUR : ENVOI D'UN CONTENU AMELIORE	73
FIGURE 63: INTERFACE DU SUPER ADMINISTRATEUR : MODERATION DES CONTENUS AMELIORES	73
FIGURE 64: INTERFACE DE LA MARQUE : VISUALISATION DE LA VERSION AMELIOREE ET DECISION FINALE	74
FIGURE 65: DIAGRAMME DE CAS D'UTILISATION DU QUATRIEME SPRINT (MARKET BRIDGE)	78
FIGURE 66: INTERFACE DES RECOMMANDATIONS IA : TOP 5 GLOBAL	80
FIGURE 67: INTERFACE DES RECOMMANDATIONS IA FILTREE	81
FIGURE 68: STRUCTURE GLOBALE DU MICROSERVICE IA.....	82
FIGURE 69: INITIALISATION ET CONFIGURATION DU MICROSERVICE DE RECOMMANDATION IA (FLASK)	82
FIGURE 70: LECTURE ET PREPARATION DES DONNEES CREATEURS DEPUIS LA BASE MYSQL	83
FIGURE 71: NETTOYAGE ET NORMALISATION DES DONNEES D'ENTREE POUR LE MOTEUR IA	83
FIGURE 72: ALGORITHME DE CALCUL DU SCORE DE COMPATIBILITE ENTRE MARQUE ET CREATEUR	83
FIGURE 73: ENDPOINT REST /RECOMMEND ET GENERATION DES RESULTATS IA (TOP 5).....	84
FIGURE 74: DIAGRAMME DE CAS D'UTILISATION DU CINQUIEME SPRINT (MARKET BRIDGE)	87
FIGURE 75: EXEMPLE DE CONVERSATION AVEC LE CHATBOT.....	90
FIGURE 76: EXEMPLE D'OFFRE PDF GENEREE.....	90
FIGURE 77: STRUCTURE DU MICROSERVICE CHATBOT IA (DOSSIER CHATBOT_AI)	91
FIGURE 78: INITIALISATION DU MICROSERVICE CHATBOT IA ET CONFIGURATION FASTAPI.....	91
FIGURE 79: GESTION DU CONTEXTE CONVERSATIONNEL ET PERSISTANCE DES ECHANGES	92
FIGURE 80: TRAITEMENT NLP ET LOGIQUE CONVERSATIONNELLE DU CHATBOT IA.....	92
FIGURE 81: GENERATION DYNAMIQUE DES OFFRES PDF ET TELECHARGEMENT.....	93
FIGURE 82: DIAGRAMME DE CAS D'UTILISATION DU SIXIEME SPRINT.....	97
FIGURE 83: DIAGRAMME DE SEQUENCE – PAIEMENT ET TELECHARGEMENT SECURISE (MARKET BRIDGE)	99
FIGURE 84: INTERFACE DE NOTATION ET PAIEMENT (MARQUE).....	100
FIGURE 85: INTERFACE DE CONFIRMATION STRIPE	100
FIGURE 86: INTERFACE STRIPE CHECKOUT (MODE TEST)	101
FIGURE 87: INTERFACE DE CONFIRMATION ET TELECHARGEMENT AUTOMATIQUE.....	101
FIGURE 88: INTERFACE DE VISUALISATION SECURISEE AVANT PAIEMENT	102
FIGURE 89: DIAGRAMME DE CAS D'UTILISATION DU SEPTIEME SPRINT (MARKET BRIDGE).....	105
FIGURE 90: INTERFACE TABLEAU DE BORD ANALYTIQUE - SUPER ADMINISTRATEUR.....	108
FIGURE 91: INTERFACE DES STATISTIQUES ET EXPORT PDF	109
FIGURE 92: SIGNATURE DU RAPPORT AVANT EXPORT	109
FIGURE 93: RAPPORT PDF FINAL GENERE PAR LA PLATEFORME	110
FIGURE 94: ARCHITECTURE DEVOPS DU PROJET MARKET BRIDGE	112
FIGURE 95: CHAINE DEVOPS DU PROJET MARKET BRIDGE – JENKINS CI/CD ET ALERTES E-MAIL	115
FIGURE 96: DIFFERENCE ENTRE MACHINE VIRTUELLE ET CONTENEUR DOCKER	115
FIGURE 97: ARCHITECTURE DOCKER COMPOSE DU PROJET MARKET BRIDGE	116
FIGURE 98: EXECUTION DES CONTENEURS DOCKER DU PROJET MARKET BRIDGE	116
FIGURE 99: VUE D'ENSEMBLE DES DEPOTS GITHUB MARKET BRIDGE (FRONTEND ET BACKEND)	118
FIGURE 100: DEPOT GITHUB MARKET BRIDGE FRONTEND SYNCHRONISE AVEC JENKINS.....	118
FIGURE 101: DEPOT GITHUB MARKET BRIDGE BACKEND SYNCHRONISE AVEC JENKINS.....	118
FIGURE 102: TABLEAU JENKINS DES JOBS CI/CD.....	119
FIGURE 103: PLUGINS INSTALLES DANS JENKINS.....	119
FIGURE 104: CONFIGURATION DES IDENTIFIANTS DOCKER HUB	120
FIGURE 105: SCRIPT DU PIPELINE CI	120
FIGURE 106: SCRIPT DU PIPELINE CD.....	121
FIGURE 107: TABLEAU SONARQUBE DU PROJET MARKET BRIDGE	121
FIGURE 108: DEPOT NEXUS MARKET BRIDGE	122

FIGURE 109: LISTE DES IMAGES DOCKER DU PROJET	122
FIGURE 110: LISTE DES CONTENEURS DOCKER ACTIFS	122
FIGURE 111: INTERFACE PROMETHEUS – ETAT DES SERVICES.....	123
FIGURE 112: DASHBOARD GRAFANA MARKET BRIDGE.....	123
FIGURE 113: E-MAIL JENKINS BACKEND – SUCCES	124
FIGURE 114: E-MAIL JENKINS – ÉCHEC	124
FIGURE 116: DUREE DES ETAPES DU PIPELINE CI/CD – BACKEND.....	125
FIGURE 117: CLONAGE DEPUIS GITHUB.....	126
FIGURE 118: BUILD ANGULAR.....	126
FIGURE 119: CREATION DE L’IMAGE DOCKER	126
FIGURE 120: PUBLICATION SUR DOCKER HUB	126
FIGURE 121: E-MAIL JENKINS FRONTEND – SUCCES	127
FIGURE 123: RESULTAT DU PIPELINE JENKINS – FRONTEND (ETAPES DU BUILD #33)	127

LISTE DES TABLEAUX

TABEAU 1: TABLEAU COMPARATIF DES PLATEFORMES DE MISE EN RELATION MARQUE-CREATEUR	7
TABEAU 2: BACKLOG DES USER STORIES DE LA PLATEFORME MARKET BRIDGE.....	15
TABEAU 3: ENVIRONNEMENT MATERIEL	23
TABEAU 4: SPRINT BACKLOG GENERAL DE LA PLATEFORME MARKET BRIDGE.....	26
TABEAU 5: SPRINT BACKLOG DU PREMIER SPRINT.....	29
TABEAU 6: DESCRIPTION TEXTUELLE DU CAS D'UTILISATION - CREER UN COMPTE.....	31
TABEAU 7: DESCRIPTION TEXTUELLE DU CAS D'UTILISATION - SE CONNECTER	32
TABEAU 8: DESCRIPTION TEXTUELLE DU CAS D'UTILISATION - VERIFIER LE CODE 2FA.....	33
TABEAU 9: DESCRIPTION TEXTUELLE DU CAS D'UTILISATION - GERER LES UTILISATEURS	34
TABEAU 10: DESCRIPTION TEXTUELLE DU CAS D'UTILISATION - GERER LE PROFIL (MARQUE OU CREATEUR)	34
TABEAU 11: SPRINT BACKLOG DU DEUXIEME SPRINT	46
TABEAU 12: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - CHOISIR UN PACK (CREATEUR).....	48
TABEAU 13: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - POSTULER A UNE FORMATION (CREATEUR)	48
TABEAU 14: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - CHOISIR UN PACK (MARQUE).....	49
TABEAU 15: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - SOUMETTRE UN FORMULAIRE (MARQUE)	50
TABEAU 16: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - GERER LES PACKS (SUPER ADMIN)	50
TABEAU 17: SPRINT BACKLOG DU TROISIEME SPRINT	62
TABEAU 18: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - CONSULTER LES NOTIFICATIONS (MARQUE)	63
TABEAU 19: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - EVALUER UN CONTENU (MARQUE)	64
TABEAU 20: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - FOURNIR UN FEEDBACK (MARQUE)	64
TABEAU 21: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - UPLOADER UN CONTENU (CREATEUR)	65
TABEAU 22: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - CONSULTER LA MODERATION (SUPER ADMIN).....	66
TABEAU 23: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - SUPERVISER LES FEEDBACKS (SUPER ADMIN)	66
TABEAU 24: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - GERER LES PACKS (SUPER ADMIN)	67
TABEAU 25: SPRINT BACKLOG DU QUATRIEME SPRINT	77
TABEAU 26: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - CONSULTER LES RECOMMANDATIONS IA	79
TABEAU 27: SPRINT BACKLOG DU CINQUIEME SPRINT.....	86
TABEAU 28: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION : UTILISER LE CHATBOT IA (MARQUE)	88
TABEAU 29: SPRINT BACKLOG DU SIXIEME SPRINT.....	95
TABEAU 30: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - PAYER UN CONTENU VALIDE VIA STRIPE (MARQUE)	97
TABEAU 31: BACKLOG DU SEPTIEME SPRINT	104
TABEAU 32: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - CONSULTER LES STATISTIQUES GLOBALES	106
TABEAU 33: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - VISUALISER LA REPARTITION PAR PACK OU PAR GENRE	106
TABEAU 34: DESCRIPTION TEXTUELLE DES CAS D'UTILISATION - EXPORTER UN RAPPORT PDF.....	107

Introduction Générale

Le marketing digital est en plein essor, de plus en plus de créateurs de contenu voient le jour et les marques sont de plus en plus dans l'obligation de produire des campagnes non seulement authentiques mais aussi engageantes et adaptées à leur audience. Toutefois, les plateformes existantes actuellement sont limitées sur plusieurs points: absence de moteur de recommandation intelligent, manque de sécurité dans les échanges, complexité des paiements et absence d'un système de communication et de suivi en temps réel.

C'est dans ce contexte que s'inscrit le présent projet de fin d'études, intitulé Market Bridge est une plateforme web intelligente et sécurisée conçue pour relier les marques aux créateurs de contenu de manière organique et efficace.

L'objectif principal est d'offrir un écosystème complet et intégré, permettant une collaboration fluide entre les acteurs à travers :

- Un **système d'authentification avancée** (JWT, 2FA, QR Code via Google Authenticator) ;
- Un **moteur de recommandation IA** basé sur l'intelligence artificielle pour un appariement optimal entre les besoins des marques et les profils des créateurs ;
- Un **chatbot IA local (offline)**, autonome et sans dépendance externe, utilisant **spaCy** pour comprendre le langage naturel et générer des **livrables PDF personnalisés** ;
- Un **système de paiement sécurisé Stripe**, garantissant la fiabilité des transactions et le déblocage contrôlé des contenus ;
- Un **système de notifications en temps réel**, informant instantanément les utilisateurs des validations, refus ou nouvelles propositions ;
- Un **module de modération et de feedbacks**, facilitant la supervision et l'amélioration continue des contenus ;
- Un **module de statistiques** et de tableaux de bord administrateur pour un suivi global des activités.

La plateforme s'appuie sur une architecture technique à la fois moderne et performante, structurée autour des éléments suivants :

- **Frontend** : Angular 15, TypeScript et Bootstrap 5 pour offrir une interface utilisateur fluide, intuitive et adaptée à tous les écrans ;

- **Backend** : développé avec Spring Boot 3, intégrant Spring Security, JWT, BCryptPasswordEncoder, JPA/Hibernate et MySQL pour assurer la sécurité des accès et la gestion des données ;
- **Moteur de recommandation IA** : basé sur Flask (Python 3.10) et scikit-learn, il permet de calculer les affinités entre marques et créateurs ;
- **Chatbot IA local** : implémenté avec FastAPI, spaCy (modèle fr_core_news_md) et ReportLab pour l'analyse du langage naturel et la génération de documents PDF ;
- **Notifications en temps réel** : une communication directe et instantanée entre les utilisateurs (marques, créateurs, superadmin), rendue possible grâce aux WebSockets et aux services Angular ;
- **Paiement** : intégration de Stripe Checkout avec redirection automatique via success_url et validation du statut de paiement ;
- **DevOps** : chaîne complète d'intégration continue et de déploiement (CI/CD) via Jenkins, avec SonarQube pour l'analyse de qualité de code, Nexus pour la gestion des artefacts, et Prometheus + Grafana pour la supervision et l'alerte

L'ensemble du projet a été **entièrement conteneurisé à l'aide de Docker Compose**, afin d'assurer une portabilité optimale, une cohérence constante et une stabilité entre tous les environnements.

Le développement s'est appuyé sur **une méthode agile de type Scrum**, découpée en plusieurs sprints successifs, chacun axé sur la réalisation d'un ensemble fonctionnel bien défini.

Le rapport est structuré comme suit :

- **Chapitre 1 – Présentation générale du projet :**
Présente le cadre du stage, l'entreprise *Bee Coders*, la problématique, les objectifs du projet *Market Bridge* et la méthodologie de travail.
- **Chapitre 2 – Analyse et spécification des besoins :**
Détaille les besoins fonctionnels et non fonctionnels, les acteurs, les cas d'utilisation UML, le backlog produit et l'architecture globale (physique et logique).
- **Chapitres 3 à 7 – Détails des sprints fonctionnels :**
Retracent les différentes étapes de développement :

- Sprint 1 : Authentification sécurisée (JWT, 2FA, QR Code) et gestion des rôles ainsi que la gestion des profils utilisateurs (marques et créateurs) ;
- Sprint 2 : Gestion des formulaires et notifications en temps réel ;
- Sprint 3 : Modération et workflow collaboratif (marques ↔ créateurs ↔ superadmin) en temps réel ;
- Sprint 4 : Moteur de recommandation IA (Flask + Spring Boot + MySQL) ;
- Sprint 5 : Chatbot IA local (FastAPI + spaCy + ReportLab) ;
- Sprint 6 : Paiement Stripe et sécurisation des contenus (filigrane, blocage de téléchargement) ;
- Sprint 7 : Module de statistiques et tableau de bord analytique ;

Chapitre 10 – Phase finale et industrialisation (Sprint DevOps):

Ce dernier chapitre est consacré à la mise en place complète de la chaîne DevOps et à la préparation du déploiement du projet.

Il détaille la conteneurisation de l'application via Docker, la configuration du pipeline CI/CD avec Jenkins, ainsi que l'intégration d'outils essentiels tels que SonarQube, Nexus, Prometheus et Grafana. L'objectif est d'assurer une supervision continue et un système d'alertes automatisées garantissant la stabilité du produit.

Cette étape marque l'aboutissement du projet, en consolidant sa fiabilité, sa maintenabilité et sa capacité à être déployé dans un environnement industriel robuste.

En conclusion, **Market Bridge** s'impose comme une plateforme **innovante, sécurisée et intelligente**, combinant intelligence artificielle, communication en temps réel et automatisation DevOps.

Ce projet démontre la maîtrise de la conception d'une application web **moderne, évolutive et performante**, répondant aux exigences du marketing digital et aux standards actuels de l'ingénierie logicielle.

Chapitre 1 : Contexte Général du Projet

Introduction

Ce premier chapitre établit le cadre global du projet de fin d'études intitulé **Market Bridge**. Il débute par une présentation de l'environnement professionnel du stage, à savoir l'entreprise d'accueil **Bee Coders**, avant de traiter de la problématique actuelle du marketing digital en Tunisie et de la solution innovante proposée dans ce contexte. Une analyse des solutions existantes a été réalisée afin d'en cerner les limites. Enfin, la méthodologie suivie, fondée sur **le cadre agile Scrum**, est décrite en détail pour expliquer le processus de conception et de développement de la plateforme.

1.1 Cadre général du projet

Ce projet a été mené dans le cadre de l'obtention du diplôme d'ingénieur en informatique à **l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)**, avec une spécialisation en Architecture Logicielle. Il a été réalisé au cours d'un stage de fin d'études d'une durée de sept mois, effectué au sein de l'entreprise **Bee Coders**.

Ce stage a permis de mobiliser l'ensemble des compétences techniques et méthodologiques acquises durant la formation, en vue de concevoir une solution complète, innovante et industrialisable.

Le projet **Market Bridge** a pour but **de numériser la relation entre les marques et les créateurs de contenu** dans un environnement centralisé, sécurisé et professionnel. Il propose une expérience intégrée regroupant la recherche de talents, la gestion des propositions, la validation des contenus, le paiement sécurisé ainsi que l'analyse des performances.

L'objectif principal est de fournir un outil intelligent et collaboratif capable d'automatiser le processus de mise en relation, tout en assurant la sécurité des données, la traçabilité des transactions et la fluidité des échanges entre les différents acteurs du marché.

1.2 Présentation de l'organisme d'accueil

1.2.1 Bee Coders

Créée en 2020, **Bee Coders** est une entreprise tunisienne spécialisée dans le développement de solutions web, mobiles et métiers sur mesure. Elle utilise des technologies récentes pour livrer des produits performants.

L'entreprise accueille aussi des stagiaires, leur offrant une première expérience concrète en développement logiciel.



Figure 1: Logo de Bee Coders

1.2.2 Services proposés par Bee Coders

Bee Coders propose plusieurs services :

- Conseil technologique
- Développement web et mobile
- Intégration de solutions sur mesure
- Formation et encadrement de jeunes développeurs

Grâce à une approche agile et collaborative, l'entreprise veille à livrer des produits de qualité, adaptés aux besoins de ses clients.

1.3 Présentation du projet Market Bridge

Le projet **Market Bridge** prend forme dans un contexte où le contenu digital explose et où les collaborations entre marques et créateurs deviennent de plus en plus variées. Cette évolution appelle un outil technologique capable de centraliser, sécuriser et automatiser ces échanges. C'est dans cette optique qu'a été pensée la plateforme **Market Bridge** : une solution web innovante destinée à structurer et **professionnaliser la relation entre marques, créateurs de contenu et administrateurs**.

Elle vise à simplifier les collaborations, renforcer la confiance et valoriser le travail créatif à l'aide d'outils **intelligents** et **automatisés**.

1.3.1 Objectif général du projet

Le but du projet est de concevoir une plateforme web nommée **Market Bridge**, dédiée à la mise en relation entre **marques et créateurs de contenu**.

Elle offre un espace centralisé où les marques peuvent exprimer leurs besoins, consulter des profils ciblés et recevoir des propositions adaptées. Les créateurs, eux, peuvent mettre en avant leurs compétences, partager leurs réalisations et suivre l'évolution de leurs partenariats.

La plateforme intègre des fonctionnalités modernes comme une **IA de recommandation**, un **chatbot d'assistance**, un **système de modération** et un **module de paiement sécurisé via Stripe Checkout**, répondant ainsi aux exigences actuelles du marketing digital.

1.3.2 Problématique

En Tunisie, malgré la croissance du marketing d'influence, la collaboration entre marques et créateurs de contenu reste souvent désorganisée. Les marques peinent à trouver rapidement des profils fiables, tandis que les créateurs ont du mal à valoriser leur travail et à suivre leurs démarches.

L'absence d'un système centralisé pour gérer les échanges, modérer les contenus, sécuriser les paiements et assurer un suivi en temps réel complique encore davantage ces interactions. D'où la nécessité d'une **plateforme unifiée, intelligente et sécurisée**, qui automatise le processus de collaboration et améliore l'expérience de tous les acteurs impliqués.

1.4 Étude préalable

Cette section présente une analyse préparatoire en trois parties : l'étude des solutions existantes, leurs limites, et l'introduction de la solution **Market Bridge**.

1.4.1 Étude de l'existant

Dans le domaine du marketing d'influence, plusieurs plateformes locales et internationales ont été développées **pour faciliter la collaboration entre marques et créateurs de contenu**. Ces solutions offrent divers outils comme la recherche de profils, la gestion de campagnes ou encore l'analyse des performances, contribuant ainsi à la structuration du secteur. Cependant, elles présentent encore des lacunes en matière de personnalisation, de sécurité et d'intégration intelligente des services.

- **À l'échelle nationale**
 - ❖ **Connect Content (Tunisie)** est l'une des rares plateformes tunisiennes dédiées aux collaborations entre marques et influenceurs. Elle permet de poster des campagnes, consulter des profils et effectuer des paiements en ligne.
Mais ses fonctionnalités restent limitées : elle ne propose ni moteur de recommandation, ni espace de formation, ni système de modération automatisé.



Figure 2: Logo Connect Content

- **À l'échelle internationale**
- ❖ **Kolsquare (France)** est une référence en Europe. Elle offre des outils d'analyse, de gestion de campagnes et des rapports détaillés sur les performances.
Cependant, l'accès à ses fonctionnalités est payant et souvent réservé à de grandes marques, ce qui la rend peu accessible aux petites structures.



Figure 3: Logo Kolsquare

- ❖ **Upfluence (États-Unis)** centralise la gestion des campagnes et propose une base de données étendue de créateurs, avec un tableau de bord complet.
Toutefois, elle n'intègre pas de services adaptés aux spécificités locales comme un chatbot ou des outils de génération de documents, ce qui limite son intérêt pour un marché émergent comme celui de la Tunisie.



Figure 4: Logo Upfluence

1.4.2 Critique de l'existant

L'analyse des plateformes actuelles met en lumière leurs apports, mais aussi des limites qui justifient la création d'une solution plus adaptée. Le tableau suivant résume les points forts et les faiblesses de trois solutions représentatives :

Tableau 1: Tableau comparatif des plateformes de mise en relation marque-créateur

Plateforme	Points forts (+)	Points faibles (-)
Connect Content	Interface simple, adaptée au marché tunisien ; prise en charge du paiement local.	Manque de moteur de recommandation ; absence de chatbot et d'outils de formation.
Kolsquare	Outils d'analyse avancés ; base de données de qualité.	Coût élevé ; accès limité aux grandes marques ; peu d'outils interactifs.
Upfluence	Gestion complète des campagnes ; visibilité internationale.	Peu adapté aux besoins locaux ; pas de fonctions IA ni de workflows automatisés.

1.4.3 Solution proposée

L'analyse des plateformes existantes comme **Connect Content**, **Kolsquare** ou **Upfluence** a mis en évidence un manque d'adaptation au marché tunisien : peu de personnalisation, absence de recommandations intelligentes, sécurisation limitée des échanges et manque d'accompagnement automatisé.

C'est pour répondre à ces lacunes qu'a été conçue **Market Bridge**, une plateforme web innovante dédiée à la mise en relation entre **marques et créateurs de contenu** digital. Elle centralise, automatise et sécurise tout le processus de collaboration dans un espace fluide, structuré et intelligent.

La solution repose sur plusieurs piliers techniques et fonctionnels :

- **Moteur de recommandation IA**

Développé avec Flask, Spring Boot et MySQL, il suggère automatiquement les 5 créateurs les plus adaptés selon différents critères (type de contenu, expérience, pays, etc.).

- **Espace collaboratif**

Les marques publient leurs besoins et reçoivent des propositions, les créateurs soumettent leurs contenus et reçoivent des feedbacks, tandis qu'un super administrateur supervise et valide l'ensemble.

- **Chatbot IA local (offline)**

Conçu avec FastAPI et spaCy, il aide à générer automatiquement les formulaires PDF sans dépendre d'API externe, assurant rapidité, sécurité et autonomie.

- **Workflow complet de modération et feedbacks**

Chaque contenu est vérifié par un administrateur. En cas de refus, une notification est envoyée au créateur avec un motif clair pour amélioration.

- **Notifications en temps réel**

Intégrées via WebSocket, elles informent instantanément les utilisateurs des validations, refus, paiements ou nouvelles demandes.

- **Paiement sécurisé via Stripe Checkout**

Les paiements se font directement sur la plateforme, avec déclenchement automatique du téléchargement du contenu validé (sans watermark).

- **Sécurité des contenus**

Tous les fichiers approuvés sont protégés par un watermark dynamique, un blocage du clic droit et une visualisation sécurisée (iframe).

- **Tableau de bord interactif**

Chaque utilisateur suit ses activités (feedbacks, paiements, performances) via des graphiques dynamiques intégrés (Chart.js, Recharts).

En résumé, **Market Bridge** se positionne comme une **plateforme tout-en-un, intelligente**, sécurisée et simple d'utilisation, pensée pour moderniser et professionnaliser le marketing d'influence en Tunisie.

1.5 Méthodologie de travail

Pour mener à bien le développement du projet **Market Bridge**, une approche agile a été adoptée afin de garantir flexibilité, efficacité et qualité. Contrairement aux méthodes traditionnelles comme le cycle en V ou le modèle en cascade, l'agilité repose sur des cycles courts et itératifs, mieux adaptés aux projets évolutifs.

Elle permet d'intégrer rapidement les retours utilisateurs, de s'adapter aux imprévus et de maintenir un bon niveau de collaboration entre toutes les parties prenantes. Cette approche favorise la transparence, l'amélioration continue et place la satisfaction utilisateur au cœur du processus.



Figure 5: Schéma du Scrum

1.5.1 Choix du cadre Scrum

Pour organiser le projet, le cadre **Scrum** a été retenu. Il repose sur des sprints courts au cours desquels des versions fonctionnelles du produit sont livrées à chaque itération.

Ce choix s'explique par plusieurs avantages :

- **Organisation structurée** : des rôles bien définis (Product Owner, Scrum Master, Équipe de développement) et des rituels agiles réguliers assurent une gestion fluide.
- **Grande adaptabilité** : les sprints permettent d'ajuster les priorités selon les besoins du projet.
- **Amélioration continue** : les rétrospectives de sprint aident à corriger les points faibles et à renforcer l'efficacité de l'équipe.
- **Communication claire** : le travail d'équipe quotidien favorise une compréhension partagée des objectifs.

Grâce à cette méthode, l'équipe a pu assurer un bon rythme de travail, tout en respectant les attentes fonctionnelles et techniques du projet.

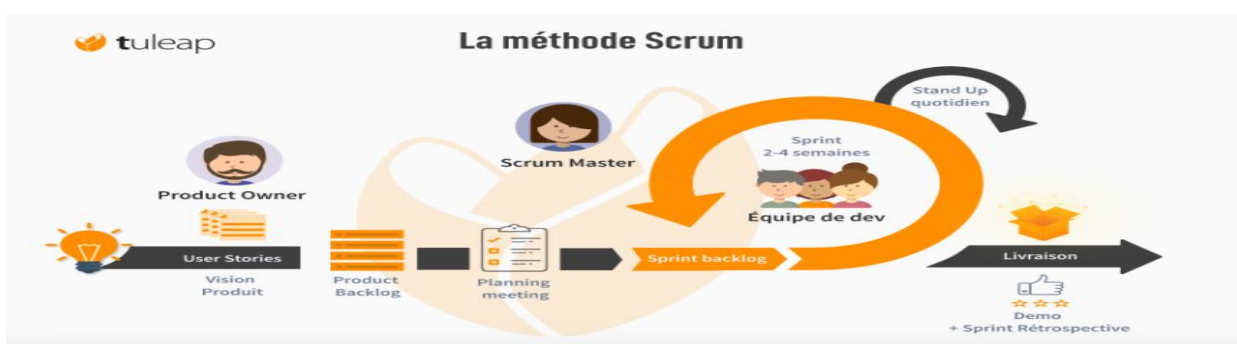


Figure 6: Explication méthode Scrum

1.5.2 Rôles dans la méthode Scrum

Scrum repose sur trois rôles clés :

- **Product Owner** : définit la vision produit, priorise les fonctionnalités et gère le backlog.
- **Scrum Master** : veille au bon respect du cadre Scrum, anime les réunions et facilite la communication.
- **Équipe de développement** : conçoit, développe, teste et livre les fonctionnalités prévues à chaque sprint.

Chez Bee Coders, **M. Ahmed Neffati** a assuré les rôles de Product Owner et de Scrum Master. Il a supervisé la planification, le suivi des tâches et la coordination entre les équipes, assurant ainsi la cohérence globale du projet.

Conclusion

Ce premier chapitre a posé les bases du projet **Market Bridge** en présentant son contexte, l'entreprise d'accueil **Bee Coders** et les enjeux du marketing d'influence en Tunisie.

Il a mis en évidence l'absence d'une plateforme centralisée, sécurisée et adaptée aux besoins locaux, tout en analysant les limites des solutions existantes.

La solution proposée se démarque par une approche intelligente, intégrant IA, automatisation et sécurité.

L'adoption de la méthode **Scrum** permet par ailleurs un développement itératif, structuré et collaboratif.

Le prochain chapitre sera consacré à l'analyse détaillée des besoins fonctionnels et techniques de la plateforme.

Chapitre 2 : Analyse et Spécification des Besoins

Introduction

Ce deuxième chapitre aborde l'analyse et la spécification des besoins du projet **Market Bridge**, une étape clé du cycle de développement. Elle permet de traduire la problématique identifiée précédemment en exigences fonctionnelles et techniques claires.

L'objectif est de définir précisément le comportement attendu du système, d'identifier les différents acteurs de la plateforme et de modéliser leurs interactions pour garantir une conception cohérente.

Nous commencerons par présenter les acteurs et leurs rôles, puis les besoins fonctionnels et non fonctionnels (performance, sécurité, fiabilité).

Seront ensuite introduits : le diagramme global des cas d'utilisation UML, le backlog produit selon **Scrum**, ainsi que le diagramme de classes illustrant la structure logique du système.

Enfin, nous décrirons l'environnement technique retenu **Angular** pour le Frontend, **Spring Boot** pour le Backend, et **des modules IA développés avec Flask et FastAPI**.

2.1 Identification des acteurs de l'application

La plateforme **Market Bridge** repose sur l'interaction de trois acteurs principaux, chacun disposant de rôles bien définis et de fonctionnalités spécifiques. Ces utilisateurs communiquent en temps réel via un système de notifications, de modération et d'automatisation intelligente.

2.1.1 Créateur de contenu

Le créateur est chargé de produire et de soumettre du contenu (images, vidéos) en réponse aux demandes des marques.

Il reçoit des notifications instantanées lorsqu'un projet lui est attribué par une marque ou le super administrateur.

Une fois son travail prêt, il le transmet via un système de modération qui peut entraîner une validation, un refus ou une demande de révision.

Il dispose également d'un profil personnel interactif lui permettant de gérer ses informations et de s'inscrire aux formations disponibles sur la plateforme.

2.1.2 Marque

La marque est le client du système, à la recherche de créateurs pour ses campagnes digitales.

Elle peut :

- Publier une demande via un **formulaire** dédié ;
- Trouver des profils adaptés grâce à **un moteur de recommandation IA** (selon le projet, le secteur, ou les performances) ;
- Être assistée par un **chatbot** qui **génère automatiquement un PDF** de demande à soumettre ;
- **Suivre en temps réel les contenus reçus**, approuver ou refuser les propositions ;
- Noter les créateurs et régler les prestations via **Stripe Checkout**.

Un espace de notifications lui permet de rester informée en direct des évolutions liées aux projets, paiements et validations.

2.1.3 Super Administrateur

Il joue un rôle clé dans la gestion globale de la plateforme. Ses missions incluent :

- Valider les formulaires des marques ;
- Assigner les demandes aux créateurs les plus pertinents ;
- Gérer la modération des contenus (validation, refus, ou retour pour amélioration) ;
- Superviser les feedbacks et relancer les créateurs si nécessaire ;
- Suivre en temps réel l'ensemble des flux via un tableau de bord interactif ;
- Consulter des statistiques globales (demandes, retours, formations suivies, etc.) pour assurer une vue d'ensemble fluide du système.

2.2 Étude des besoins

2.2.1 Besoins fonctionnels

Cette section décrit les principales fonctionnalités attendues de la plateforme **Market Bridge**, selon les rôles des trois types d'utilisateurs : **Marque**, **Créateur** et **Super Administrateur**. Ces besoins ont été définis à partir du diagramme de cas d'utilisation global et des objectifs établis en phase d'analyse.

❖ Marque

- **Soumettre une demande** : compléter un formulaire décrivant ses besoins (visuel, vidéo, etc.), avec l'aide d'un chatbot IA capable de générer un PDF automatiquement.

- **Rechercher un créateur** : accéder à un moteur de recommandation IA qui propose les profils les plus adaptés selon plusieurs critères (pack, type de contenu, performance, etc.).
- **Recevoir des notifications** : être informée en temps réel des propositions, validations ou retours via un système de notifications dynamiques.
- **Évaluer un contenu** : noter ou refuser un contenu avec justification ; en cas de refus, le feedback est transmis à l'administrateur.
- **Valider et payer** : accepter une proposition et effectuer un paiement sécurisé via **Stripe Checkout** pour débloquer le contenu sans filigrane.
- **Télécharger un contenu** : obtenir le fichier validé (sans watermark) et le visualiser directement sur la plateforme.
- **Gérer son profil** : modifier ses informations, logo, mot de passe, et consulter son historique.

❖ Créateur

- **Recevoir des propositions** : être notifié lorsqu'un projet lui est assigné par une marque ou l'administrateur.
- **Uploader un contenu** : soumettre une création (image ou vidéo), qui sera automatiquement intégrée au processus de modération.
- **Accéder aux formations** : consulter la liste des packs de formation disponibles et envoyer une demande d'inscription.
- **Gérer son profil** : mettre à jour ses informations, mot de passe, et avatar.

❖ Super Administrateur

- **Gérer les demandes** : consulter les formulaires soumis, valider ou refuser les propositions, et assigner les projets aux créateurs adaptés.
- **Modérer les contenus** : vérifier les créations reçues et décider de leur validation, refus ou amélioration.
- **Superviser les feedbacks** : analyser les retours des marques et renvoyer une demande d'amélioration au créateur si besoin.
- **Gérer les packs** : modifier ou supprimer les packs de services et de formations.

- **Administrer les comptes** : activer, désactiver ou supprimer les comptes des utilisateurs.
- **Suivre les statistiques** : accéder à un tableau de bord global (utilisateurs, répartition des packs, comparatif marque/créateur) pour évaluer l'activité de la plateforme.

2.2.2 Besoins non fonctionnels

Pour garantir la performance, la fiabilité et l'évolutivité de la plateforme **Market Bridge**, plusieurs exigences non fonctionnelles ont été prises en compte.

❖ Sécurité:

L'accès est sécurisé par **Spring Security**, l'authentification JWT et une gestion fine des rôles (Super Admin, Marque, Créateur).

Les mots de passe sont chiffrés avec **BCrypt**, et la **double authentification (2FA)** renforce la protection des comptes.

❖ Fiabilité:

L'infrastructure conteneurisée avec **Docker** et automatisée via **Jenkins** assure une haute disponibilité et un redémarrage automatique des services sans perte de données.

❖ Maintenabilité:

L'architecture du code est modulaire, facilitant les évolutions.

L'intégration continue (CI/CD) avec **Jenkins**, combinée à **SonarQube**, permet un suivi constant de la qualité et de la stabilité du projet.

❖ Scalabilité:

La plateforme est pensée pour évoluer sans perte de performance, même en cas d'augmentation du nombre d'utilisateurs ou de contenus.

2.3 Gestion du projet avec scrum

2.3.1 Backlog du produit

Dans le cadre de la conception de Market Bridge, une plateforme intelligente de mise en relation entre marques et créateurs de contenu, un backlog produit a été élaboré.

Ce document recense l'ensemble des fonctionnalités à développer, classées par priorité et centrées sur les besoins des utilisateurs.

Tableau 2: Backlog des user stories de la plateforme Market Bridge

ID	Module	User Story	Priorité
US01	Authentification & sécurité	En tant qu'utilisateur, je veux m'authentifier de manière sécurisée via JWT et 2FA afin d'accéder à mon espace personnel.	Haute
US02	Gestion de compte	En tant qu'utilisateur (marque ou créateur), je veux gérer et modifier mon profil pour actualiser mes informations.	Moyenne
US03	Marque – Formulaire de demande	En tant que marque, je veux choisir un pack service et soumettre un formulaire pour exprimer mon besoin.	Haute
US04	Marque – Recommandation IA	En tant que marque, je veux accéder au moteur de recommandation IA afin de trouver les créateurs les plus adaptés à ma campagne.	Haute
US05	Marque – Chatbot IA	En tant que marque, je veux interagir avec un chatbot intelligent afin d'obtenir automatiquement un PDF récapitulatif de ma demande.	Moyenne
US06	Marque – Évaluation et feedback	En tant que marque, je veux évaluer un contenu reçu, accepter ou refuser une proposition, et fournir un feedback au créateur.	Haute
US07	Marque – Paiement Stripe	En tant que marque, je veux effectuer un paiement sécurisé via Stripe afin de débloquent le contenu validé sans watermark.	Haute
US08	Créateur – Notifications en temps réel	En tant que créateur, je veux recevoir en temps réel les propositions de missions ou d'amélioration pour réagir rapidement.	Haute

US09	Créateur – Upload de contenu	En tant que créateur, je veux téléverser mon contenu (image, vidéo, PDF) pour qu'il soit modéré et validé par le super administrateur.	Haute
US10	Créateur – Formations et packs	En tant que créateur, je veux consulter les packs de formation, m'y inscrire et suivre ma progression.	Moyenne
US11	Créateur – Amélioration de contenu	En tant que créateur, je veux soumettre une nouvelle version d'un contenu suite à une demande d'amélioration.	Moyenne
US12	Super administrateur – Gestion des utilisateurs	En tant que super administrateur, je veux gérer les utilisateurs (créateurs et marques) pour maintenir la fiabilité de la plateforme.	Haute
US13	Super administrateur – Modération	En tant que super administrateur, je veux consulter et valider les contenus modérés afin d'en assurer la conformité.	Haute
US15	Super administrateur – Gestion des packs	En tant que super administrateur, je veux créer, modifier et supprimer les packs de formation ou de services.	Moyenne
US16	Super administrateur – Feedbacks	En tant que super administrateur, je veux superviser les feedbacks pour suivre les échanges entre marques et créateurs.	Moyenne
US17	Super administrateur – Statistiques et tableau de bord	En tant que super administrateur, je veux consulter les statistiques globales (nombre d'utilisateurs, répartition par pack, taux de participation) via un tableau de bord.	Moyenne
US18	Notifications & temps réel	En tant qu'utilisateur, je veux recevoir des notifications instantanées lors des actions importantes (validation, paiement, amélioration).	Haute

2.5 Diagramme de classes

Le diagramme de classes décrit l'architecture interne de *Market Bridge*.

Il présente les entités principales, leurs attributs et les relations qui les unissent, permettant ainsi de visualiser la structure logique des données et de mieux comprendre le modèle métier de la plateforme.

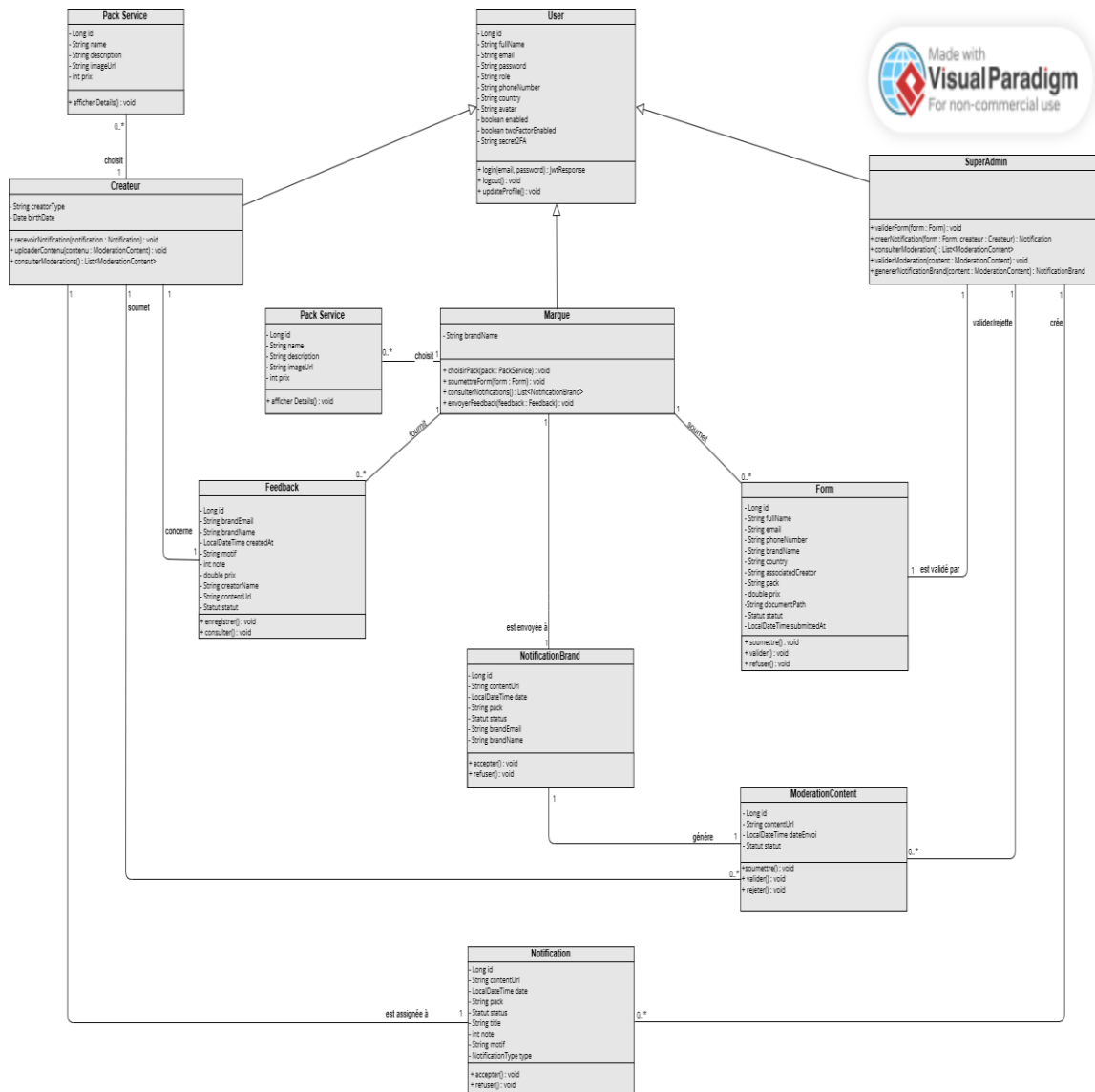


Figure 8: Diagramme de classes global de la plateforme Market Bridge

2.6 Environnement de travail dans ce stage

2.6.1 Frameworks et langages utilisés

- **Spring Boot:** utilisé pour le back-end. Il gère la logique métier, la sécurité (JWT, Spring Security) et la connexion à la base **MySQL**.



Figure 9: Logo Spring Boot

- **Angular** (*TypeScript*) : framework front-end pour créer une interface utilisateur réactive connectée aux API REST.



Figure 10: Logo Angular

- **Flask** (*Python*) : utilisé pour le développement du chatbot et la génération automatique de fichiers PDF.



Figure 11: Logo Flask

- **FastAPI** (*Python*) : exploité pour le moteur de recommandation IA, permettant le matching intelligent entre marques et créateurs.



Figure 12: Logo FastAPI

- **MySQL** : base de données relationnelle stockant les utilisateurs, contenus, formulaires et feedbacks.



Figure 13: Logo MySQL

2.6.2 Environnement logiciel

Pour assurer le bon fonctionnement, la portabilité et le suivi du projet, plusieurs outils ont été utilisés :

- **Docker & Docker Compose** : pour conteneuriser les services (front, back, base de données) et faciliter le déploiement.



Figure 14: Logo Docker

- **Postman** : pour tester et valider les appels API (authentification, paiement, envoi de contenu...).



Figure 15: Logo Postman

- **GitHub** : gestion de version et collaboration, avec intégration du pipeline CI/CD.



Figure 16: Logo GitHub

- **Jira** : planification des sprints, gestion des user stories et suivi des tâches Scrum.



Figure 17: Logo Jira

- **Visual Paradigm** : création des diagrammes UML (cas d'utilisation, classes).



Figure 18: Logo Visual Paradigm

- **Jenkins** : automatisation des builds, des tests (JUnit) et des déploiements via Docker.



Figure 19: Logo Jenkins

- **SonarQube** : analyse continue de la qualité du code pour détecter erreurs, duplications et failles.



Figure 20: Logo SonarQube

- **Nexus** : gestion centralisée des dépendances et artefacts générés à chaque build.



Figure 21: Logo Nexus

- **Prometheus & Grafana** : outils de monitoring et visualisation en temps réel des performances du système.



Figure 22: Logo Prometheus & Grafana

- **IntelliJ IDEA** : IDE principal pour le développement Spring Boot.



Figure 23: Logo IntelliJ IDEA

- **Visual Studio Code** : utilisé pour Angular, Flask et FastAPI, avec extensions pour Python et debug.



Figure 24: Logo Visual Studio Code

2.6.3 Environnement matériel

L'architecture matérielle utilisée pour la réalisation et le déploiement du projet *Market Bridge* est présentée dans le tableau suivant :

Tableau 3: Environnement matériel

Élément	Détail
Marque / Modèle	ASUS VivoBook M1603QA
Processeur	AMD Ryzen 7 5800H with Radeon Graphics (3.20 GHz)
Mémoire RAM	16 Go (15,4 Go utilisable)
Carte graphique	AMD Radeon™ Graphics
Architecture système	Système d'exploitation 64 bits, processeur x64
Système d'exploitation	Windows 11 Famille

2.7 Architecture globale

D'un point de vue technique, la plateforme Market Bridge s'appuie sur une structure en deux volets complémentaires :

L'architecture logique, qui définit l'organisation fonctionnelle des composants applicatifs,

L'architecture physique, qui présente les éléments concrets nécessaires au déploiement ainsi que les interactions entre eux.

Le modèle architectural suivi est de type N-tiers, garantissant une séparation nette entre la couche de présentation, la couche métier, la couche de données, ainsi que les services externes. Ce découpage permet d'assurer une maintenabilité optimale, une sécurité renforcée, et une évolutivité facilitée de la plateforme.

2.7.1 Architecture physique

La plateforme repose sur une stack conteneurisée avec Docker Compose, déployée sur une machine hôte Windows et isolée dans un réseau Docker dédié.

L'utilisateur final interagit avec la plateforme à travers un navigateur web, qui accède à l'application via le frontend Angular. Ce dernier communique directement avec le backend monolithique Spring Boot, lequel expose des API REST pour gérer la logique serveur.

Le backend centralise :

- La logique métier complète,
- Les aspects de sécurité (Spring Security, authentification JWT, gestion des rôles, 2FA),
- L'accès à la base de données MySQL, elle-même hébergée dans un conteneur Docker avec un volume persistant pour garantir la sauvegarde durable des données.

Les fonctionnalités d'intelligence artificielle sont externalisées dans des microservices Python, également déployés sous forme de conteneurs et accessibles via des API REST :

- Flask gère le chatbot ainsi que la génération automatique de documents PDF,
- FastAPI héberge le moteur de recommandation IA, qui met en relation les marques et les créateurs selon des critères comme le type de contenu, l'expérience, ou le pack sélectionné.

L'ensemble des échanges entre composants utilise le protocole HTTP et des requêtes REST/JSON, garantissant des communications efficaces et légères.

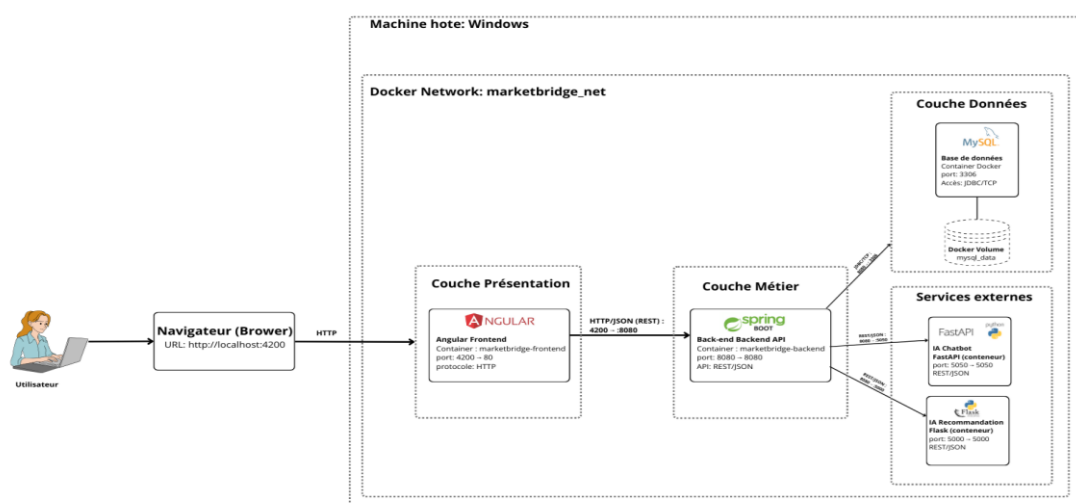


Figure 25: Architecture physique de la plateforme Market Bridge

2.7.2 Architecture logique

L'architecture logique de Market Bridge est construite sur un modèle modulaire MVC (Model – View – Controller). Ce choix favorise une structure de code claire, moderne et facile à maintenir.

- Couche Présentation (Angular)

Cette couche gère l'interface utilisateur, la navigation, les formulaires, ainsi que l'affichage des contenus.

Elle communique avec le backend via des appels REST HTTP sécurisés.

- Couche Métier (Spring Boot)
 - Elle regroupe toutes les règles métier de l'application :
 - Authentification et autorisation (JWT, 2FA),
 - Gestion des utilisateurs et des rôles,
 - Modération des contenus,
 - Paiements via Stripe,
 - Coordination des échanges avec les services d'IA.

L'accès aux données est réalisé via JPA/Hibernate, selon un schéma MVC : Controllers, Services, Entities.

- Services IA externes (Flask & FastAPI)
 - Flask : gère les échanges du chatbot et la génération de fichiers PDF.
 - FastAPI : implémente le moteur IA de recommandation pour associer créateurs et marques selon des critères ciblés.
- Couche Données (MySQL)

Elle centralise toutes les données clés de la plateforme : utilisateurs, contenus, formulaires, paiements, notifications et historiques.

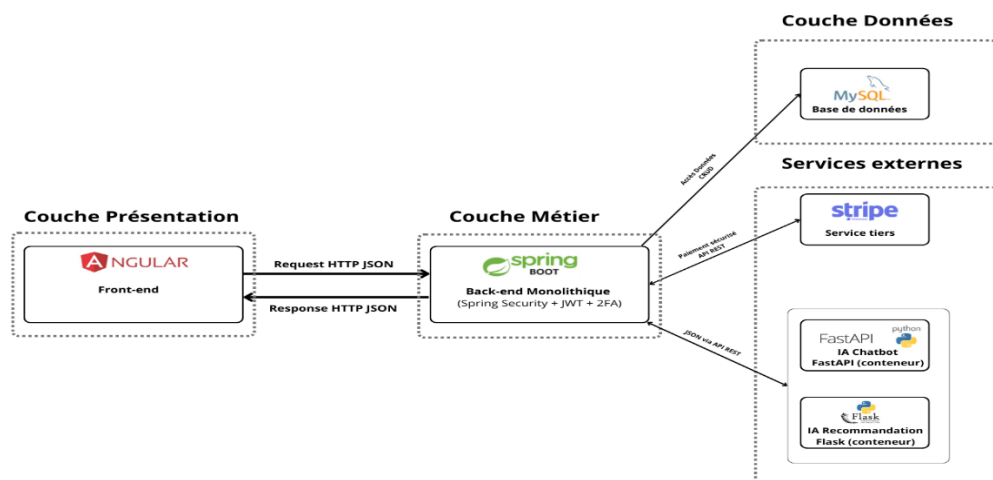


Figure 26: Architecture logique de la plateforme Market Bridge

2.8 Sprint Backlog Général de la plateforme Market Bridge

Le Sprint Backlog général regroupe l'ensemble des fonctionnalités planifiées tout au long du développement de la plateforme *Market Bridge*.

Chaque sprint correspond à une étape clé du projet, suivant la méthodologie **Scrum**, permettant une progression incrémentale et contrôlée jusqu'à la version finale du produit.

Tableau 4: Sprint Backlog Général de la plateforme Market Bridge

Sprint	Gestion / Fonctionnalités réalisées
Sprint 0	Analyse des besoins, rédaction du cahier des charges, étude de faisabilité, définition des rôles utilisateurs (SuperAdmin, Marque, Créateur), conception UML et choix des technologies (Angular, Spring Boot, MySQL, Flask, FastAPI, Docker).
Sprint 1	Mise en place du système d'inscription et d'authentification (JWT, BCrypt, Spring Security), gestion des rôles et permissions, intégration du 2FA (code QR et validation).
Sprint 2	Développement du module de soumission de formulaires côté Marque, gestion des demandes (Brand Request), validation par le SuperAdmin et génération des notifications correspondantes.

Sprint 3	Création du module de modération : upload de contenu par le créateur (image, vidéo), validation du contenu par le SuperAdmin, et envoi de notifications à la Marque via <i>NotificationBrand</i> .
Sprint 4	Implémentation du module de feedback et de suivi : la Marque peut accepter ou refuser une proposition, noter le créateur et envoyer un motif de refus. Ajout de la gestion des améliorations (nouvelle notification type <i>AMÉLIORATION</i>).
Sprint 5	Intégration du module de paiement sécurisé avec Stripe Checkout : validation automatique du feedback « PAYÉ », téléchargement automatique du contenu sans watermark après paiement.
Sprint 6	Développement du module d'intelligence artificielle : moteur de recommandation (FastAPI) pour le matching Marque ↔ Créateur, et chatbot intelligent (Flask) pour la génération automatique de PDF d'assistance.
Sprint 7	Implémentation du module de profil pour les utilisateurs (Marques et Créateurs) : mise à jour dynamique des informations, gestion d'avatar, sécurité JWT et contrôle des accès.
Sprint 8	Mise en place du module de statistiques et tableau de bord: affichage du nombre de marques, créateurs (Pro / Formé), répartition par pack, taux de validation, et suivi global des activités.
Sprint 9	Phase DevOps : conteneurisation complète avec Docker, intégration continue et déploiement via Jenkins, analyse de qualité avec SonarQube, gestion des artefacts sur Nexus, monitoring et alerting via Prometheus & Grafana. Tests finaux et optimisation des performances.

Conclusion

Ce deuxième chapitre a posé les bases techniques et fonctionnelles du projet *Market Bridge*, indispensables à son développement agile.

Les trois acteurs clés **SuperAdmin**, **Marque** et **Créateur** ont été identifiés, avec une définition claire de leurs rôles et interactions via les cas d'utilisation.

Les besoins fonctionnels et non fonctionnels ont été formulés pour garantir performance, sécurité et évolutivité.

Nous avons également présenté le backlog produit, les diagrammes UML (cas d'utilisation, classes) et l'environnement de travail, comprenant des technologies modernes comme **Angular, Spring Boot, MySQL, Flask, FastAPI, Docker, Jenkins, SonarQube, Nexus, Prometheus et Grafana.**

Enfin, l'architecture physique et logique de la plateforme a été détaillée, illustrant les échanges entre le frontend, le backend, les microservices IA, la base de données et l'infrastructure DevOps.

L'ensemble de ces éléments offre une base solide pour aborder la prochaine étape : la conception détaillée et le développement des modules de *Market Bridge*.

CHAPITRE 3 : ÉTUDE ET RÉALISATION DU PREMIER SPRINT

Introduction

Ce chapitre marque le début du développement concret de *Market Bridge* à travers la mise en œuvre du premier sprint.

Cette première itération vise à établir les fondations techniques et fonctionnelles de la plateforme, notamment les fonctionnalités d'inscription, de connexion sécurisée (JWT), de double authentification (2FA), la gestion des rôles (SuperAdmin, Marque, Créateur), ainsi que l'administration des profils utilisateurs.

Ces éléments sont essentiels pour garantir la sécurité, la fiabilité et le bon accès aux différents espaces de la plateforme.

La phase inclut également la création de la base de données utilisateurs, la configuration de **Spring Security**, l'intégration du filtre **JWT**, ainsi que la gestion des codes de double authentification via **Google Authenticator**.

Le chapitre commence par la présentation du sprint backlog, puis détaille les spécifications fonctionnelles implémentées, appuyées par des diagrammes UML et des descriptions précises. L'objectif est de construire une base solide et sécurisée pour les sprints suivants, en assurant dès le départ une cohérence entre les aspects techniques et fonctionnels.

3.1 Sprint Backlog

Ce premier sprint a été dédié à la mise en place des fonctionnalités de sécurité essentielles, permettant aux utilisateurs marques, créateurs et administrateur de s'inscrire, se connecter et accéder à la plateforme selon leurs rôles.

Il a également permis de configurer l'authentification via **JWT** et d'activer le mécanisme de **double authentification (2FA)**.

Tableau 5: Sprint Backlog du premier sprint

ID	User Story	Priorité
US01	En tant qu'utilisateur (Marque ou Créateur), je veux pouvoir créer un compte afin d'accéder à la plateforme.	Haute

US02	En tant qu'utilisateur, je veux pouvoir me connecter de manière sécurisée via JWT.	Haute
US03	En tant qu'utilisateur, je veux pouvoir activer la double authentification (2FA) pour renforcer la sécurité.	Haute
US04	En tant que Superadmin, je veux pouvoir activer, bloquer ou supprimer un utilisateur.	Haute
US05	En tant qu'utilisateur, je veux pouvoir consulter et mettre à jour mon profil personnel (nom, e-mail, téléphone, avatar, mot de passe).	Moyenne

3.2 Spécification fonctionnelle du premier sprint

Cette section présente les spécifications fonctionnelles développées lors du premier sprint de *Market Bridge*, axé sur la **gestion des utilisateurs** et la **sécurisation de l'accès**.

Elle définit avec clarté :

- Les principales fonctionnalités à développer,
- Les interactions entre les utilisateurs,
- Ainsi que les comportements attendus du système.

Les spécifications sont exprimées à travers :

- Un **diagramme UML** illustrant les cas d'utilisation,
- et des **descriptions textuelles** précisant les enchaînements fonctionnels et règles métier.

3.2.1 Diagramme de cas d'utilisation du premier sprint

Le diagramme ci-dessous synthétise les principales fonctionnalités mises en œuvre dans cette première itération :

- Inscription des utilisateurs (*Marque, Créateur*),
- Connexion sécurisée via **JWT**,
- Vérification du code de **double authentification (2FA)** à la première connexion,
- Gestion des comptes par le **Superadmin** (activation, blocage, suppression).

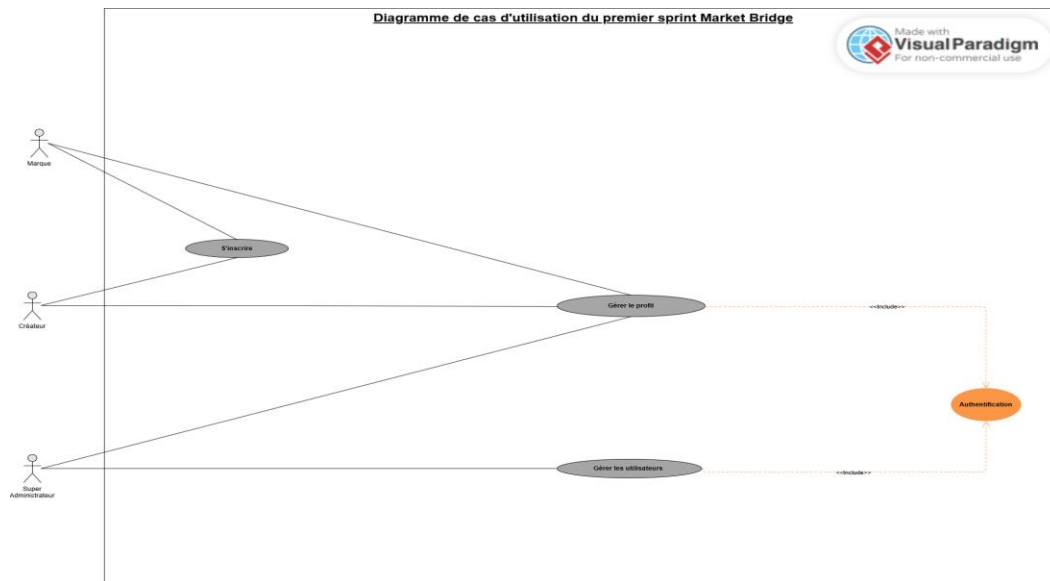


Figure 27: Diagramme de cas d'utilisation du premier sprint (Market Bridge)

3.2.2 Description textuelle des cas d'utilisation du premier sprint

Afin de mieux comprendre le diagramme précédent, les cas d'utilisation essentiels de cette itération sont décrits textuellement.

Chaque description précise les **objectifs**, les **acteurs impliqués**, les **préconditions**, le **scénario nominal** et les **post-conditions attendues**.

3.2.2.1 Cas d'utilisation : Créer un compte

Tableau 6: Description textuelle du cas d'utilisation - Créer un compte

Élément	Description
Acteurs	Utilisateur anonyme (non connecté), Système d'authentification
Pré-condition	L'utilisateur accède à la page d'inscription et n'est pas encore connecté.
Post-condition	Le compte est enregistré dans la base de données, un rôle par défaut (<i>Marque</i> ou <i>Créateur</i>) est attribué, et un QR Code 2FA est généré.

Scénario nominal	1. L'utilisateur ouvre la page <i>Inscription</i>. 2. Il choisit son type de compte (<i>Marque</i> ou <i>Créateur</i>). 3. Il saisit ses informations personnelles (nom complet, email, pays, téléphone, mot de passe et confirmation). 4. Il valide les conditions générales d'utilisation. 5. Le système enregistre les données et crée le compte. 6. Une fois inscrit, un QR Code Google Authenticator s'affiche pour activer la double authentification (optionnelle). 7. L'utilisateur peut scanner le QR Code pour sécuriser son compte.
Scénario d'exception	1. Si l'email existe déjà → « <i>Cette adresse e-mail est déjà utilisée.</i> » 2. Si le mot de passe est trop faible → « <i>Le mot de passe doit contenir au moins 8 caractères.</i> » 3. Si la confirmation du mot de passe ne correspond pas → « <i>Les mots de passe ne correspondent pas.</i> »

3.2.2.2 Cas d'utilisation : Se connecter

Tableau 7: Description textuelle du cas d'utilisation - Se connecter

Élément	Description
Acteurs	Utilisateur enregistré (Marque ou Créateur), Système d'authentification
Pré-condition	L'utilisateur dispose d'un compte existant avec un email et un mot de passe valides.
Post-condition	L'utilisateur est connecté et reçoit un token JWT ; si 2FA est activé, il doit saisir le code avant d'accéder à son espace.
Scénario nominal	1. L'utilisateur accède à la page <i>Connexion</i>. 2. Il saisit son email et son mot de passe. 3. Le système vérifie les identifiants. 4. Si la double authentification est activée, un champ <i>Code 2FA</i> s'affiche. 5. L'utilisateur saisit le code à 6 chiffres généré par <i>Google Authenticator</i>.

	6. Le système valide le code et génère un token JWT. 7. L'utilisateur est redirigé vers son tableau de bord selon son rôle (<i>Marque</i> ou <i>Créateur</i>).
Scénario d'exception	1. Si l'email n'existe pas → « <i>Aucun compte trouvé avec cet email.</i> » 2. Si le mot de passe est incorrect → « <i>Mot de passe invalide.</i> » 3. Si le code 2FA est erroné ou expiré → « <i>Code invalide, veuillez réessayer.</i> » 4. En cas de plusieurs échecs consécutifs, le système bloque temporairement la connexion.

3.2.2.3 Cas d'utilisation : Vérifier le code 2FA

Tableau 8: Description textuelle du cas d'utilisation - Vérifier le code 2FA

Élément	Description
Acteurs	Utilisateur connecté, Système de sécurité
Pré-condition	L'utilisateur a activé la double authentification (QR Code scanné via <i>Google Authenticator</i>).
Post-condition	Le code TOTP est validé et l'accès complet à la plateforme est accordé.
Scénario nominal	1. Après la première connexion, le système affiche le champ <i>Code 2FA</i>. 2. L'utilisateur ouvre <i>Google Authenticator</i> et saisit le code à 6 chiffres généré. 3. Le système vérifie la validité du code (temps réel). 4. Si le code est valide, l'accès complet à la plateforme est accordé.

Scénario d'exception	1. Si le code est expiré → « <i>Code expiré, veuillez réessayer.</i> » 2. Si le code est incorrect → « <i>Code invalide.</i> » 3. Si 2FA n'est pas activé, la vérification est ignorée.
-----------------------------	---

3.2.2.4 Cas d'utilisation : Gérer les utilisateurs (Superadmin)

Tableau 9: Description textuelle du cas d'utilisation - Gérer les utilisateurs

Élément	Description
Acteurs	Superadmin, Système de gestion des comptes
Pré-condition	Le Superadmin est connecté avec un accès d'administration.
Post-condition	L'état de l'utilisateur est mis à jour (activé, bloqué ou supprimé).
Scénario nominal	1. Le Superadmin accède à la page <i>Gestion des utilisateurs</i> . 2. Il visualise la liste complète des comptes (<i>Marques et Créateurs</i>). 3. Il peut : – Activer un compte inactif – Bloquer un compte suspect – Supprimer un utilisateur définitivement. 4. Le système applique l'action et affiche un message de confirmation.
Scénario d'exception	1. Si l'utilisateur sélectionné n'existe plus → « <i>Utilisateur introuvable.</i> » 2. En cas d'erreur de base de données → « <i>Échec de la mise à jour.</i> » 3. Si le Superadmin tente de bloquer un autre administrateur → « <i>Action non autorisée.</i> »

3.2.2.5 Cas d'utilisation : Gérer le profil (Marque ou Créateur)

Tableau 10: Description textuelle du cas d'utilisation - Gérer le profil (Marque ou Créateur)

Élément	Description
Acteurs	Utilisateur connecté (Marque ou Créateur), Système de gestion du profil

Pré-condition	L'utilisateur est authentifié via JWT et dispose d'un accès valide à son tableau de bord.
Post-condition	Les informations du profil sont mises à jour dans la base de données et affichées dans l'interface utilisateur.
Scénario nominal	1. L'utilisateur accède à la section "Mon profil" depuis le tableau de bord. 2. Le système récupère les informations de l'utilisateur (nom, e-mail, téléphone, pays, avatar). 3. L'utilisateur peut modifier les champs souhaités et choisir un nouvel avatar. 4. Il clique sur "Enregistrer les modifications". 5. Le frontend envoie une requête PUT /brands/update ou /creators/update avec les nouvelles données et le token JWT dans le header. 6. Le backend vérifie le token, met à jour les données dans MySQL et encode le mot de passe si modifié. 7. Le système retourne une réponse "200 OK" et le frontend affiche un message de confirmation : "Profil mis à jour avec succès."
Scénario d'exception	1. Si un champ obligatoire est vide → "Veuillez remplir tous les champs." 2. Si le token JWT est invalide ou expiré → "Session expirée, veuillez-vous reconnecter." 3. Si le format d'image de l'avatar n'est pas autorisé → "Format d'image non supporté."

3.3 Conception du premier sprint

3.3.1 Diagrammes de séquence

Les diagrammes de séquence UML décrivent chronologiquement les échanges entre les acteurs et les composants du système.

Dans *Market Bridge*, ils illustrent les interactions entre le **frontend Angular**, le **backend Spring Boot** et la **base MySQL** pour la gestion des utilisateurs et des accès sécurisés.

3.3.1.1 Créer un compte (Créateur ou Marque)

Lorsqu'un utilisateur s'inscrit depuis l'interface Angular, il sélectionne son rôle (Marque ou Créateur) et renseigne ses informations.

La requête d'inscription est envoyée via POST /api/auth/register au backend.

Le **AuthController** valide les données, vérifie l'unicité de l'e-mail, encode le mot de passe avec **BCrypt** et, si le 2FA est activé, génère une clé sous forme de QR Code pour **Google Authenticator**.

En cas de succès, une réponse 201 est renvoyée avec un message de confirmation.

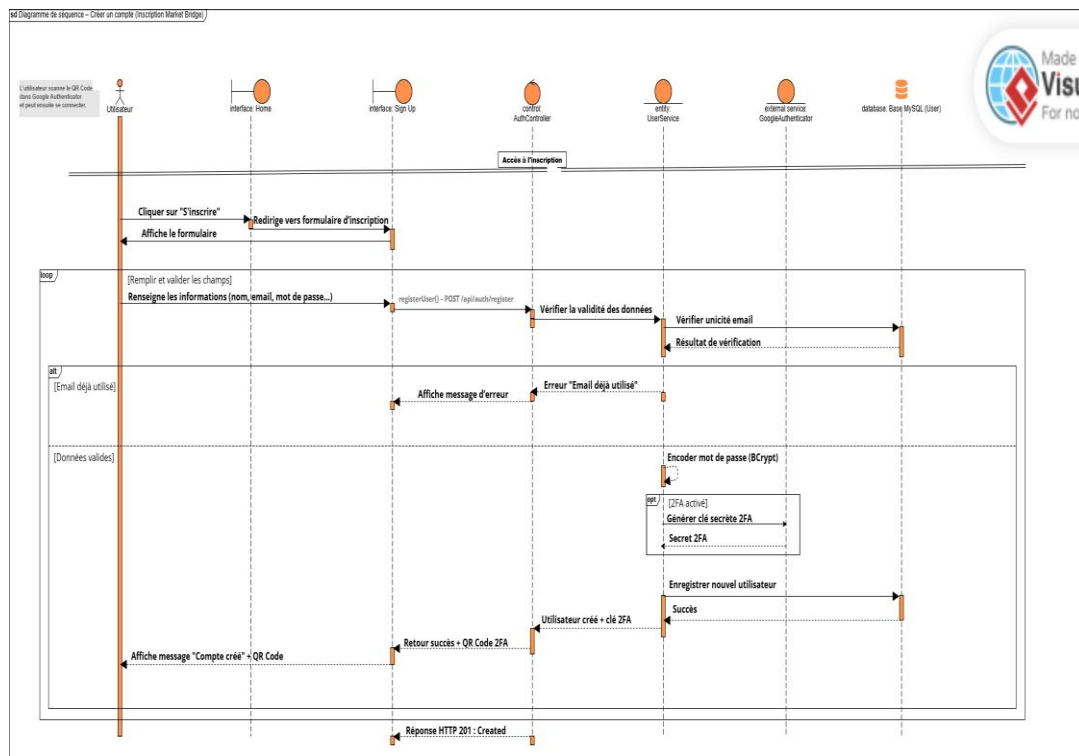


Figure 28: Diagramme de séquence : Créer un compte

3.3.1.2 Se connecter (JWT + 2FA)

À la connexion, l'utilisateur saisit son e-mail et mot de passe.

Une requête POST /api/auth/login est envoyée, vérifiée par le backend. Si le compte est actif et le 2FA activé, un code est exigé via **Google Authenticator**.

Une fois validé, le système génère un **Access Token** et un **Refresh Token JWT**.

En cas d'erreur (identifiants invalides, compte bloquée, code incorrect), un message d'erreur s'affiche côté frontend.

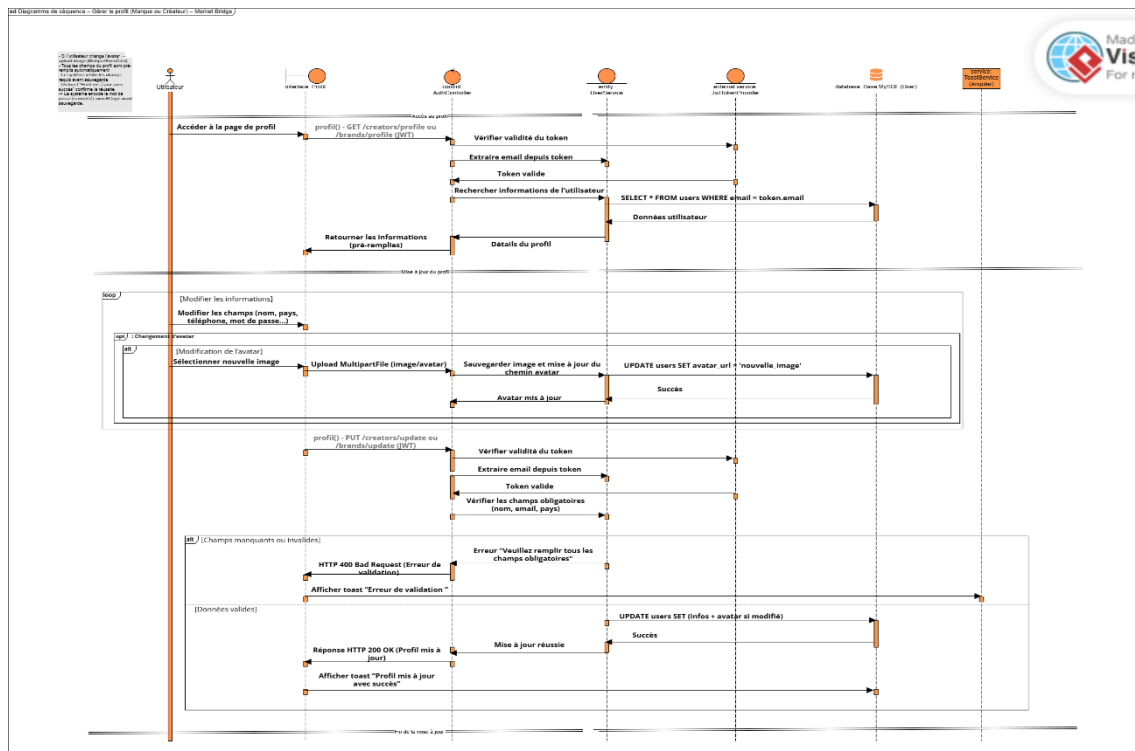


Figure 30: Diagramme de séquence : Gérer le profil

3.3.1.4 Gérer les utilisateurs (Super Administrateur)

Le Super Admin peut consulter et gérer tous les comptes via l'interface dédiée.

À l'ouverture, une requête GET /api/admin/users est envoyée avec le **token JWT**. Le backend vérifie que le rôle est bien **SUPERADMIN**, puis charge la liste depuis **MySQL**.

Le Super Admin peut ensuite activer, bloquer (PUT) ou supprimer (DELETE) un compte.

Chaque action affiche un message de confirmation et met à jour automatiquement la liste. En cas d'accès non autorisé, une erreur 403 est renvoyée.

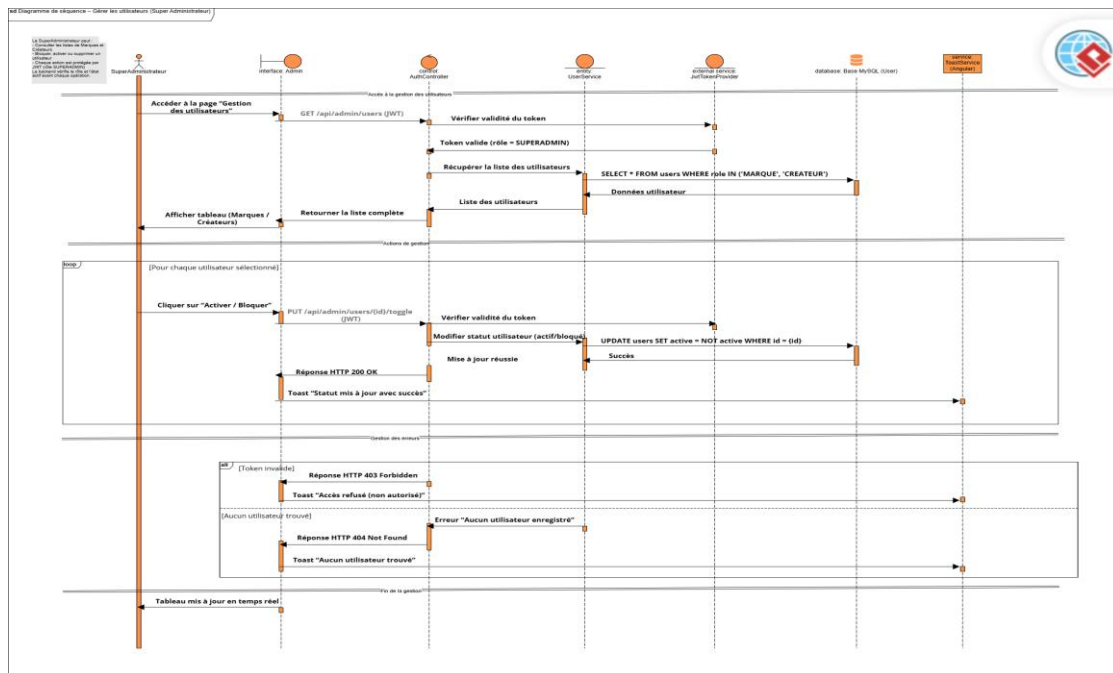


Figure 31: Diagramme de séquence : Gérer les utilisateurs

3.4 Réalisation du première sprint

Après avoir défini le périmètre fonctionnel du premier sprint et validé le backlog associé, la phase de réalisation a permis de mettre en œuvre les fonctionnalités fondamentales de gestion des utilisateurs sur la plateforme **Market Bridge**.

L'objectif principal était de permettre aux marques et aux créateurs de :

- créer un compte sécurisé,
- s'authentifier en toute sécurité via JWT et authentification à deux facteurs (2FA),
- et gérer leur profil personnel via une interface claire et fluide.

Le backend, développé avec **Spring Boot**, suit une architecture **MVC** et s'appuie sur un système d'authentification sécurisé basé sur **JWT**.

Lors de l'inscription, les informations de l'utilisateur sont vérifiées, le mot de passe est encodé avec **BCrypt**, et une clé secrète 2FA est générée et associée à un **QR Code Google Authenticator**.

Côté frontend, l'application Angular communique avec le backend via des **requêtes HTTP sécurisées** contenant le token JWT.

Toutes les données sont stockées dans une base **MySQL** hébergée dans un environnement **Docker**, garantissant la portabilité et l'isolation des services.

Cette phase a posé les bases essentielles du projet : une authentification robuste, une gestion des comptes fiable, et une personnalisation des profils utilisateurs selon leur rôle.

3.4.1 Interfaces réalisées

La figure ci-dessous montre l'interface d'inscription dédiée aux **marques**.

Elle permet à la marque de renseigner les informations nécessaires telles que le nom de l'entreprise, le pays, le contact, le mot de passe et l'acceptation des conditions d'utilisation.



The image shows a registration form titled "Connectez-vous à votre compte" (Connect to your account) for brands. The form is set against a dark background with orange and white text. It includes the following fields and options:

- Nom complet** (Full name): A text input field.
- Adresse e-mail** (Email address): A text input field.
- Mot de passe** (Password): A text input field with a visibility toggle icon.
- Confirmer le mot de passe** (Confirm password): A text input field with a visibility toggle icon.
- Nom de la marque (optionnel)** (Brand name, optional): A text input field.
- Indicatif du pays** (Country code): A dropdown menu currently showing "Tunisie (+216)".
- Numéro de téléphone** (Phone number): A text input field.
- Ajouter votre logo ici** (Add your logo here): A dashed orange box with an upload icon and the text "AJOUTER VOTRE LOGO ICI".
- ☐ **Activer la double authentification (2FA)** (Activate two-factor authentication).
- ☐ **J'accepte les termes et conditions** (I accept the terms and conditions).
- S'inscrire** (Sign up): A large orange button at the bottom.

Figure 32: Interface d'inscription Marque

La figure suivante illustre l'interface d'inscription réservée aux **créateurs de contenu**.

Elle comporte les champs nécessaires à la création d'un compte créateur, notamment les informations personnelles et les identifiants d'accès sécurisés.

Connectez-vous à votre compte

Nom complet

Le nom complet est requis.

Mot de passe

Mot de passe requis. Majuscule + chiffre requis.

Date de naissance

Date de naissance requise.

Pays

Numéro de téléphone

Numéro requis.

Adresse e-mail

Email requis.

Confirmer le mot de passe

Confirmation requise.

Type de créateur

Type requis.

Genre

Le genre est requis.



AJOUTER VOTRE AVATAR ICI

☒ Activer la double authentification (2FA)
 ☒ J'accepte les termes et conditions

Figure 33: Interface d'inscription Créateur

La figure suivante illustre l'interface d'inscription affichant le **QR Code Google Authenticator** pour la configuration de l'authentification à deux facteurs (2FA). L'utilisateur peut alors scanner ce code afin d'activer la sécurité renforcée de son compte.

Connectez-vous à votre compte

Nom complet

Mot de passe

Nom de la marque (optionnel)

Numéro de téléphone

Adresse e-mail

Confirmer le mot de passe

Indicatif du pays



AJOUTER VOTRE LOGO ICI

☒ Activer la double authentification (2FA)
 ☒ J'accepte les termes et conditions

Scannez ce QR Code avec Google Authenticator



J'ai scanné le QR Code

Connectez-vous à votre compte

Nom complet

Mot de passe

Nom de la marque (optionnel)

Numéro de téléphone

Adresse e-mail

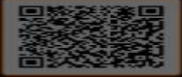
Confirmer le mot de passe

Indicatif du pays

Vérification 2FA

Entrez le code à 6 chiffres généré par Google Authenticator :

Scannez le QR Code avec Google Authenticator



J'ai scanné le QR Code

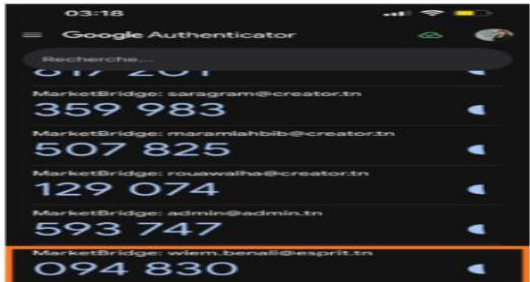


Figure 34: Interface d'inscription avec QR Code 2FA

La figure suivante présente l'interface de **connexion sécurisée avec 2FA**, où l'utilisateur saisit d'abord ses identifiants puis le code généré par Google Authenticator.

Ce processus garantit une authentification forte et empêche tout accès non autorisé.



Figure 35: Interface de connexion sécurisée (JWT + 2FA)

La figure suivante montre la version de la **connexion sécurisée sans 2FA**, lorsque la double authentification n'est pas activée.

L'utilisateur accède directement à son tableau de bord après validation du mot de passe chiffré.



Figure 36: Interface de connexion sécurisée sans 2FA

La figure suivante illustre l'interface de gestion du profil utilisateur (Marque ou Créateur).

L'utilisateur peut modifier ses informations personnelles, son avatar, ainsi que ses coordonnées, le tout via une interface fluide et intuitive.

Gérer mon **Profil Créateur**



Avatar actuel

Nom complet
Sara Gram

Numéro de téléphone
+21654102330

Genre
Femme

Adresse e-mail
saragram@creator.tn

Date de naissance
15/03/1998

Nouveau mot de passe

Mettre à jour

Figure 37: Interface de profil utilisateur (Créateur)

Gérer mon **Profil Marque**



Avatar actuel

Nom complet
Wiem benali

Nom de la marque
Wiem ESPRIT

Numéro de téléphone
+21623213381

Adresse e-mail
wiem.benali@esprit.tn

Sélectionner le pays
Tunisie

Nouveau mot de passe

Mettre à jour

Figure 38: Interface de profil utilisateur (Marque)

Les deux figures suivantes présentent les interfaces de **gestion des utilisateurs** accessibles exclusivement au **Super Administrateur**.

Elles permettent d'afficher les listes d'utilisateurs selon leur rôle et de gérer leurs statuts (activation, désactivation, suppression).

MarketBridge

Tableau de Bord

- Tableau de Bord
- Paramètres
- Statistiques
- Tableaux
- Utilisateurs
- Créateurs**
- Modérateurs
- Commentaires des Marques
- Commentaires des Créateurs
- Feedbacks

Paramètres du compte

- Paramètres
- Mot de passe
- Authentification 2FA
- Document

Liste des utilisateurs Créateurs

[Rafraîchir](#)

Avatar	Nom	Email	Pays	Téléphone	Date de naissance	Type	Statut	Actions
	Sara Gram	saragram@creator.tn	TN	+21654102330	1998-03-15	pro	Actif	
	Marim Labib	marimlabib@creator.tn	TN	+21665124553	1997-09-22	forme	Bloqué	
	Rioua Walid	riouawalid@creator.tn	TN	+21653778910	1999-01-05	pro	Actif	
	Adam ProVision	adamprovision@creator.tn	TN	+21667122544	1995-02-18	pro	Actif	
	Rania Art	raniazart@creator.tn	TN	+21622365412	1998-07-29	pro	Actif	
	Youssef Design	youssefdesign@creator.tn	FR	+3321452688	1996-12-09	pro	Actif	
	Mina Luxe	minaluxe@creator.tn	US	+12345678	1997-10-12	pro	Actif	
	Omar Studio	omarstudio@creator.tn	US	+187654321	1995-04-23	pro	Actif	
	Lina Beauty Pro	linabeautypro@creator.tn	DE	+4912457890	1999-06-03	pro	Actif	

Figure 39: Interface de gestion des utilisateurs – Créateurs

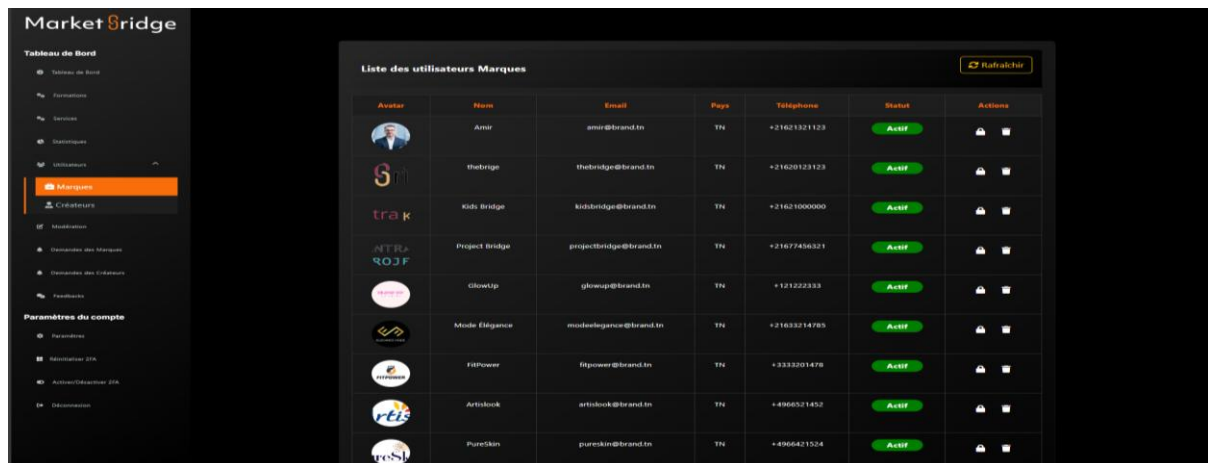


Figure 40: Interface de gestion des utilisateurs – Marques

3.5 Revue du premier sprint

Une réunion de revue a été tenue en fin de sprint avec le **Scrum Master** et le **Product Owner** pour évaluer les résultats et valider les fonctionnalités livrées.

❖ *Fonctionnalités développées*

- Création de comptes pour **Marques** et **Créateurs** avec mot de passe chiffré (BCrypt)
- Authentification via **JWT** et gestion des rôles (*SUPERADMIN*, *MARQUE*, *CREATEUR*)
- Intégration de la **double authentification (2FA)** via *Google Authenticator*
- Mise à jour des profils utilisateurs (données personnelles et avatar)
- Gestion des comptes utilisateurs par le **SuperAdmin** (activation, suppression...)

❖ *Validation & retours*

Le Product Owner a souligné :

- La fluidité du parcours d'inscription et de connexion
- La clarté dans la séparation des rôles
- La fiabilité du système de sécurité (JWT + 2FA)

❖ *Axes d'amélioration identifiés pour le sprint suivant*

- Optimiser la gestion des erreurs côté interface
- Améliorer la validation des formulaires Angular
- Structurer le futur module de **tableau de bord** et **notifications**

- Préparer une gestion plus fine des permissions administratives

3.6 Rétrospective du premier sprint

Toutes les tâches prévues ont été livrées dans les délais avec un bon niveau de qualité.

L'équipe a bien maîtrisé la stack utilisée (*Spring Boot, Angular, MySQL, JWT, Docker, 2FA*).

❖ *Défis rencontrés*

- Configuration du filtre JWT et gestion des rôles avec Spring Security
- Intégration du système 2FA
- Synchronisation des données entre Angular et Spring Boot
- Modélisation des relations entre utilisateurs, rôles et profils

Ces difficultés ont été surmontées grâce à une bonne coordination d'équipe et un suivi rigoureux des tâches via **Jira**.

Le sprint a ainsi permis de poser un socle technique stable pour la suite du projet.

Conclusion

Ce chapitre a présenté la réalisation du premier sprint de *Market Bridge*, centré sur la gestion sécurisée des comptes utilisateurs.

Les fonctionnalités clés (inscription, authentification, rôles, profils) ont été mises en place avec succès, constituant une base solide.

CHAPITRE 4 : ÉTUDE ET RÉALISATION DU DEUXIÈME SPRINT

Introduction

Ce chapitre présente la mise en œuvre du deuxième sprint du projet **Market Bridge**, une étape importante dans l'évolution fonctionnelle de la plateforme.

Après avoir mis en place les fondations techniques lors du premier sprint (authentification sécurisée, gestion des rôles), cette nouvelle itération introduit les **premières interactions actives entre les utilisateurs**, selon leurs rôles respectifs.

Les principales fonctionnalités livrées sont :

- **Créateurs** : consultation des formations, choix de pack, et candidature à un programme correspondant à leur profil.
- **Marques** : sélection d'un pack créateur, remplissage d'un formulaire détaillé, et envoi de demandes en ligne.
- **Super Administrateur** : suivi en temps réel des demandes via un système de notifications, et gestion des packs de formation et de service.

Ce sprint amorce la **mise en relation directe entre les acteurs** de la plateforme, posant les bases des prochaines étapes centrées sur la modération et la validation des contenus.

4.1 Sprint Backlog

Ce deuxième sprint s'est concentré sur la gestion des formulaires, la configuration des packs (formations et services), ainsi que le suivi en temps réel des demandes entre utilisateurs.

Les principaux utilisateurs stories développées sont résumés dans le tableau suivant :

Tableau 11: Sprint Backlog du deuxième sprint

ID	User Story	Priorité
US-01	En tant que créateur, je veux consulter les formations et postuler à un pack adapté à mon profil.	Haute
US-02	En tant que marque, je veux choisir un pack créateur, remplir un formulaire et soumettre une demande.	Haute

US-03	En tant que Super Admin, je veux consulter en temps réel les demandes soumises par marques et créateurs.	Haute
US-04	En tant que Super Admin, je veux gérer les packs pour les modifier ou les rendre visibles.	Haute
US-05	En tant que système, je veux notifier automatiquement le Super Admin lors de la soumission d'un formulaire.	Moyenne

Ce sprint a permis de connecter efficacement **le frontend Angular** au **backend Spring Boot**, en assurant un flux complet de gestion et de suivi des demandes.

La réactivité et la fluidité des interactions ont été renforcées grâce à l'intégration du système de **notifications en temps réel via WebSocket**.

4.2 Spécification fonctionnelle du deuxième sprint

Cette section présente les spécifications fonctionnelles issues du développement du deuxième sprint.

Elles s'appuient sur la modélisation UML afin de décrire les interactions entre les acteurs (**Marque, Créateur, Super Administrateur**) et les cas d'utilisation principaux :

- Choix et gestion des packs
- Postulation et soumission de formulaires
- Gestion des demandes et notifications.

4.2.1 Diagramme de cas d'utilisation du deuxième sprint

Le diagramme ci-dessous résume les interactions fonctionnelles entre les trois rôles.

Les créateurs et les marques peuvent sélectionner un pack et soumettre leur demande.

Le Super Admin reçoit ces demandes en temps réel, et peut gérer les offres de services et de formations disponibles.

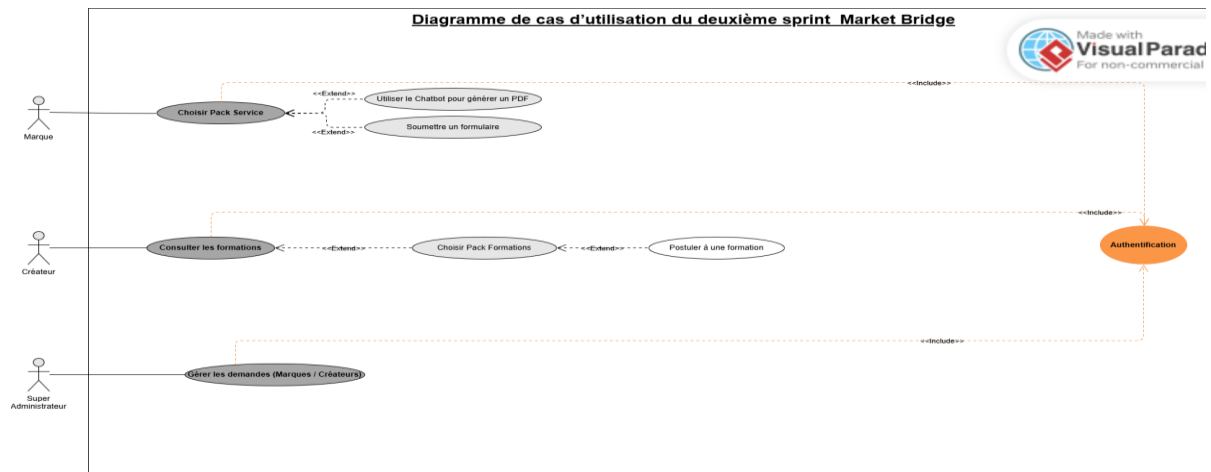


Figure 41: Diagramme de cas d'utilisation du deuxième sprint (Market Bridge):

4.2.2 Description textuelle des cas d'utilisation

4.2.2.1 Cas d'utilisation : Choisir un pack (Créateur)

Tableau 12: Description textuelle des cas d'utilisation - Choisir un pack (Créateur)

Élément	Description
Acteurs	Créateur, Système
Pré-condition	Le créateur est connecté à la plateforme avec un compte validé.
Post-condition	Le créateur sélectionne un pack et accède au formulaire de candidature.
Scénario nominal	1. Le créateur accède à la page d'accueil "Choisir un pack de formation". 2. Il consulte la description et le prix de chaque pack (Basique, Intermédiaire, Premium). 3. Il clique sur "Postuler". 4. Le système redirige vers le formulaire de postulation.
Scénario d'exception	Si le créateur n'est pas connecté, le système le redirige vers la page d'authentification.

4.2.2.2 Cas d'utilisation : Postuler à une formation (Créateur)

Tableau 13: Description textuelle des cas d'utilisation - Postuler à une formation (Créateur)

Élément	Description
Acteurs	Créateur, Système, Super Administrateur
Pré-condition	Le créateur est connecté et a sélectionné un pack de formation.
Post-condition	La demande est enregistrée et une notification est envoyée en temps réel au Super Administrateur.
Scénario nominal	1. Le créateur remplit les champs requis (nom, pays, email, téléphone, pack choisi). 2. Il clique sur “Envoyer”. 3. Le système vérifie la validité des données et enregistre la demande. 4. Une notification est transmise instantanément au Super Administrateur. 5. Le Super Administrateur visualise la nouvelle demande dans la section “Demandes des Créateurs”.
Scénario d'exception	Si un champ obligatoire est vide, le système affiche : “Veuillez remplir tous les champs”.

4.2.2.3 Cas d'utilisation : Choisir un pack (Marque)

Tableau 14: Description textuelle des cas d'utilisation - Choisir un pack (Marque)

Élément	Description
Acteurs	Marque, Système
Pré-condition	La marque est connectée avec un compte valide.
Post-condition	La marque accède au formulaire de demande du pack sélectionné.
Scénario nominal	1. La marque accède à la page “Choisissez un pack pour commencer”. 2. Elle consulte les packs (Basique, Intermédiaire, Premium) et leurs tarifs. 3. Elle clique sur “Postuler”.

	4. Le système la redirige vers le formulaire de collaboration.
Scénario d'exception	Si la marque n'est pas connectée, elle est redirigée vers la page d'authentification.

4.2.2.4 Cas d'utilisation : Soumettre un formulaire (Marque)

Tableau 15: Description textuelle des cas d'utilisation - Soumettre un formulaire (Marque)

Élément	Description
Acteurs	Marque, Système, Super Administrateur
Pré-condition	La marque a sélectionné un pack et un créateur associé.
Post-condition	Le formulaire est enregistré dans la base de données et une notification est envoyée au Super Administrateur.
Scénario nominal	1. La marque remplit les champs du formulaire (nom, pays, marque, email, pack demandé, créateur associé, prix). 2. Elle clique sur "Envoyer". 3. Le système valide les données et sauvegarde la demande. 4. Une notification en temps réel est transmise au Super Administrateur via WebSocket. 5. Le Super Administrateur peut consulter la nouvelle demande dans la section "Demandes des Marques".
Scénario d'exception	Si un champ obligatoire est manquant, le système affiche : "Veuillez remplir tous les champs".

4.2.2.5 Cas d'utilisation : Gérer les packs (Super Administrateur)

Tableau 16: Description textuelle des cas d'utilisation - Gérer les packs (Super Admin)

Élément	Description
---------	-------------

Acteurs	Super Administrateur
Pré-condition	Le Super Administrateur est connecté à la plateforme.
Post-condition	Les informations des packs (formations/services) sont mises à jour dans la base de données.
Scénario nominal	1. Le Super Administrateur accède à son tableau de bord. 2. Il consulte la liste des packs de formations et services. 3. Il sélectionne un pack à modifier. 4. Il modifie le nom, la description, le prix ou l'image. 5. Il enregistre les changements. 6. Le système met à jour automatiquement les données affichées côté créateur ou marque.
Scénario d'exception	Si un champ est vide ou invalide, un message d'erreur s'affiche : "Veuillez vérifier les informations du pack."

4.3 Conception du deuxième sprint

4.3.1 Diagrammes de séquence

Les diagrammes de séquence décrivent les échanges entre les utilisateurs (Créateur, Marque, Super Admin) et les composants clés du système (Angular, Spring Boot, MySQL, WebSocket).

Ils permettent de visualiser comment les différentes actions sont traitées en temps réel, notamment la postulation à un pack de formation et la gestion des notifications.

4.3.1.1 Postulation à un pack et notification (Créateur)

Ce diagramme illustre le parcours d'un créateur souhaitant postuler à une formation via la plateforme.

Depuis l'interface Angular, il consulte les packs (Basique, Intermédiaire, Premium) et sélectionne celui qui lui convient.

Il remplit ensuite le formulaire, qui est envoyé au backend via le contrôleur **AuthController**.

Les données sont traitées par le service **CreatorRequestService**, puis enregistrées dans la base MySQL.

Une fois la postulation validée, une notification est envoyée en **temps réel** au Super Administrateur via **NotificationSocketController (STOMP/WebSocket)**.

La nouvelle demande s'affiche immédiatement sur le tableau de bord admin, indiquant qu'un créateur vient de postuler.

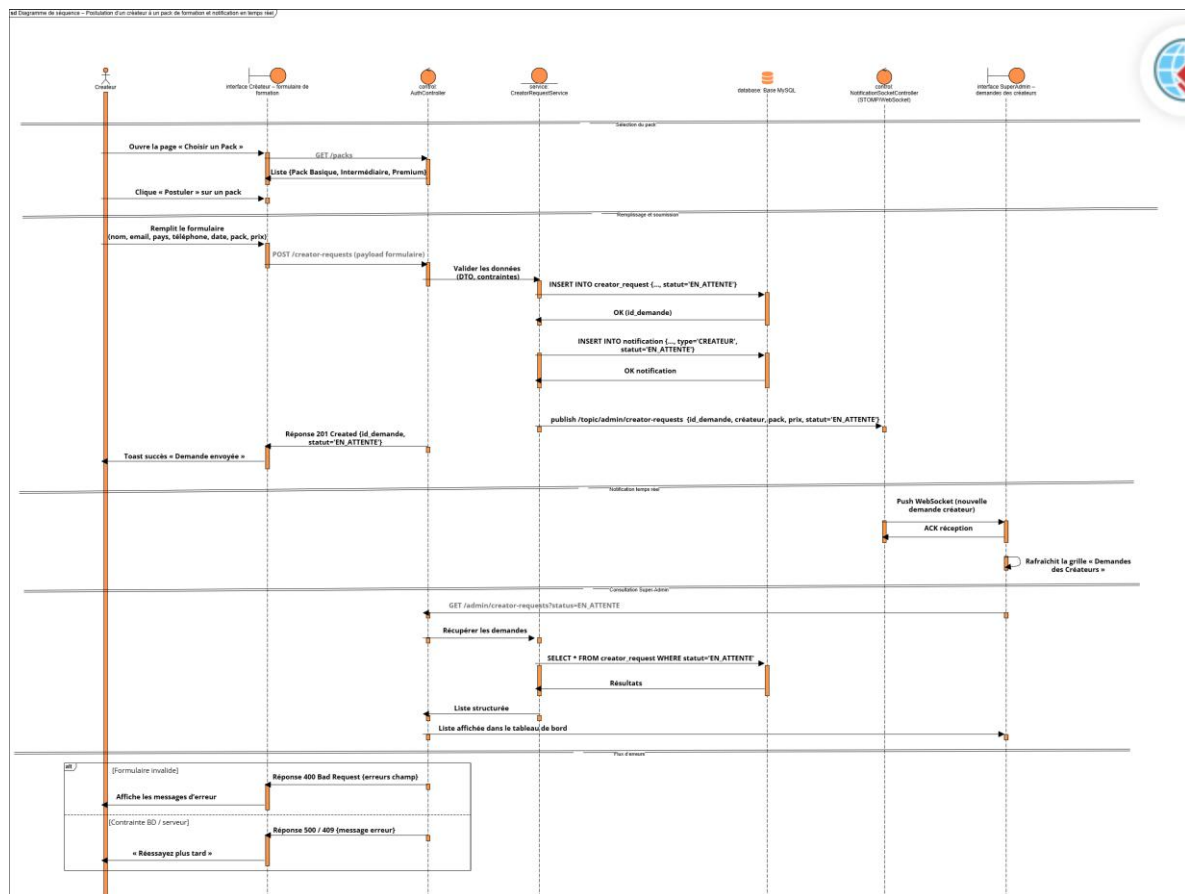


Figure 42: Diagramme de séquence : Postulation d'un créateur et notification en temps réel

4.3.1.2 Postulation d'une marque à un pack et notification en temps réel

Ce diagramme montre le parcours d'une marque postulant à un pack de service sur **Market Bridge**.

Depuis l'interface Angular, elle consulte **les packs** proposés (Basique, Intermédiaire, Premium), choisit celui qui correspond à son besoin, puis remplit un formulaire contenant ses informations (nom, email, pays, téléphone), le nom de sa marque, le créateur associé, le pack sélectionné, et un fichier **PDF** décrivant son projet.

Une fois le formulaire soumis, la requête est traitée par le contrôleur **AuthController**, puis transmise au service **FormService** qui valide et enregistre les données dans la base MySQL. Une notification est ensuite déclenchée via le contrôleur **NotificationSocketController** (**STOMP/WebSocket**).

Elle s'affiche instantanément sur le tableau de bord du Super Administrateur, dans la section "**Demandes des Marques**", permettant un suivi en temps réel des nouvelles soumissions.

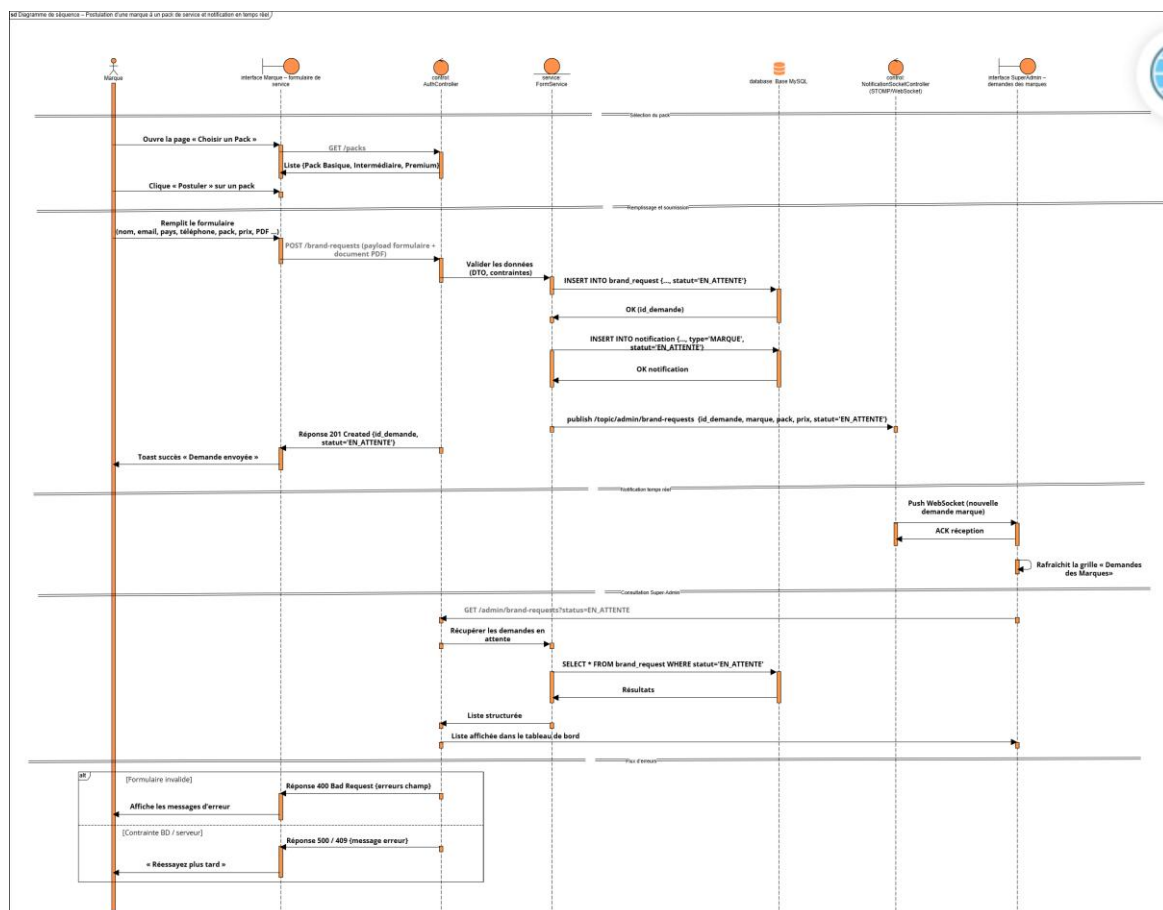


Figure 43: Diagramme de séquence : Postulation d'une marque et notification en temps réel

4.3.1.3 Gestion des demandes (Marques et Créateurs)

Ce diagramme illustre comment le Super Administrateur traite les demandes reçues depuis son tableau de bord.

Il peut consulter l'ensemble des formulaires envoyés par les marques et les créateurs, puis décider de les valider ou de les refuser.

Une fois l'action effectuée, le statut de la demande est mis à jour, et une notification est automatiquement envoyée à l'utilisateur concerné.

Le tout fonctionne **en temps réel** via **WebSocket**, garantissant une synchronisation fluide entre l'interface du Super Admin et celles des marques ou créateurs.

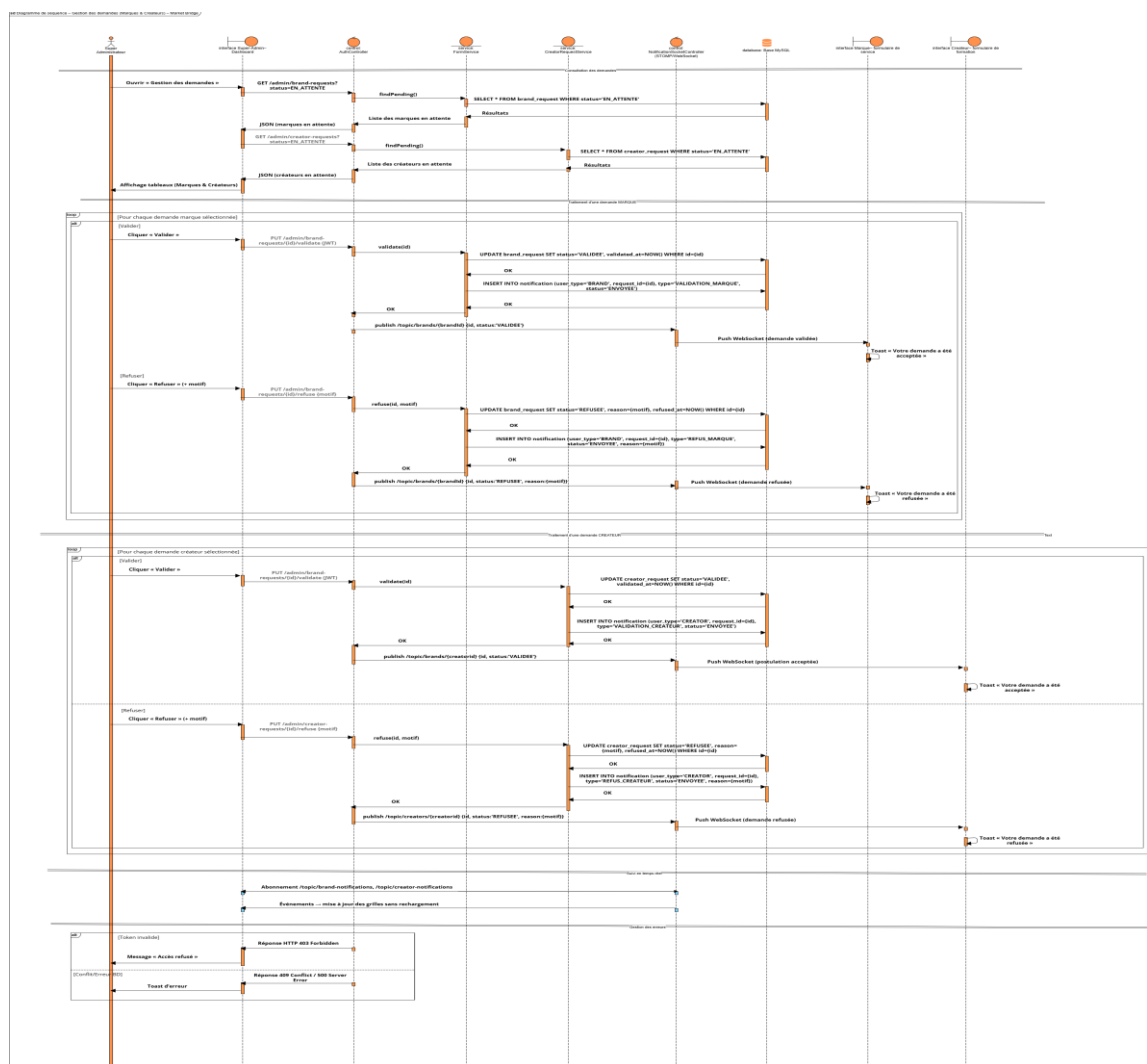


Figure 44: Diagramme de séquence : Gestion des demandes (Marques et Créateurs)

4.4 Réalisation du sprint

Après validation du backlog et des besoins fonctionnels, l'équipe a mis en œuvre les modules prévus pour ce sprint.

L'objectif principal était d'assurer une interaction fluide entre marques, créateurs et Super Administrateur, notamment pour la gestion des postulations en temps réel.

Les créateurs peuvent désormais postuler à des packs de formation, les marques soumettre des demandes de collaboration, et le Super Administrateur suivre et gérer toutes ces demandes via son tableau de bord.

Le développement s'appuie sur Angular (frontend), Spring Boot (backend), MySQL (base de données) et WebSocket pour les notifications en temps réel.

4.4.1 Interfaces réalisées

4.4.1.1 Interface des packs de formation

Affiche les packs proposés aux créateurs (Basique, Intermédiaire, Premium), avec description, prix et bouton de postulation.

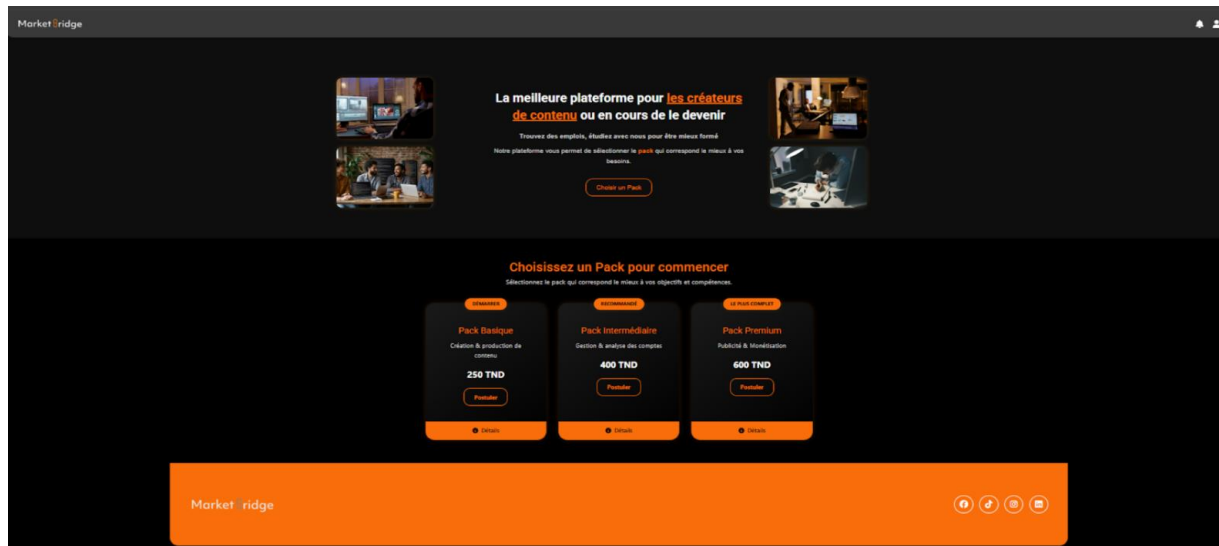


Figure 45: Interface des packs de formation

4.4.1.2 Interface de postulation d'un créateur

Permet au créateur de remplir un formulaire avec ses informations personnelles.

Une fois validée, la demande est enregistrée et une notification est envoyée au Super Administrateur.

Postuler à un Pack Créateur

Nom complet
Sara Gram

Adresse e-mail
saragram@creator.tn

Sélectionner le pays
Tunisie

Numéro de téléphone
+21654102330

Date de naissance
15/03/1998

Pack choisi
Pack Intermédiaire

Prix
400 TND

Envoyer

Figure 46: Interface de postulation d'un créateur

4.4.1.3 Interface des packs de service

Destinée aux marques, cette interface présente les différents packs disponibles avec leurs caractéristiques.

Chaque pack peut être sélectionné pour initier une demande.

MarketBridge

La meilleure plateforme pour les marques établies ou en cours de développement

Trouvez des créateurs formés par nos soins ou déjà partenaires, prêts à répondre à vos demandes.

Notre plateforme vous permet de sélectionner le pack qui correspond le mieux à vos besoins.

Choisir un Pack

Choisissez un Pack pour commencer

Sélectionnez le pack qui correspond le mieux à vos objectifs.

LE BASIQUE	RECOMMANDÉ	LE PLUS COMPLET
Pack Basique Contenu simple : Une seule image ou un post visuel adapté aux réseaux sociaux.	Pack Intermédiaire Contenu vidéo court : Une vidéo de moins d'une minute, optimisée pour les réseaux sociaux.	Pack Premium Contenu vidéo long : Une vidéo de plus d'une minute, créative et professionnelle.
250 TND	400 TND	600 TND
Postuler	Postuler	Postuler
Détails	Détails	Détails

Figure 47: Interface des packs de service

4.4.1.4 Interface de postulation d'une marque

Une fois le pack choisi, la marque remplit un formulaire avec ses informations, le créateur visé, et un document PDF.

La demande est enregistrée et notifiée en temps réel.

Postuler à un Pack Marque

Nom complet
Wiem benali

Sélectionner le pays
Tunisie

Numéro de téléphone
+21623213381

Prix
250 TND

Adresse e-mail
wiem.benali@esprit.tn

Nom de la marque
Wiem ESPRIT

Pack demandé
Pack Basique

Créateur associé
Sara Gram

Document PDF

Offre_Wiembenali_20251106_030242 (8).pdf

Envoyer

Figure 48: Interface de postulation d'une marque

4.4.1.5 Interface des demandes des marques (Super Admin)

Affiche toutes les demandes soumises par les marques.

Le Super Admin peut accepter ou refuser chaque demande, déclenchant automatiquement une notification.

MarketSridge

Tableau de Bord

- [Tableau de Bord](#)
- [Formulaires](#)
- [Services](#)
- [Statistiques](#)
- [Utilisateurs](#)
- [Modération](#)
- [Demandes des Marques](#)
- [Demandes des Créateurs](#)
- [Feedbacks](#)

Paramètres du compte

- [Paramètres](#)
- [Régénérer 2FA](#)
- [Activer/Désactiver 2FA](#)
- [Déconnexion](#)

Demandes des Marques

Tout
En attente
Acceptées

Nom Complet	Email	Pays	Nom de Marque	Numéro	Pack demandé	Prix	Créateur associé	Description	Actions
Wiem benali	wiem.benali@esprit.tn	TN	Wiem ESPRIT	+21623213381	Pack Basique	250 TND	Sara Gram		Voir 🔊 ✕

Figure 49: Interface des demandes des marques

4.4.1.6 Interface des demandes des créateurs (Super Admin)

Même principe que pour les marques, cette interface permet de gérer les demandes des créateurs en temps réel.

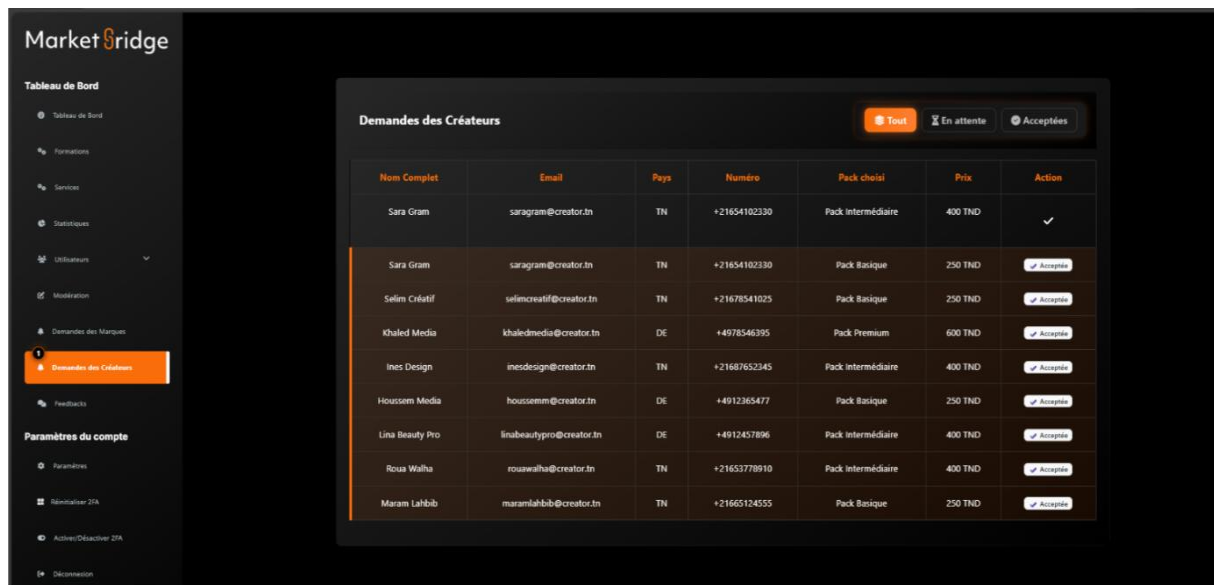


Figure 50: Interface des demandes des créateurs

4.4.1.7 Interface de gestion des packs (Super Admin)

Permet d'ajouter, modifier ou supprimer les packs de formations et services depuis un seul espace.

Les changements sont immédiatement visibles pour les utilisateurs.

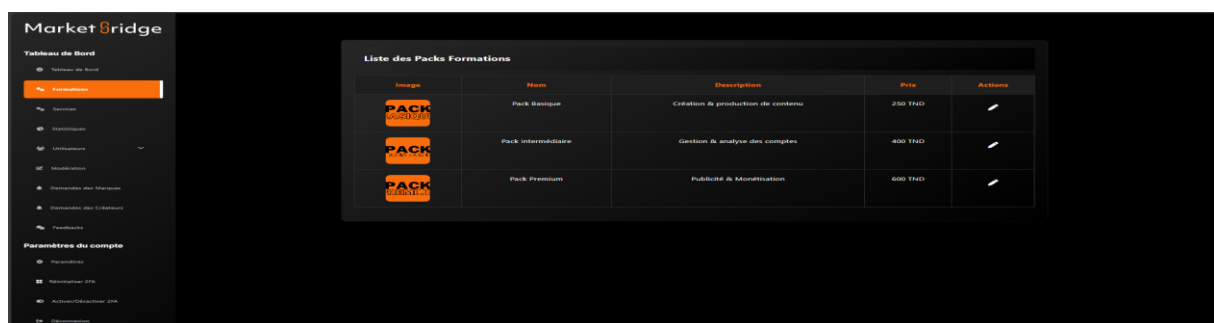


Figure 51: Interface de gestion des packs de formations

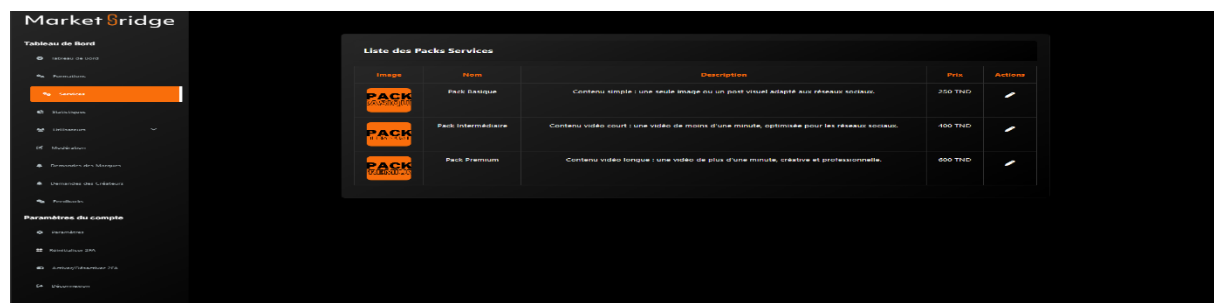


Figure 52: Interface de gestion des packs de services

4.4.1.8 Interface de suivi des demandes (Marque)

Offre à la marque un tableau de bord pour suivre ses demandes:

nom du pack, créateur, date d'envoi et statut (En attente, Acceptée, Refusée).

Les données sont synchronisées en temps réel.

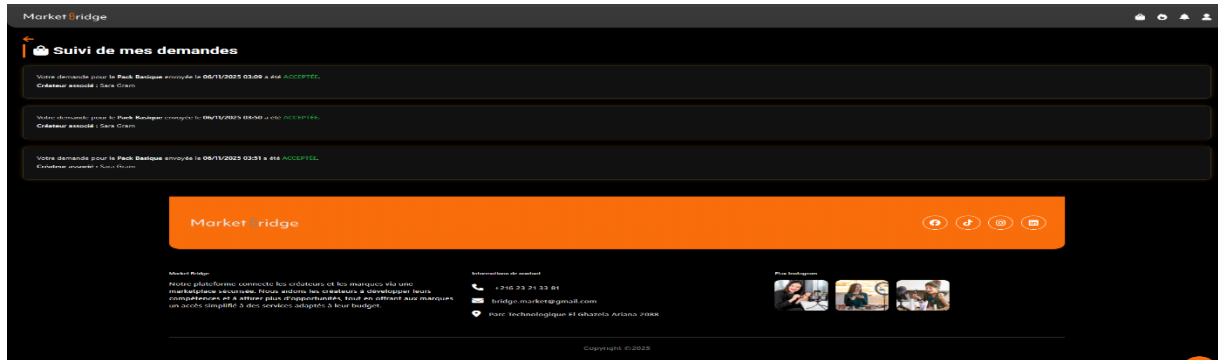


Figure 53: Interface de suivi des demandes

4.5 Revue du deuxième sprint

À la fin du sprint, une réunion de revue a réuni le Scrum Master, le Product Owner et l'équipe de développement pour évaluer les résultats obtenus par rapport aux objectifs fixés.

Ce sprint visait à établir un cycle complet de postulation entre marques, créateurs et Super Administrateur.

Toutes les fonctionnalités prévues ont été livrées, testées et validées avec succès, notamment :

- Les interfaces de sélection et de postulation pour les marques et les créateurs
- L'enregistrement des demandes avec validation côté backend
- L'intégration des notifications en temps réel via WebSocket/STOMP
- Un tableau de bord pour le Super Admin permettant de gérer les demandes reçues
- Une meilleure expérience utilisateur grâce à des messages de confirmation clairs

Le Product Owner a salué la fluidité du parcours utilisateur, la réactivité du système, ainsi que l'ergonomie et la cohérence des interfaces.

Ce sprint a structuré les fondations du mécanisme de collaboration dynamique entre les acteurs de la plateforme.

4.6 Rétrospective du deuxième sprint

Le sprint a été mené dans les délais, avec une livraison complète et stable des fonctionnalités prévues.

La collaboration entre les équipes frontend et backend a été fluide, facilitant l'intégration avec la base de données MySQL.

Quelques défis techniques ont néanmoins émergé :

- L'implémentation du canal WebSocket pour les notifications
- La synchronisation des statuts entre les demandes des marques et des créateurs
- La validation stricte des champs côté backend
- L'optimisation des requêtes SQL pour un affichage rapide
- L'unification du design des formulaires et tableaux

Ces points ont été résolus grâce à une approche collaborative, des tests fréquents et une communication constante entre les membres de l'équipe.

Conclusion

Ce chapitre a présenté la réalisation du deuxième sprint, axé sur la gestion des demandes, leur validation et la notification en temps réel.

Il a permis de concrétiser l'interaction entre marques, créateurs et Super Administrateur, tout en améliorant nettement l'expérience utilisateur.

CHAPITRE 5 : ÉTUDE ET RÉALISATION DU TROISIÈME SPRINT

Introduction

Ce cinquième chapitre est consacré à la mise en œuvre du troisième sprint de la plateforme Market Bridge. Il marque une étape clé : le passage d'un simple système de soumission à un processus complet de modération et d'échange en temps réel entre les différents utilisateurs.

Après avoir permis aux marques et aux créateurs de soumettre leurs demandes et postulations lors du sprint précédent, cette itération introduit un **workflow collaboratif** structurant les échanges entre :

- Les **Marques**,
- Les **Créateurs**,
- Et le **Super Administrateur**.

L'objectif est d'assurer une **modération fluide, sécurisée et traçable** des contenus (images ou vidéos) soumis, tout en garantissant une communication instantanée entre les acteurs à chaque étape du cycle.

Ce processus couvre l'ensemble du parcours :

- **La marque** envoie une demande (fonctionnalité déjà mise en place).
- **Le Super Administrateur** la valide et l'assigne à un créateur.
- **Le créateur** reçoit une notification, produit et soumet le contenu.
- **Le Super Administrateur** modère la soumission (validation ou refus).
- **La marque** valide, demande une amélioration ou effectue le paiement avec feedback.

Ce **cycle collaboratif complet**, centré sur la validation et le suivi, structure désormais l'ensemble des échanges sur Market Bridge.

5.1 Sprint Backlog

Ce sprint a été dédié à l'intégration du module de **modération**, à la **gestion des contenus** et à l'**échange en temps réel** entre les rôles via WebSocket.

Les objectifs étaient de garantir :

- La transparence des échanges,
- La traçabilité des actions,
- Et la fluidité des interactions entre marques, créateurs et administrateur.

Tableau 17: Sprint Backlog du troisième sprint

ID	User Story	Priorité
US-01	En tant que Super Admin, je veux valider les demandes des marques et les assigner à un créateur.	Haute
US-02	En tant que Créateur, je veux recevoir une notification lorsqu'une nouvelle proposition m'est attribuée et pouvoir uploader mon contenu.	Haute
US-03	En tant que Super Admin, je veux modérer les contenus soumis (valider ou refuser).	Haute
US-04	En tant que Marque, je veux être notifiée en temps réel du statut de validation d'un contenu.	Haute
US-05	En tant que système, je veux transmettre les notifications en temps réel via WebSocket.	Moyenne

Grâce à ce sprint, un **flux de travail interactif et sécurisé** a été mis en place entre les trois profils de la plateforme, renforçant la réactivité et la cohérence du système.

5.2 Spécification fonctionnelle du troisième sprint

Cette partie détaille les spécifications fonctionnelles développées lors du sprint, avec un focus sur la **modération** et la **validation des contenus**. Les interactions ont été modélisées via des **diagrammes UML**, illustrant :

- La validation ou le rejet d'une proposition,
- Le traitement des feedbacks par les marques,
- Les notifications temps réel envoyées aux utilisateurs,
- Et la mise à jour des statuts et paiements.

5.2.1 Diagramme de cas d'utilisation du troisième sprint

Le diagramme suivant illustre les interactions fonctionnelles entre les trois rôles principaux du système dans le cadre du troisième sprint.

Il montre comment la **Marque** consulte et évalue les propositions reçues (acceptation, refus ou feedback), comment le **Créateur** répond en téléversant de nouveaux contenus (nouvelle proposition ou amélioration), et comment le **Super Administrateur** supervise l'ensemble du processus à travers la **modération** et la **gestion centralisée des feedbacks**.

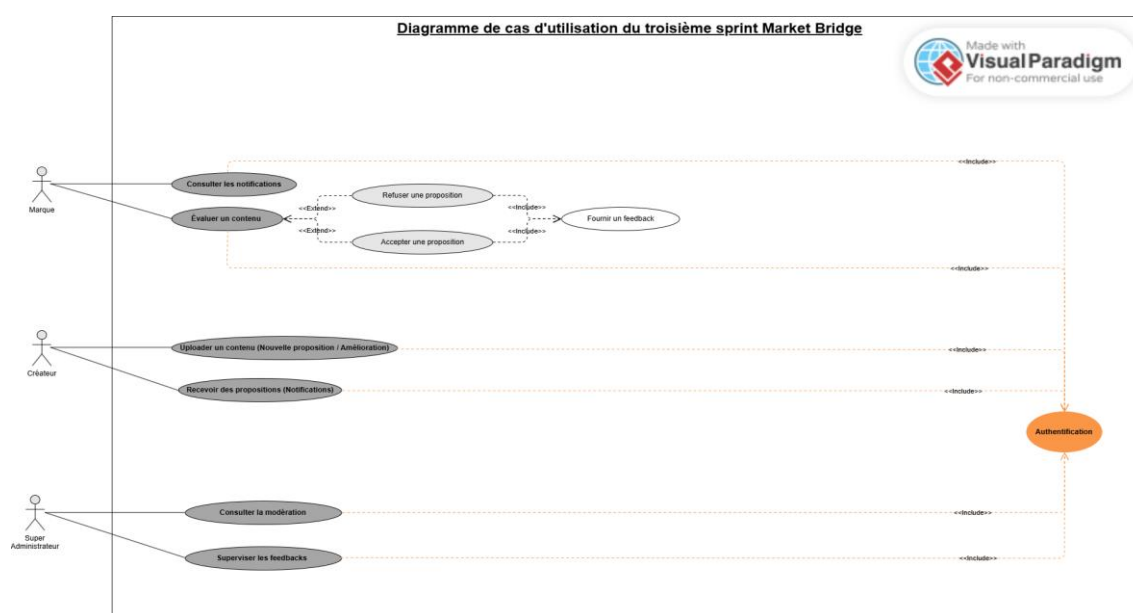


Figure 54: Diagramme de cas d'utilisation du troisième sprint (Market Bridge)

5.2.2 Description textuelle des cas d'utilisation

5.2.2.1 Cas d'utilisation : Consulter les notifications (Marque)

Tableau 18: Description textuelle des cas d'utilisation - Consulter les notifications (Marque)

Élément	Description
Acteurs	Marque, Système
Pré-condition	La marque est connectée à la plateforme.
Post-condition	Les notifications récentes (contenus validés, refusés ou en attente) sont affichées.

Scénario nominal	1. La marque accède à la section “Mes Propositions” . 2. Le système affiche la liste des propositions avec leur statut (En attente, Validée, Refusée). 3. La marque peut consulter le contenu associé (image ou vidéo avec filigrane).
Scénario d'exception	Si aucune notification n'est disponible, un message s'affiche : “Aucune proposition pour le moment.”

5.2.2.2 Cas d'utilisation : Évaluer un contenu (Marque)

Tableau 19: Description textuelle des cas d'utilisation - Evaluer un contenu (Marque)

Élément	Description
Acteurs	Marque, Système, Super Administrateur
Pré-condition	Un contenu modéré et validé par le Super Administrateur est disponible.
Post-condition	La marque choisit entre accepter ou refuser le contenu et fournit un feedback .
Scénario nominal	1. La marque ouvre une proposition reçue. 2. Elle visualise le contenu validé par le Super Administrateur. 3. Elle attribue une note au créateur. 4. Elle clique sur “Accepter” ou “Refuser” selon son appréciation. 5. En cas d'acceptation, le système affiche le bouton “Payer ce pack” . 6. En cas de refus, la marque saisit un motif et valide le formulaire.
Scénario d'exception	Si la marque tente d'envoyer un feedback sans note, un message d'erreur s'affiche : “Veuillez attribuer une note avant de valider.”

5.2.2.3 Cas d'utilisation : Fournir un feedback (Marque)

Tableau 20: Description textuelle des cas d'utilisation - Fournir un feedback (Marque)

Élément	Description
Acteurs	Marque, Système, Super Administrateur

Pré-condition	Le contenu a été évalué (accepté ou refusé).
Post-condition	Le feedback est enregistré et transmis au Super Administrateur.
Scénario nominal	1. Après avoir accepté ou refusé une proposition, la marque saisit une note et un commentaire. 2. Le système enregistre ces informations dans la table Feedback. 3. En cas de refus, une notification d'amélioration est envoyée au créateur. 4. En cas d'acceptation, le statut passe à Validé et la marque procède au paiement.
Scénario d'exception	Si le serveur ne répond pas, un message d'erreur s'affiche : “Erreur d'enregistrement du feedback.”

5.2.2.4 Cas d'utilisation : Uploader un contenu (Créateur)

Tableau 21: Description textuelle des cas d'utilisation - Uploader un contenu (Créateur)

Élément	Description
Acteurs	Créateur, Système, Super Administrateur
Pré-condition	Le créateur a reçu une notification de nouvelle proposition ou de demande d'amélioration.
Post-condition	Le fichier est enregistré et transmis pour modération .
Scénario nominal	1. Le créateur accède à la section “Mes Propositions”. 2. Il consulte la demande reçue (nouvelle ou amélioration). 3. Il sélectionne un fichier (image, vidéo ou PDF). 4. Il clique sur “Uploader & Envoyer”. 5. Le système sauvegarde le contenu et notifie le Super Administrateur via WebSocket.
Scénario d'exception	Si le fichier dépasse la taille maximale autorisée, un message s'affiche : “Taille du fichier non conforme.”

5.2.2.5 Cas d'utilisation : Consulter la modération (Super Administrateur)

Tableau 22: Description textuelle des cas d'utilisation - Consulter la modération (Super Admin)

Élément	Description
Acteurs	Super Administrateur, Système
Pré-condition	Un contenu a été téléversé par un créateur.
Post-condition	Le statut du contenu est mis à jour selon la décision (Validé ou Refusé).
Scénario nominal	1. Le Super Administrateur accède à la section “Modération”. 2. Il visualise la liste des contenus soumis par les créateurs. 3. Il consulte chaque contenu via un aperçu intégré (image, vidéo, PDF). 4. Il clique sur “Valider” ou “Refuser” selon la qualité du contenu. 5. Le système met à jour le statut et envoie une notification à la marque concernée.
Scénario d'exception	Si le fichier est illisible, le système affiche : “ Erreur de lecture du contenu. ”

5.2.2.6 Cas d'utilisation : Superviser les feedbacks (Super Administrateur)

Tableau 23: Description textuelle des cas d'utilisation - Superviser les feedbacks (Super Admin)

Élément	Description
Acteurs	Super Administrateur, Système
Pré-condition	Les marques ont déjà envoyé leurs feedbacks .
Post-condition	Le Super Administrateur visualise les feedbacks refusés et validés .
Scénario nominal	1. Le Super Administrateur ouvre la section “Feedbacks”. 2. Il consulte les feedbacks refusés (motif, note, créateur associé). 3. Il peut cliquer sur “Renvoyer une demande d'amélioration” pour notifier le créateur.

	4. Il consulte également les contenus acceptés et payés .
Scénario d'exception	Si aucun feedback n'est enregistré, la liste reste vide avec le message : “Aucun feedback disponible.”

5.2.2.7 Cas d'utilisation : Gérer les packs (Super Administrateur)

Tableau 24: Description textuelle des cas d'utilisation - Gérer les packs (Super Admin)

Élément	Description
Acteurs	Super Administrateur
Pré-condition	Le Super Administrateur est connecté à la plateforme.
Post-condition	Les packs (Formations / Services) sont mis à jour dans la base de données.
Scénario nominal	1. Le Super Administrateur accède à la section “Formations” ou “Services”. 2. Il visualise la liste des packs existants (Basique, Intermédiaire, Premium). 3. Il peut modifier le nom, la description, le prix ou l'image d'un pack. 4. Le système met à jour automatiquement les données affichées côté créateur ou marque.
Scénario d'exception	Si un champ est invalide, un message s'affiche : “Veuillez vérifier les informations du pack.”

5.3 Conception du troisième sprint

5.3.1 Diagrammes de séquence

Les diagrammes de séquence de ce sprint illustrent les échanges entre les différents rôles (Marque, Créateur, Super Administrateur) et les composants du système (Spring Boot, Angular, MySQL, Stripe, WebSocket), dans le cadre du **workflow collaboratif** mis en place.

Ils permettent de suivre les étapes clés du cycle, depuis la **soumission du contenu** jusqu'à son **évaluation**, sa **modération**, son **amélioration éventuelle**, puis son **paiement final**.

5.3.1.1 Diagramme de séquence : Évaluation et boucle d'amélioration

Ce diagramme décrit la phase d'**évaluation du contenu** par la **marque**, suite à la modération du **Super Administrateur**. Deux cas de figure sont possibles :

❖ Cas 1 : Acceptation

- La **marque** valide le contenu reçu.
- Elle procède au **paiement sécurisé via Stripe**.
- Le système met à jour le **statut de la proposition** (VALIDÉ) et enregistre le **feedback** avec l'état PAYÉ.
- Une **notification finale** est envoyée à la marque et au Super Admin pour confirmer la disponibilité du contenu.

❖ Cas 2 : Refus

- La **marque** émet une **note et un motif de refus**.
- Le **Super Admin** envoie une **notification en temps réel** au **créateur**, demandant une **amélioration**.
- Le créateur soumet une **nouvelle version** (V2) du contenu.
- Le contenu passe de nouveau en **modération**.
- Ce cycle se répète autant de fois que nécessaire, jusqu'à validation finale.
- Le statut final est mis à jour (VALIDÉ_APRÈS_AMÉLIORATION) et une notification de clôture est envoyée.

Ce scénario illustre l'efficacité du workflow de Market Bridge : **feedback structuré**, **amélioration continue**, **synchronisation en temps réel** et **paiement intégré**.

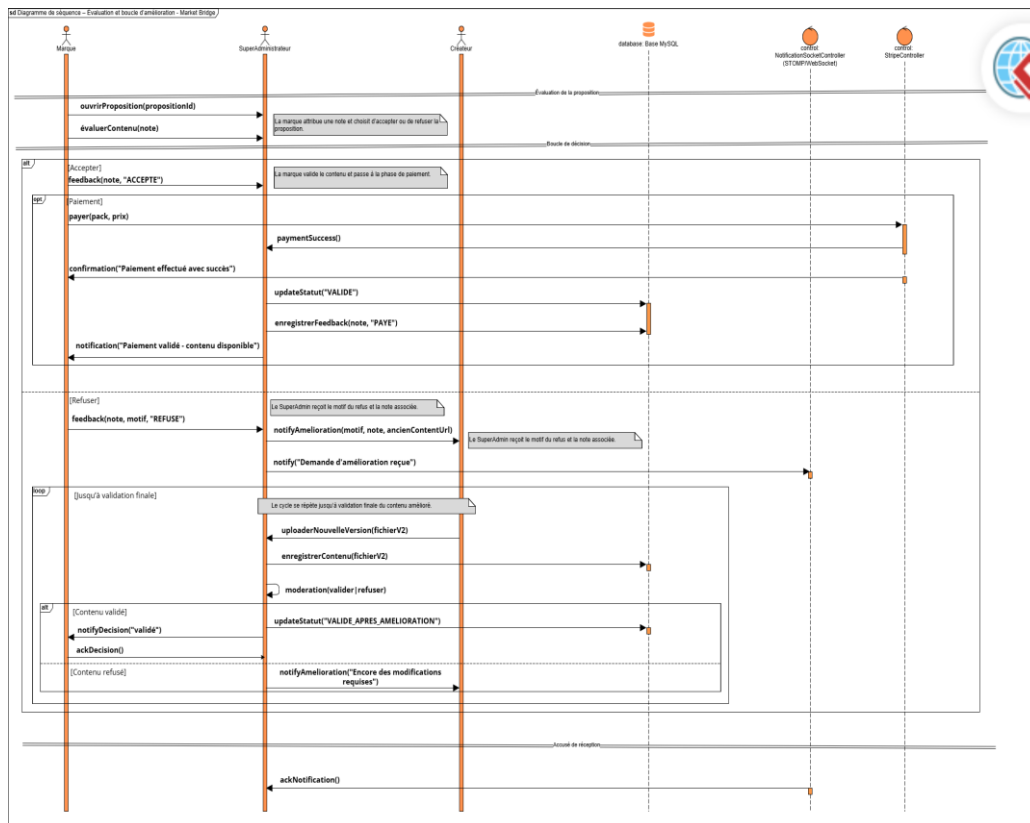


Figure 55: Diagramme de séquence : Évaluation et boucle d'amélioration

5.3.1.2 Diagramme de séquence : Nouvelle proposition (Marche → Créateur via Super Admin)

Ce diagramme décrit le flux de **création d'une nouvelle proposition** initiée par une marque :

- Une **marque** soumet une **demande de collaboration**, accompagnée d'un document explicatif (image ou vidéo).
- Le **Super Admin** examine la demande, la **valide**, puis l'**attribue à un créateur**.
- Le **créateur** reçoit une **notification instantanée**, accède à son espace personnel, téléverse le contenu demandé, puis l'envoie pour **modération**.
- Le **Super Administrateur** modère le contenu, puis notifie la **marque** du résultat (validation ou refus).
- Toutes les étapes sont **enregistrées** dans la base de données et **synchronisées en temps réel** grâce à WebSocket.

Ce diagramme met en lumière :

- La **traçabilité** des propositions,
- La **réactivité** du système,

- Et l'intégration fluide entre les rôles.

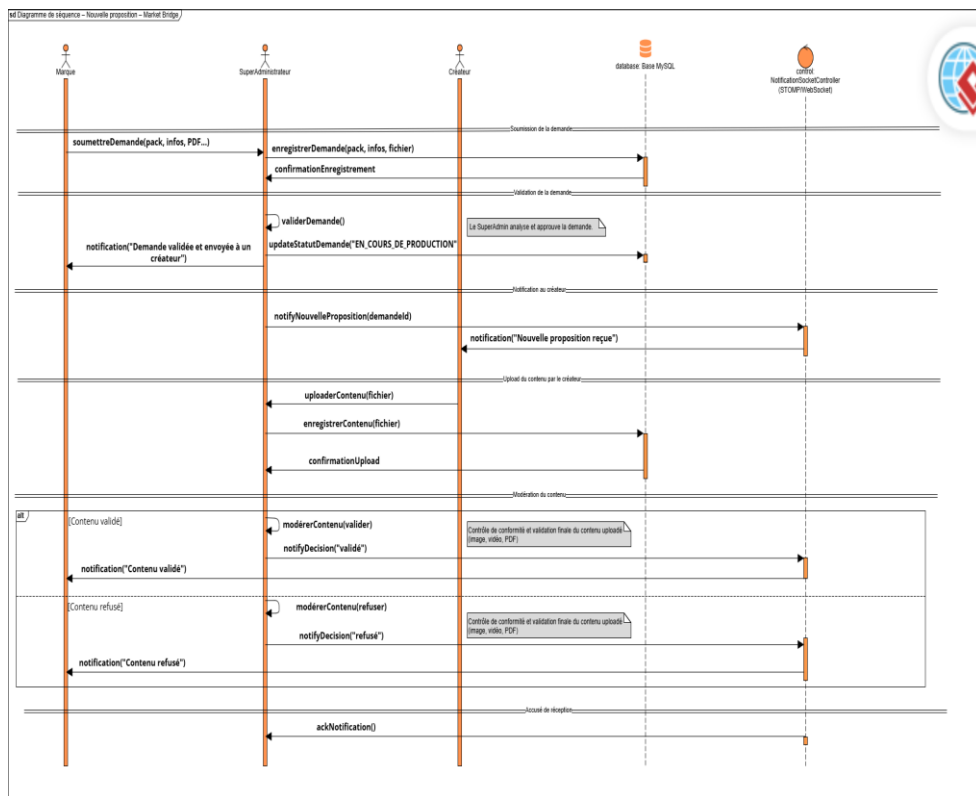


Figure 56: Diagramme de séquence : Nouvelle proposition

5.4 Réalisation du sprint

Après validation des objectifs, l'équipe a développé l'ensemble des fonctionnalités nécessaires à la gestion complète du cycle de production et de validation de contenu sur Market Bridge.

Ce sprint a permis de finaliser le flux de collaboration entre les marques, les créateurs et le Super Administrateur, en combinant :

- **Angular** pour les interfaces dynamiques,
- **Spring Boot** pour les APIs sécurisées (JWT),
- **MySQL** pour la persistance des données,
- **WebSocket/STOMP** pour la communication temps réel,
- Et **Stripe** pour la simulation de paiement sécurisé.

5.4.1 Interfaces réalisées

5.4.1.1 Réception d'une proposition (Créateur)

Le créateur reçoit une proposition validée, avec les détails du pack et la date.

Il peut y téléverser un fichier (image ou vidéo), qui sera ensuite transmis pour modération.

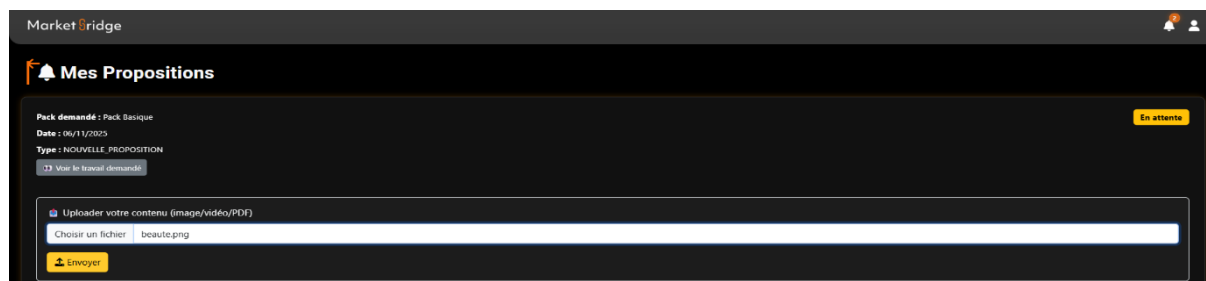


Figure 57: Interface du créateur : Réception d'une nouvelle proposition

5.4.1.2 Modération des contenus (Super Administrateur)

Le Super Admin accède à une interface listant toutes les propositions reçues.

Il peut prévisualiser chaque contenu, puis valider ou refuser la soumission.

Une notification est envoyée à la marque en temps réel.

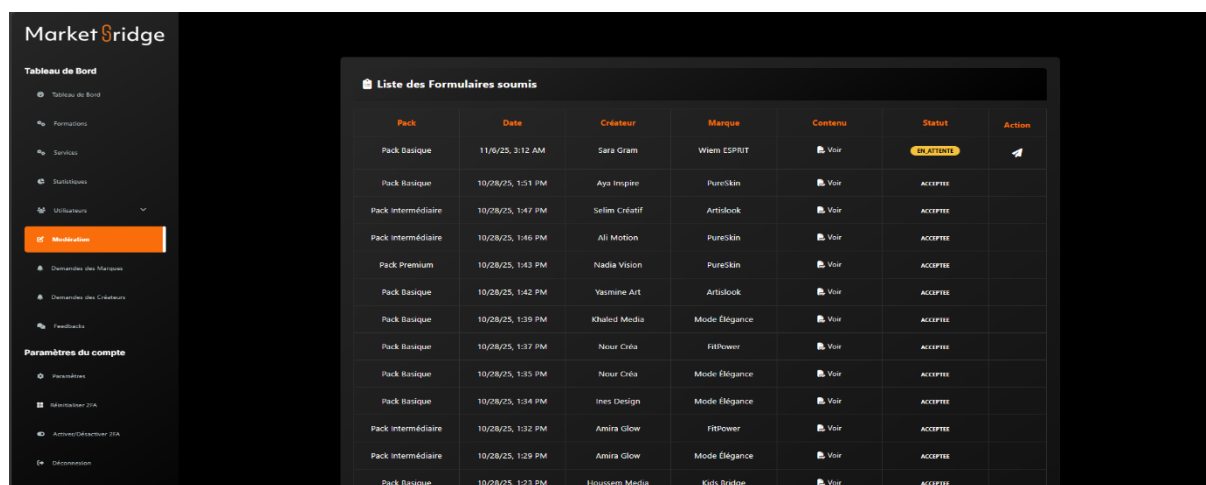


Figure 58: Interface du Super Administrateur : Modération des contenus

5.4.1.3 Évaluation du contenu (Marque)

Une fois le contenu validé par le Super Administrateur, la marque reçoit une notification pour le consulter.

Elle accède à une interface qui lui permet d'évaluer la proposition, avec deux options possibles :

❖ En cas d'acceptation:

La marque visualise le contenu avec filigrane, attribue une note au créateur, puis peut

procéder au paiement via Stripe.

Une fois le paiement validé, le fichier original devient accessible sans watermark.

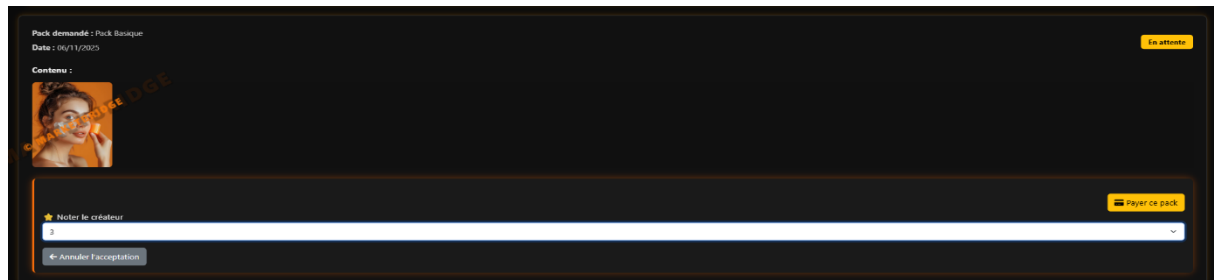


Figure 59: Interface de la marque : Évaluation du contenu (cas d'acceptation)

❖ En cas de refus:

La marque sélectionne une note et renseigne un motif détaillé de refus.

La décision est transmise en temps réel au Super Administrateur, qui notifie alors le créateur pour une amélioration.

Le filigrane reste actif tant que le paiement n'a pas été effectué, garantissant la sécurité et la propriété du contenu.

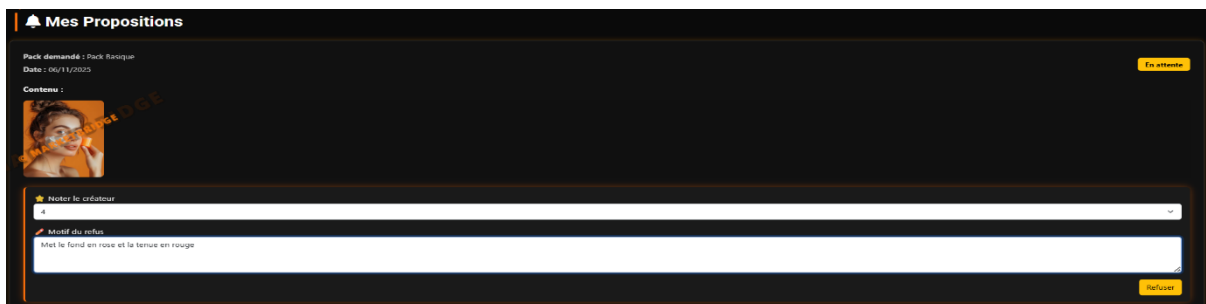


Figure 60: Interface de la marque : Évaluation du contenu (cas de refus)

5.4.1.4 Suivi des feedbacks (Super Administrateur)

Le tableau de bord permet au Super Admin de visualiser :

- Les propositions refusées (avec motifs, notes, créateurs),
- Les propositions acceptées (avec détails des marques, notes et prix).

Il peut également déclencher une demande d'amélioration.

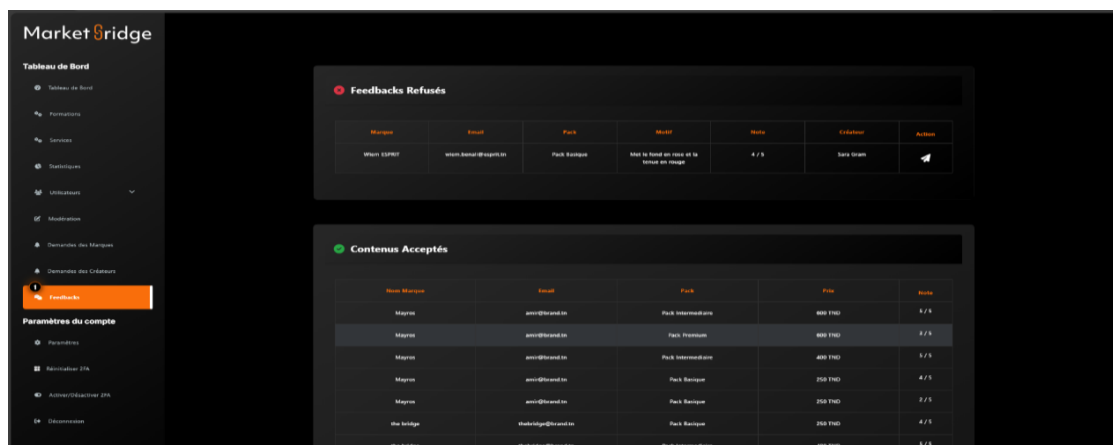


Figure 61: Interface du Super Administrateur : Feedbacks et validations

5.4.1.5 Amélioration d'un contenu (Créateur)

En cas de refus, le créateur est notifié avec le motif.

Il peut consulter la version précédente et soumettre une **nouvelle version (V2)** corrigée.

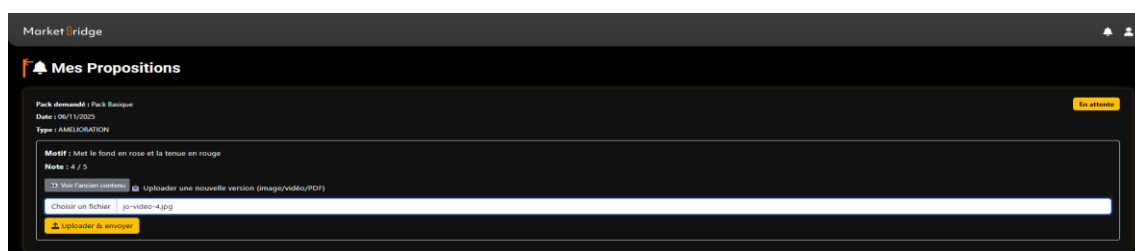


Figure 62: Interface du créateur : Envoi d'un contenu amélioré

5.4.1.6 Modération des contenus améliorés (Super Administrateur)

Les contenus améliorés sont listés avec leur statut ("EN_ATTENTE" ou "ACCEPTEE").

Chaque validation met à jour le statut global et notifie la marque.

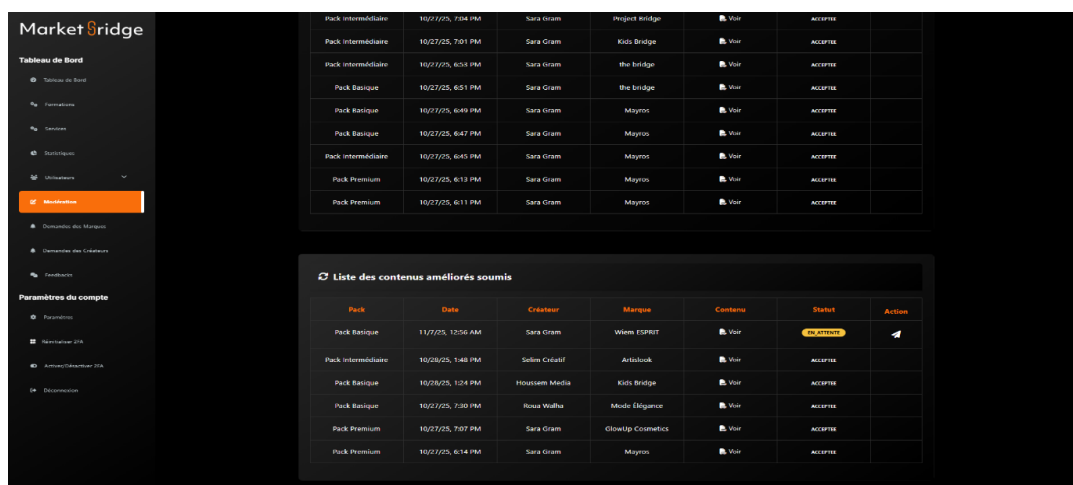


Figure 63: Interface du Super Administrateur : Modération des contenus améliorés

5.4.1.7 Visualisation finale (Marque)

Après validation de la version améliorée du contenu par le Super Administrateur, la marque accède à une interface récapitulative affichant la nouvelle proposition.

Le contenu est présenté avec un **filigrane de sécurité (watermark)**, et les boutons **“Accepter”** et **“Refuser”** restent actifs, permettant à la marque de confirmer ou non la proposition finale.

Cette interface permet à la marque de :

- Consulter le visuel amélioré,
- Prendre une **décision définitive** (acceptation ou refus motivé),
- Et finaliser le processus de collaboration avec un **paiement ou un feedback**.

Elle marque l'étape ultime du cycle de production, tout en laissant une **dernière marge de décision à la marque**.

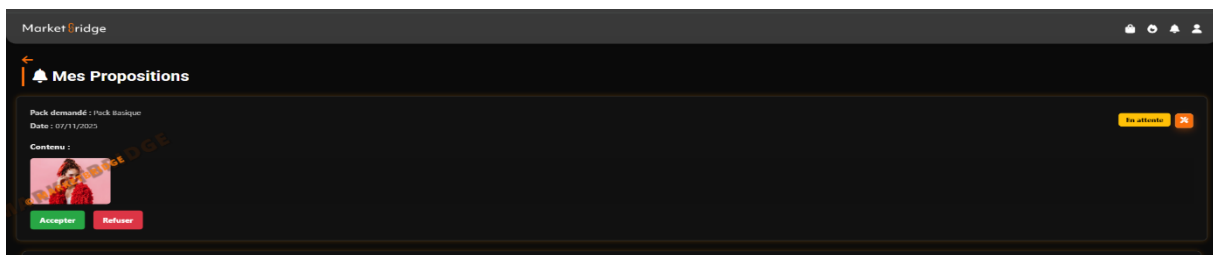


Figure 64: Interface de la marque : Visualisation de la version améliorée et décision finale

5.5 Revue du troisième sprint

À la fin du sprint, une revue a réuni le Scrum Master, le Product Owner et l'équipe de développement pour évaluer les fonctionnalités livrées.

Ce sprint a marqué une avancée majeure dans la plateforme Market Bridge en instaurant un **cycle complet de collaboration** entre les marques, les créateurs et le Super Administrateur : modération, validation de contenu, feedback et paiement.

Les objectifs ont été atteints avec succès grâce à :

- L'ajout d'un **module de modération** pour valider ou refuser les contenus soumis par les créateurs.
- Un système de **notifications en temps réel** via WebSocket/STOMP entre tous les rôles.

- La **protection des contenus** avec filigrane, empêchant l'accès aux fichiers originaux avant paiement.
- L'intégration de **Stripe** pour sécuriser le paiement des propositions acceptées.
- La mise en place d'une **boucle d'amélioration**, permettant aux créateurs de corriger leur contenu après refus.
- Une **gestion centralisée des feedbacks** pour un meilleur suivi par le Super Administrateur.

Le Product Owner a souligné la qualité de l'implémentation, la cohérence du flux de travail et la fiabilité du système, positionnant Market Bridge comme une plateforme professionnelle, sécurisée et bien structurée.

Ce sprint a donc permis de relier tous les modules précédents au sein d'un **écosystème fluide, automatisé et complet**, du dépôt de la demande jusqu'à la validation finale du contenu.

5.6 Rétrospective du troisième sprint

Ce sprint s'est déroulé comme prévu, avec une livraison complète du module de modération et de feedback.

La coordination entre les équipes frontend (Angular) et backend (Spring Boot) a permis une intégration fluide avec :

- MySQL pour la gestion des données,
- Stripe pour les paiements,
- Et WebSocket pour les échanges temps réel.

Malgré cela, plusieurs défis ont été relevés :

- Assurer une **synchronisation en temps réel** entre les rôles utilisateurs.
- Gérer les **statuts complexes** (EN_ATTENTE, VALIDE, REFUSE, etc.).
- Appliquer un **filigrane dynamique** protégeant les contenus non payés.
- Concevoir un **module de feedback efficace** pour gérer les refus et améliorations.
- Intégrer **Stripe Checkout** avec mise à jour automatique du statut.
- Optimiser les requêtes SQL pour un suivi fiable des améliorations successives.

Grâce à une approche agile, des tests continus et une bonne collaboration, une version stable et fonctionnelle a pu être livrée à la fin du sprint.

Conclusion

Ce troisième sprint a transformé Market Bridge en une plateforme complète de gestion de contenu collaboratif entre marques et créateurs.

Les principales avancées :

- Un **workflow centralisé** de validation et feedback.
- La **protection des créations** grâce au filigrane.
- Un **paiement sécurisé** via Stripe intégré dans le processus.
- Une **communication fluide** et traçable via WebSocket.
- Une **boucle d'amélioration** pour garantir la qualité finale des contenus.

CHAPITRE 6 : ÉTUDE ET RÉALISATION DU QUATRIÈME SPRINT

Introduction

Ce sprint a marqué l'intégration du moteur de **recommandation intelligent** dans la plateforme **Market Bridge**.

Développé sous forme de **microservice IA avec Flask (Python)**, ce module aide les marques à identifier les créateurs les plus pertinents pour leurs campagnes, en fonction de plusieurs critères.

Grâce à cette brique intelligente, la plateforme propose désormais des recommandations personnalisées, renforçant la pertinence des collaborations.

Les principaux objectifs du sprint étaient :

- Intégrer une **IA de recommandation** basée sur les données des créateurs (type, pays, genre, note moyenne, performance).
- Assurer une **communication fluide** entre Spring Boot, Flask et MySQL.
- Proposer une interface intuitive pour que les marques puissent **consulter et filtrer les recommandations**.
- Afficher rapidement un **Top 5** des créateurs les mieux notés, avec un score et une étiquette de performance (Excellent, Bon, Moyen).

6.1 Sprint Backlog

Tableau 25: Sprint Backlog du quatrième sprint

ID	User Story	Priorité
US-01	En tant que marque, je veux consulter une liste de créateurs recommandés par l'IA.	Haute
US-02	En tant que système, je veux analyser les profils pour calculer les correspondances.	Haute
US-03	En tant que marque, je veux filtrer les résultats par genre, pays et type.	Moyenne

US-04	En tant que marque, je veux voir le Top 5 des créateurs les plus compatibles.	Haute
US-05	En tant qu'administrateur, je veux que l'IA reste synchronisée avec les données des créateurs.	Moyenne

6.2 Spécification fonctionnelle du quatrième sprint

Le moteur de recommandation fonctionne comme un **microservice Flask** connecté à la base de données MySQL contenant les profils des créateurs.

Chaque profil inclut : type (pro ou formé), genre, pays, note moyenne, et taux de réussite sur les collaborations précédentes.

Lorsqu'une marque accède à la section **Recommandations IA** :

- Le backend Spring Boot envoie une **requête REST** au **microservice Flask**, avec les critères sélectionnés.
- Flask traite les données en appliquant un algorithme de similarité, basé notamment sur la note moyenne et les critères choisis.
- Il retourne au backend les **5 créateurs les plus compatibles**.
- Le frontend (Angular) affiche les résultats sous forme de **cartes visuelles** avec score et étiquette de performance.

6.2.1 Diagramme de cas d'utilisation du quatrième sprint

Le diagramme ci-dessous présente le principal cas d'utilisation de ce sprint, où l'acteur *Marque*, une fois connecté, interagit avec le système pour consulter les recommandations IA.

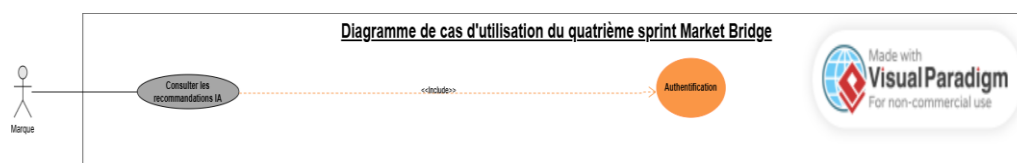


Figure 65: Diagramme de cas d'utilisation du quatrième sprint (Market Bridge)

6.2.2 Description textuelle du cas d'utilisation

6.2.2.1 Cas d'utilisation : Consultation les recommandations IA

Tableau 26: Description textuelle des cas d'utilisation - Consulter les recommandations IA

Élément	Description
Cas d'utilisation	Consulter les recommandations IA
Acteurs	Marque, Système (Flask + Spring Boot)
Pré-condition	La marque est connectée à la plateforme.
Post-condition	Le système affiche une liste de créateurs recommandés selon les critères choisis.
Scénario nominal	1. La marque accède à la section "Recommandations IA". 2. Le système affiche des filtres : genre, pays et type de créateur. 3. L'utilisateur clique sur "Générer mes créateurs". 4. Le backend Spring Boot envoie la requête au service Flask. 5. Flask renvoie le Top 5 des créateurs selon les critères et leur score. 6. L'interface affiche les créateurs classés (Excellent, Bon, Moyen) avec leur note moyenne.
Scénario d'exception	Si aucun créateur ne correspond aux critères, un message s'affiche : "Aucun résultat trouvé pour cette sélection."

6.3 Réalisation du sprint

L'intégration du **moteur IA dans Market Bridge** s'est appuyée sur trois composantes principales :

- **Flask (Python 3.10)** : héberge le microservice IA, traite les requêtes et calcule les correspondances à partir des données MySQL.
- **Spring Boot** : fait le lien entre le frontend Angular et le service Flask, en transmettant les critères de recherche et en récupérant les résultats.
- **MySQL** : stocke les profils des créateurs et les notes issues des feedbacks.

Étapes clés de développement :

- **Création de l'API Flask :**
 - Endpoint /recommender recevant les critères (genre, pays, type) via une requête POST.
 - Calcul du score de compatibilité pour chaque créateur.
 - Envoi d'une réponse JSON avec les 5 meilleurs profils, leur score et une étiquette de performance.
- **Intégration Spring Boot ↔ Flask :**
 - Appel REST via RestTemplate.
 - Gestion des délais et erreurs de réponse.
- **Affichage sur Angular :**
 - Composant recommendation.component.ts **avec filtres dynamiques** et bouton « Générer mes créateurs ».
 - Affichage sous forme de cartes avec nom, type, pays, note et badge de performance coloré.

6.3.1 Interfaces réalisées

6.3.1.1 Interface sans filtre (recommandations globales)

L'interface affiche le **Top 5 des créateurs les mieux notés** selon les performances récentes, sans critère particulier.

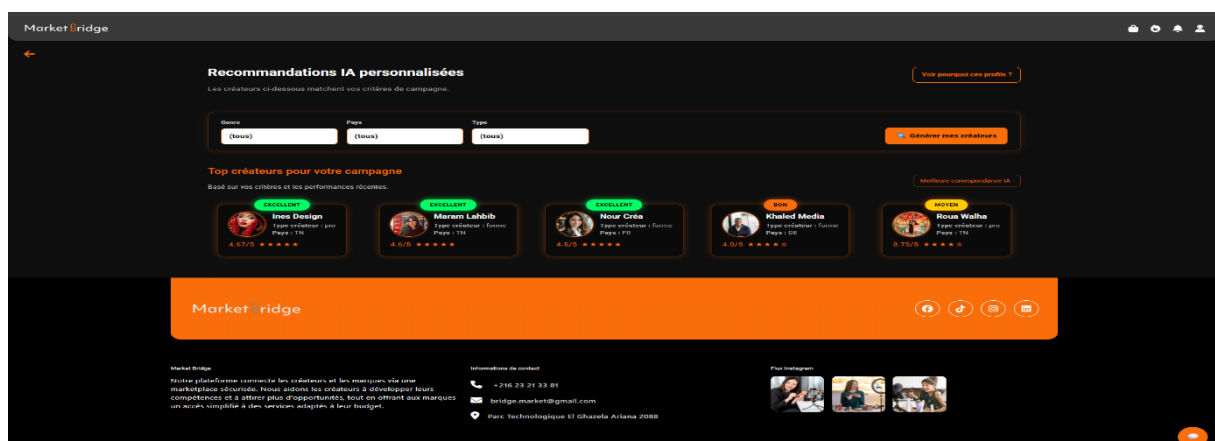


Figure 66: Interface des recommandations IA : Top 5 global

6.3.1.2 Interface avec filtres (genre, pays, type)

Permet aux marques de filtrer les résultats par genre, pays et type (ex. créateurs professionnels tunisiens).

Chaque carte contient :

- Nom
- Type de créateur
- Pays
- Note moyenne (sur 5)
- Badge de performance : *Excellent*, *Bon* ou *Moyen*

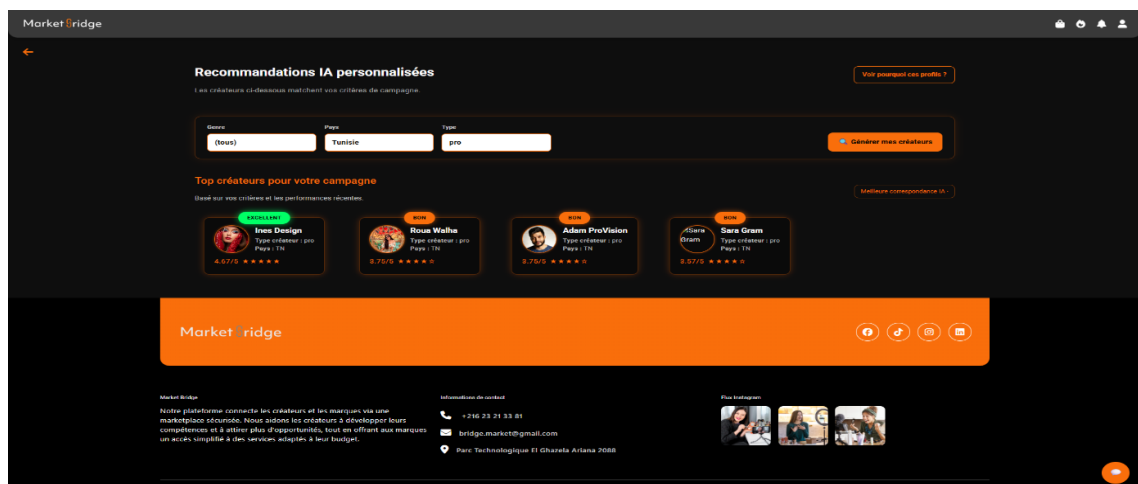


Figure 67: Interface des recommandations IA filtrée

6.3.2 Implémentation du microservice de recommandation IA (Flask)

Implémentation du moteur de recommandation intelligent conçu en Python à l'aide du framework **Flask**.

Les figures ci-dessous illustrent de manière concrète la **logique IA**, la **communication avec la base de données** ainsi que le **calcul des scores**.

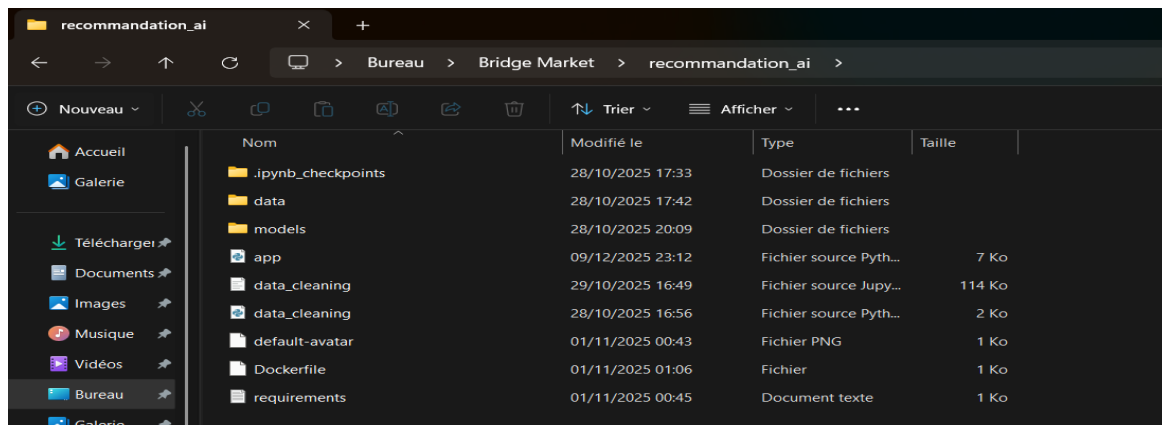


Figure 68: Structure globale du microservice IA

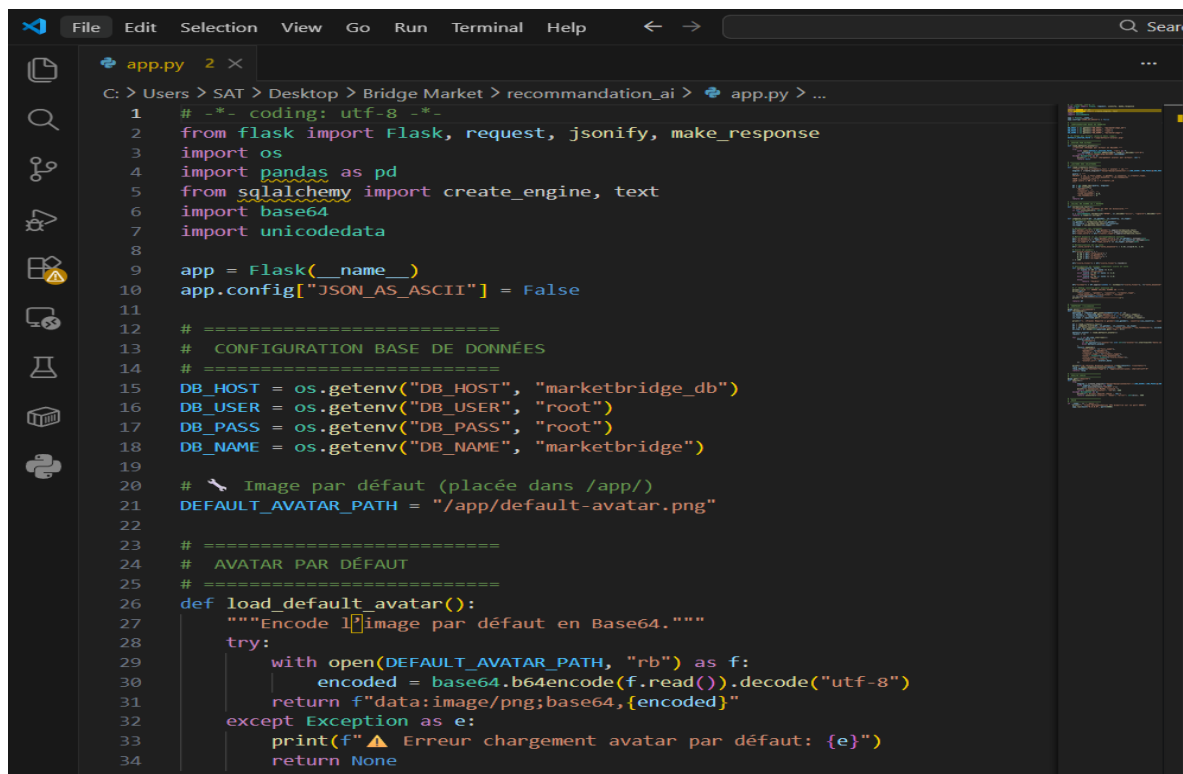


Figure 69: Initialisation et configuration du microservice de recommandation IA (Flask)

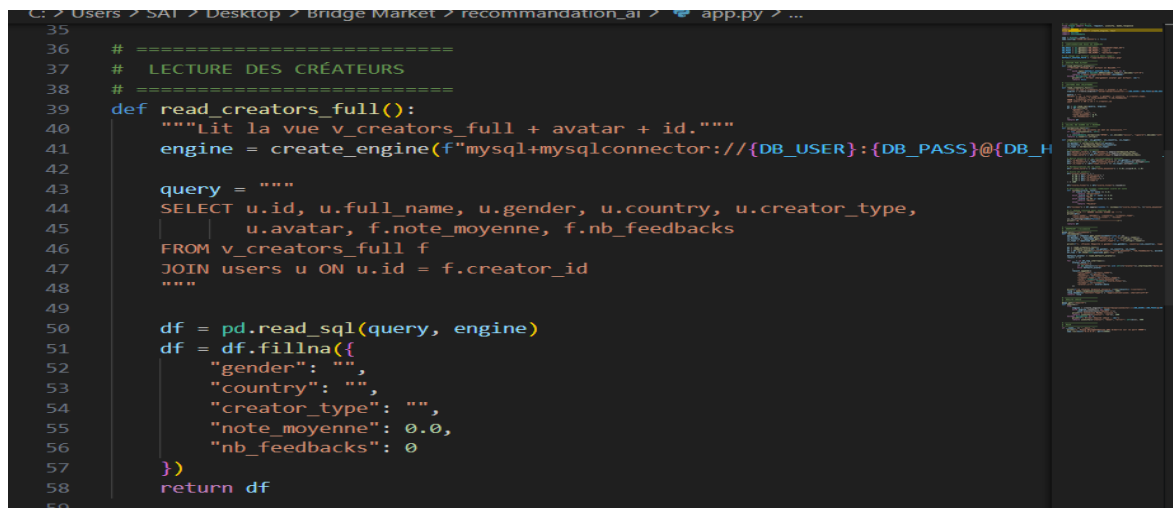


Figure 70: Lecture et préparation des données créateurs depuis la base MySQL

```

1 import pandas as pd
2 import mysql.connector
3 from sqlalchemy import create_engine
4
5 # 1 Connexion MySQL (dans Docker)
6 engine = create_engine('mysql+mysqlconnector://root:root@localhost:3306/marke
7
8 # 2 Lecture directe depuis MySQL
9 query = """
10 SELECT id, full_name, email, country, type, feedback_score
11 FROM users
12 WHERE role = 'CREATEUR';
13 """
14 df = pd.read_sql(query, con=engine)
15
16 # 3 Nettoyage de base
17 print("Avant nettoyage :", df.shape)
18
19 # Supprimer les doublons
20 df = df.drop_duplicates()
21
22 # Supprimer les valeurs manquantes
23 df = df.dropna()
24
25 # Normaliser le texte
26 df['country'] = df['country'].str.strip().str.title()
27 df['type'] = df['type'].str.upper()
28
29 # Convertir le score en float
30 df['feedback_score'] = pd.to_numeric(df['feedback_score'], errors='coerce').f
31
32 print("Après nettoyage :", df.shape)
33 print(df.head())
34
35 # 4 Export CSV propre
36 df.to_csv('data/clean_creators.csv', index=False, encoding='utf-8')
37
38 print("✅ Données nettoyées enregistrées dans data/clean_creators.csv")
39

```

Figure 71: Nettoyage et normalisation des données d'entrée pour le moteur IA

```

60 # =====
61 # CALCUL DU SCORE IA + NIVEAU
62 # =====
63 def normalize_text(s):
64     """Supprime les accents et met en minuscule."""
65     if not isinstance(s, str):
66         return ""
67     s = unicodedata.normalize("NFKD", s).encode("ascii", "ignore").decode("ut
68     return s.lower().strip()
69
70 def compute_score(df, in_gender, in_country, in_type):
71     # Normaliser les filtres reçus
72     in_gender = normalize_text(in_gender)
73     in_country = normalize_text(in_country)
74     in_type = normalize_text(in_type)
75
76     # Normaliser les colonnes
77     df["gender_norm"] = df["gender"].apply(normalize_text)
78     df["country_norm"] = df["country"].apply(normalize_text)
79     df["type_norm"] = df["creator_type"].apply(normalize_text)
80
81     # Match binaire (1 si correspondance exacte)
82     df["m_gender"] = (df["gender_norm"] == in_gender).astype(int)
83     df["m_country"] = (df["country_norm"] == in_country).astype(int)
84     df["m_type"] = (df["type_norm"] == in_type).astype(int)
85
86     # Normalisation de la note
87     df["note_norm"] = (df["note_moyenne"] / 5.0).clip(0.0, 1.0)
88
89     # Score IA pondéré
90     df["score_final"] = (
91         0.50 * df["note_norm"] +
92         0.20 * df["m_gender"] +
93         0.20 * df["m_country"] +
94         0.10 * df["m_type"]
95     ) * 100
96
97     df["score_final"] = df["score_final"].round(1)
98
99     # Attribution du niveau combinant score et note
100     def niveau(score, note):
101         if score >= 85 or note >= 4.5:
102             return "Excellent"
103         elif score >= 65 or note >= 4.0:
104             return "Bon"
105         elif score >= 45 or note >= 3.0:
106             return "Moyen"
107         else:
108             return "Faible"
109
110     df["niveau"] = df.apply(lambda r: niveau(r["score_final"], r["note_moyenn
111
112     # Debug console pour analyse IA
113     print("\n🔍 --- DEBUG CALCUL SCORE IA ---")
114     print(df[["
115         "full_name", "gender", "country", "creator_type",
116         "note_moyenne", "score_final", "niveau"
117     ]].to_string(index=False))
118     print("🔍 -----")
119
120     return df

```

Figure 72: Algorithme de calcul du score de compatibilité entre marque et créateur

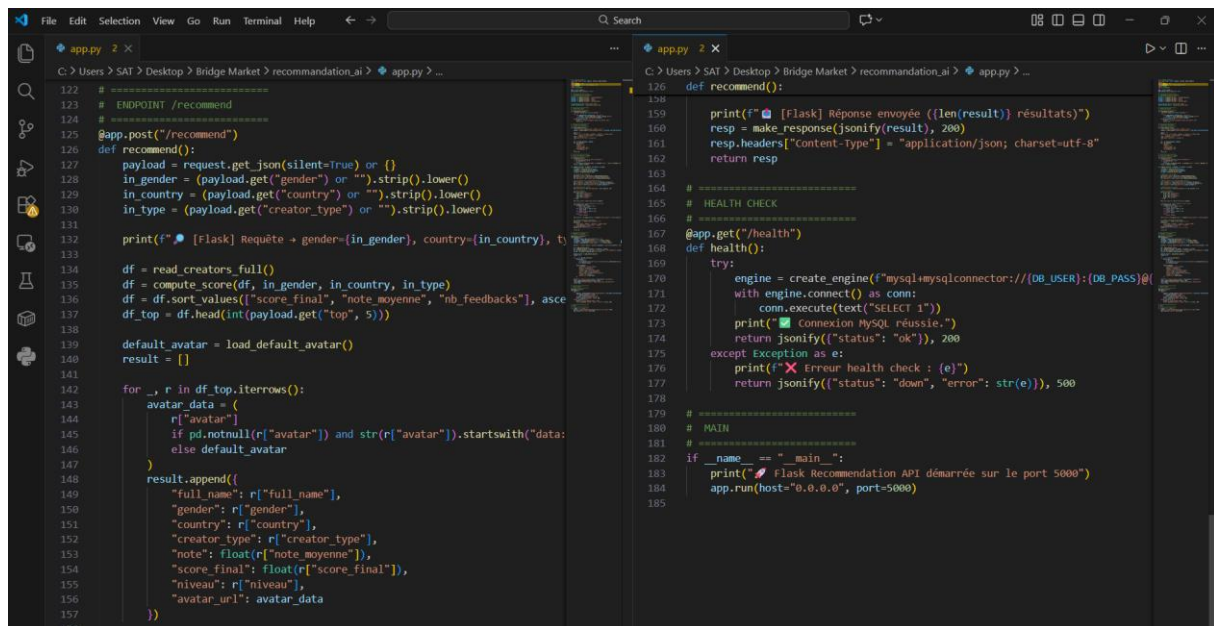


Figure 73: Endpoint REST /recommend et génération des résultats IA (Top 5)

Ces figures illustrent l'implémentation complète du moteur de recommandation, depuis la préparation des données jusqu'au calcul du score final et à l'exposition des résultats via une API REST. Cette approche garantit un moteur IA modulaire, explicable et facilement extensible.

6.4 Revue du quatrième sprint

Une revue a été organisée avec le Scrum Master et le Product Owner pour évaluer la qualité du moteur IA.

Les résultats ont été jugés très satisfaisants :

- Communication fluide entre Spring Boot et Flask.
- Recommandations générées rapidement (moins de 2 secondes).
- Résultats cohérents et personnalisés.
- Interface intuitive, bien intégrée dans l'univers visuel de la plateforme.

Le Product Owner a particulièrement apprécié la pertinence des suggestions et la clarté de l'interface.

6.5 Rétrospective du quatrième sprint

Malgré une livraison réussie, certains défis ont été relevés :

- Optimisation du temps de réponse du service Flask.
- Uniformisation de données créateurs pour améliorer la fiabilité des scores.

- Configuration du CORS et des headers sécurisés entre Flask et Spring Boot.

Ces points ont été résolus grâce à une collaboration étroite entre les équipes backend et IA, avec des tests réguliers et une validation continue.

Conclusion

Ce sprint marque une avancée majeure avec l'intégration d'un moteur de recommandation intelligent.

Grâce à l'analyse des données créateurs et à une IA performante, les marques bénéficient désormais d'un système de matching rapide, pertinent et personnalisé.

Ce module renforce le positionnement de Market Bridge comme une plateforme intelligente, évolutive et pilotée par les données préparant l'arrivée de futures fonctionnalités analytiques et prédictives.

CHAPITRE 7: ÉTUDE ET RÉALISATION DU CINQUIÈME SPRINT

Introduction

Ce sprint a été consacré au développement d'un **chatbot intelligent**, intégré localement à la plateforme Market Bridge. Il permet aux marques de dialoguer avec un assistant virtuel pour générer automatiquement des offres de service au format PDF.

Contrairement aux solutions cloud dépendant d'API payantes (comme OpenAI ou DialogFlow), ce chatbot fonctionne entièrement hors ligne grâce à **FastAPI** et **spaCy**, garantissant une exécution rapide, une sécurité maximale et une totale indépendance vis-à-vis de services externes.

Ce module combine **IA, génération de documents** et intégration sécurisée avec la base de données **MySQL** et le système d'authentification **JWT** de la plateforme, constituant ainsi une vraie valeur ajoutée.

7.1 Sprint Backlog

Tableau 27: Sprint Backlog du cinquième sprint

ID	User Story	Priorité
US5.1	En tant que marque, je veux discuter avec un assistant virtuel pour générer une offre de service.	Haute
US5.2	En tant que marque, je veux que le chatbot comprenne mes instructions en langage naturel.	Haute
US5.3	En tant que marque, je veux obtenir un PDF personnalisé avec mes informations, thèmes et identités visuelles.	Haute
US5.4	En tant qu'administrateur, je veux un chatbot offline sans dépendance à des API externes.	Haute
US5.5	En tant qu'utilisateur, je veux que le chatbot reconnaisse mon identité grâce au token JWT.	Moyenne

7.2 Spécification fonctionnelle du cinquième sprint

Le chatbot permet aux marques de créer leurs offres sans intervention humaine. Il comprend les instructions en français grâce à **spaCy**, génère un document PDF professionnel via **ReportLab**, et l'associe automatiquement à l'utilisateur connecté via JWT.

Fonctionnalités clés :

- **Compréhension du langage naturel (NLP)** via le modèle `fr_core_news_md` de `spaCy`.
- **Identification automatique** de l'utilisateur via JWT (nom, avatar, rôle).
- **Génération PDF** contenant : nom, avatar, pack choisi, couleur dominante, thème et objectif.
- **Fonctionnement 100 % local** et sécurisé grâce à FastAPI.

Architecture technique

- **Frontend** : Angular (interface de chat, visualisation du PDF).
- **Backend principal** : Spring Boot (authentification et récupération des infos utilisateur).
- **Backend IA** : FastAPI (Python).
- **NLP** : `spaCy`.
- **PDF** : `ReportLab`.
- **Base de données** : MySQL (utilisateurs, historique des conversations).

7.2.1 Diagramme de cas d'utilisation du cinquième sprint

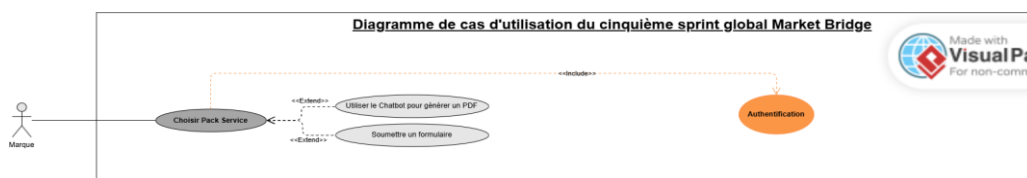


Figure 74: Diagramme de cas d'utilisation du cinquième sprint (Market Bridge)

L'acteur **Marque** peut, après authentification sécurisée :

Choisir un pack de service

- Remplir un formulaire ou soumettre une demande
- Utiliser le **chatbot IA** pour générer automatiquement un **document PDF** personnalisé

Le tout dans un environnement sécurisé et offline, connecté à la plateforme.

7.2.2 Description textuelle des cas d'utilisation

Tableau 28: Description textuelle des cas d'utilisation : Utiliser le Chatbot IA (Marque)

Cas d'utilisation	Description
Nom	Utiliser le Chatbot IA pour générer une offre personnalisée
Acteur principal	Marque
Pré-condition	L'utilisateur (marque) est connecté à la plateforme via JWT.
Post-condition	Le fichier PDF est généré et disponible dans la fenêtre du chatbot.
Scénario nominal	1. La marque clique sur l'icône du chatbot. 2. Le chatbot s'ouvre et salue la marque par son nom et avatar. 3. La marque saisit une instruction libre (ex. : "Je veux un post beauté fond orange pour le pack basique"). 4. Le message est transmis au serveur FastAPI. 5. spaCy détecte les entités : type de contenu, thème, couleur, pack. 6. Le chatbot confirme les choix et demande validation. 7. Après confirmation, ReportLab génère un PDF d'offre de service.
Scénario d'exception	- Si spaCy ne comprend pas la requête → message : <i>"Je n'ai pas bien compris, pouvez-vous reformuler ?"</i>. - Si le PDF ne peut être généré → notification d'erreur.

7.3 Réalisation du sprint

L'intégration du chatbot IA local dans Market Bridge s'est appuyée sur une architecture distribuée reposant sur trois composants principaux :

- **FastAPI (Python 3.10)** : héberge le microservice NLP, traite les requêtes textuelles et génère dynamiquement des fichiers PDF.
- **Spring Boot** : fait le lien entre Angular et FastAPI, en transmettant les messages et en recevant les offres générées.
- **MySQL** : stocke les informations liées à la marque (nom, avatar, historique de conversation, etc.).

Étapes clés de développement

- **Création du microservice FastAPI :**
 - Endpoint `/generate-offer` pour recevoir les requêtes du chatbot.
 - Traitement NLP avec **spaCy** (`fr_core_news_md`) pour extraire le thème, le type de contenu, la couleur, le pack, etc.
 - Génération automatique d'un PDF personnalisé via **ReportLab**, selon les données extraites.
 - Connexion à **MySQL** pour récupérer les informations du brand connecté.
- **Traitement linguistique (NLP) :**
 - Reconnaissance de formulations variées grâce à une gestion des synonymes.
 - Analyse des entités-clés nécessaires à la génération du document (ex : “post beauté”, “fond orange”, etc.).
- **Génération de l'offre PDF :**
 - Structure visuelle intégrée : avatar, logo Market Bridge, en-tête, tableau de synthèse, signature.
 - Export automatique au format `offre_service_{brand}.pdf`.
- **Intégration frontend (Angular) :**
 - Composant `chat-window.component.ts`.
 - Interface conversationnelle intégrée à la page de sélection de pack.
 - Communication avec FastAPI via `HttpClient` et téléchargement automatique du fichier PDF généré.

7.3.1 Interfaces réalisées

7.3.1.1 Interface conversationnelle

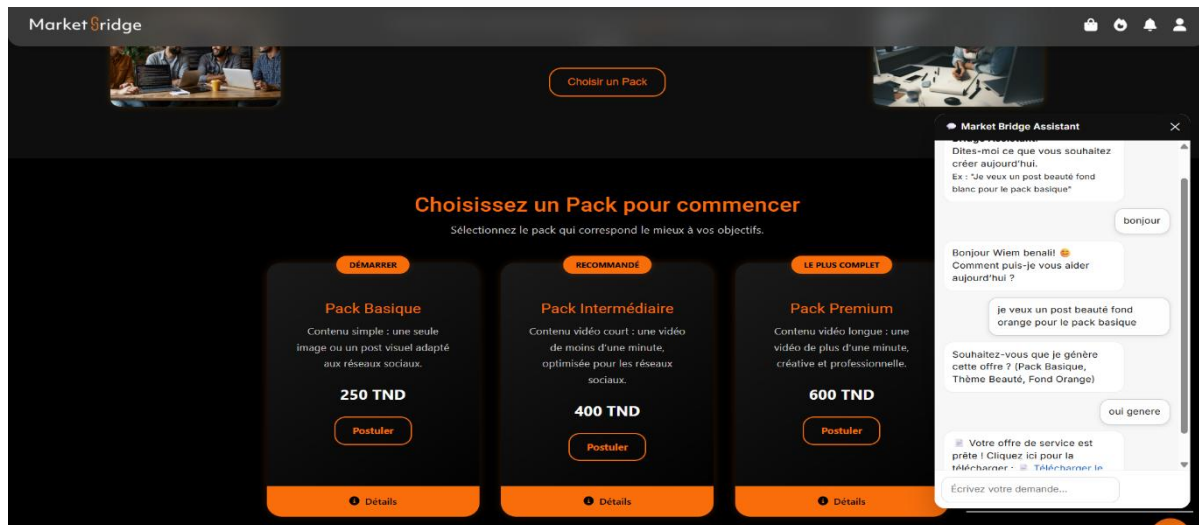


Figure 75: Exemple de conversation avec le chatbot

L'utilisateur saisit par exemple:

“Je veux un post beauté fond orange pour le pack basique”

Le chatbot interprète le message, reformule la demande, puis propose de générer une offre personnalisée.

Si l'utilisateur confirme, le fichier PDF est automatiquement généré.

7.3.1.2 Interface de visualisation PDF



Figure 76: Exemple d'offre PDF générée

Le document PDF contient :

- L’avatar de la marque
- Le nom du brand
- Le pack sélectionné
- Le thème, la couleur dominante, l’objectif
- La date limite
- Une mention de copyright: “© Market Bridge – Document généré automatiquement”

7.3.2 Implémentation du microservice Chatbot IA (FastAPI)

- Organisation du microservice Chatbot IA montrant la séparation des ressources, des fichiers sources Python, des dépendances et des fichiers générés (PDF).

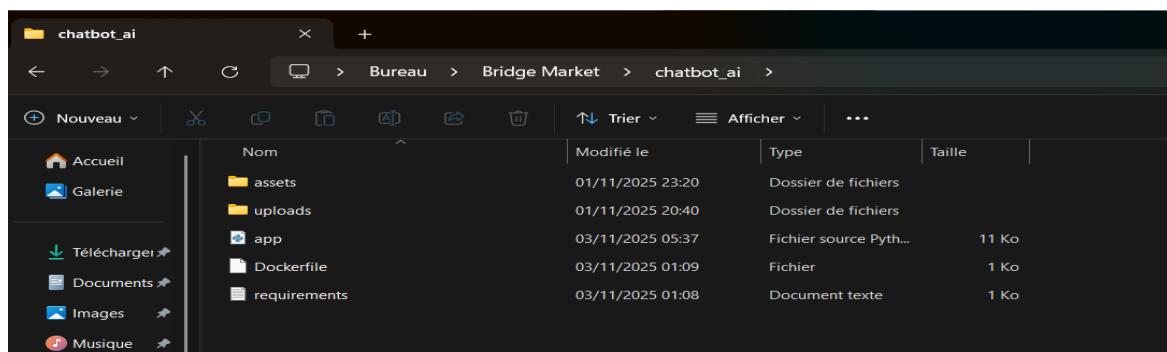


Figure 77: Structure du microservice chatbot IA (dossier chatbot_ai)

- Initialisation du microservice FastAPI, configuration CORS, chargement du modèle NLP spaCy et paramètres de connexion aux bases de données.

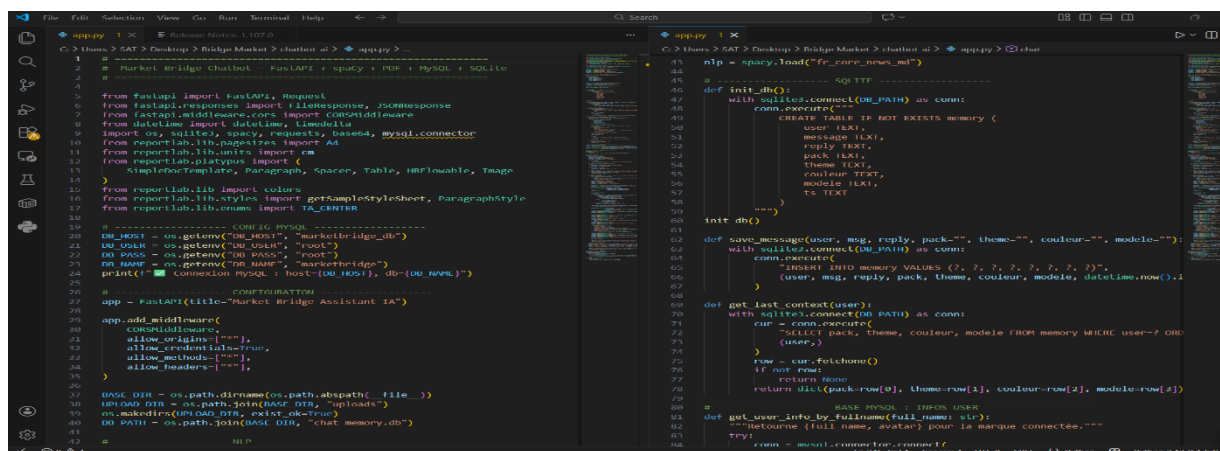


Figure 78: Initialisation du microservice Chatbot IA et configuration FastAPI

- Implémentation de la mémoire conversationnelle permettant au chatbot de conserver le contexte utilisateur (pack, thème, couleur, modèle) entre plusieurs interactions.

```

79
80 ----- BASE MYSQL : INFOS USER -----
81 def get_user_info_by_fullname(full_name: str):
82     """Retourne {full_name, avatar} pour la marque connectée."""
83     try:
84         conn = mysql.connector.connect(
85             host=DB_HOST,
86             user=DB_USER,
87             password=DB_PASS,
88             database=DB_NAME
89         )
90         cur = conn.cursor(dictionary=True)
91
92         # Recherche flexible : full_name exact, simplifié, ou email
93         cur.execute("""
94             SELECT full_name, avatar
95             FROM users
96             WHERE LOWER(full_name) = LOWER(%s)
97                OR LOWER(REPLACE(full_name, ' ', '')) = LOWER(REPLACE(%s, ' ', ''))
98                OR LOWER(email) = LOWER(%s)
99             LIMIT 1
100         """, (full_name, full_name, full_name))
101
102         row = cur.fetchone()
103         conn.close()
104
105         if row:
106             print(f"Utilisateur trouvé : {row['full_name']}")
107         else:
108             print(f"Aucun utilisateur trouvé pour : {full_name}")
109         return row
110
111     except Exception as e:
112         print(f"Erreur MySQL:", e)
113         return None
114
115 ----- STYLES PDF -----

```

Figure 79: Gestion du contexte conversationnel et persistance des échanges

- Logique de traitement des messages utilisateurs, analyse linguistique, gestion des intentions et construction dynamique des réponses du chatbot IA.

```

115 # Styles PDF
116 styles = getSampleStyleSheet()
117 TITLE = ParagraphStyle("title0", parent=styles["title"], alignment=TA_CENTER,
118     font_size=22, text_color=colors.hexcolor("#f96000"))
119 TEXT = ParagraphStyle("Text0", parent=styles["normal"], font_size=12, leading=
120     FOOTER = ParagraphStyle("Foot0", parent=styles["normal"], alignment=TA_CEN
121     font_size=9, text_color=colors.grey)
122
123 # ----- HEALTH -----
124 @app.get("/health")
125 def health():
126     return {"status": "ok", "time": datetime.utcnow().isoformat()}
127
128 # ----- CHAT -----
129 @app.post("/chat")
130 async def chat(req: Request):
131     data = await req.json()
132     message_raw = data.get("message", "")
133     message = message_raw.lower().strip()
134     user_key = data.get("user", "Utilisateur")
135
136     if not message:
137         return JSONResponse({"error": "Message vide"}, status_code=400)
138
139     # Lecture du profil MySQL
140     ui = get_user_info_by_fullname(user_key)
141     if ui:
142         full_name = ui.get("full_name") or "Utilisateur"
143         avatar_url = ui.get("avatar")
144     else:
145         full_name = user_key
146         avatar_url = data.get("avatar") or None
147
148     # Salutation
149     if any(w in message for w in ["bonjour", "salut", "hey", "coucou", "hi"]):
150         reply = f"Bonjour {full_name}! Comment puis-je vous aider aujourd'hui?"
151         save_message(user_key, message_raw, reply)
152         return {"reply": reply}
153
154 # Génération PDF
155 if any(x in message for x in ["oui", "vas-y", "ok", "genere", "génère", "
156     ctx = get_last_context(user_key)
157     if not ctx:
158         reply = "Je n'ai pas encore les détails du brief, pouvez-vous les
159         save_message(user_key, message_raw, reply)
160         return {"reply": reply}
161
162     pack = ctx.get("pack", "Basique")
163     theme = ctx.get("theme", "Bleuté")
164     couleur = ctx.get("couleur", "Blanc")
165     modele = ctx.get("modele", "Post Instagram")
166
167     safe_user = "".join(c for c in full_name if c.isalnum() or c in (" ",
168     pdf_name = f"offre_{safe_user}_{datetime.now().strftime('%Y%m%d_%H%M
169     pdf_path = os.path.join(UPLOAD_DIR, pdf_name)
170
171     doc = SimpleDocTemplate(pdf_path, pagesize=A4,
172         leftMargin=2*cm, rightMargin=2*cm,
173         topMargin=2*cm, bottomMargin=2*cm)
174
175     elements = []
176
177     # Avatar + Date ---
178     avatar_path = None
179     if avatar_url:
180         try:
181             avatar_path = os.path.join(UPLOAD_DIR, f"avatar_{safe_user}.p
182             if avatar_url.startswith("data:image"):
183                 header, encoded = avatar_url.split(",", 1)
184                 with open(avatar_path, "wb") as f:
185                     f.write(base64.b64decode(encoded))
186             else:
187                 r = requests.get(avatar_url)
188                 if r.status_code == 200:
189                     with open(avatar_path, "wb") as f:
190                         f.write(r.content)
191         except Exception as e:
192             print(f"Erreur de chargement avatar:", e)

```

Figure 80: Traitement NLP et logique conversationnelle du Chatbot IA

- Génération automatique d'un document PDF personnalisé à l'aide de ReportLab et mise à disposition via un endpoint REST sécurisé.

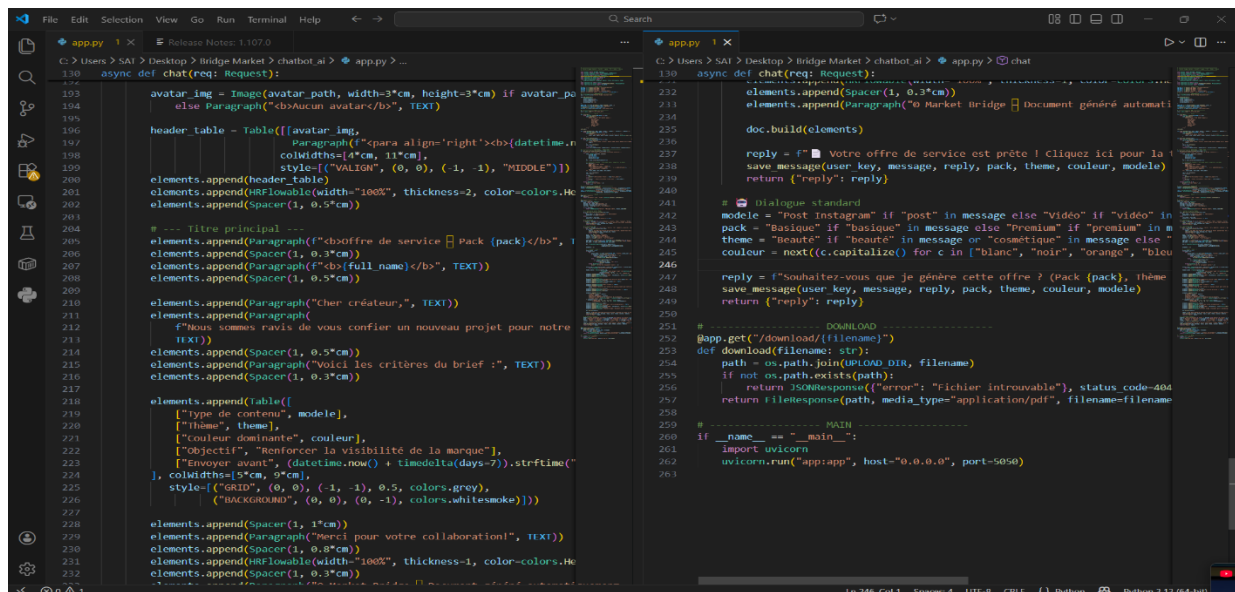


Figure 81: Génération dynamique des offres PDF et téléchargement

7.4 Revue du cinquième sprint

Une revue a été organisée avec le Scrum Master et le Product Owner afin d'évaluer les fonctionnalités livrées.

Résultats validés :

- Fonctionnement **100 % offline** sans dépendance à des services tiers (OpenAI, Dialogflow, etc.)
- Interprétation fiable des instructions en français grâce à **spaCy**
- Génération de fichiers PDF professionnels avec contenu structuré
- Reconnaissance automatique de l'identité de la marque (via JWT)
- Interface fluide et bien intégrée à l'expérience utilisateur

Le Product Owner a particulièrement salué l'autonomie du module, la fluidité des échanges et la valeur ajoutée apportée à la plateforme.

7.5 Rétrospective du cinquième sprint

- Malgré la réussite de l'intégration, plusieurs défis techniques ont été rencontrés :
- **Optimisation du temps de génération des fichiers PDF**
- **Amélioration de la compréhension NLP** pour gérer diverses formulations
- **Support des caractères accentués** dans les documents générés avec ReportLab

Ces difficultés ont été surmontées grâce à une approche itérative, des tests fréquents et une communication continue entre les équipes backend, IA et frontend.

Conclusion

Ce cinquième sprint marque une étape majeure dans l'évolution de Market Bridge grâce à l'ajout d'un **chatbot IA local, intelligent et autonome**.

En combinant traitement du langage naturel (**spaCy**), génération de documents (**ReportLab**) et communication en temps réel (**FastAPI + Angular**), la plateforme permet désormais aux marques de créer automatiquement leurs offres de services via une simple discussion.

Ce module renforce l'automatisation, la sécurité des données et l'expérience utilisateur, tout en ouvrant la voie à d'autres fonctionnalités intelligentes à venir.

CHAPITRE 8 : ÉTUDE ET RÉALISATION DU SIXIÈME SPRINT

Introduction

Ce sprint a été consacré à l'intégration du module de **paiement sécurisé Stripe** et à la **protection des contenus validés** par filigrane.

L'objectif principal était de sécuriser l'ensemble du cycle de transaction :

- La **marque accepte** une proposition,
- Elle **note le créateur**,
- Elle **paye en ligne** (via Stripe en mode test),
- Puis elle peut **télécharger le contenu original** sans watermark.

Avant le paiement, toutes les images et vidéos sont affichées avec un filigrane dynamique “© *Market Bridge*”, empêchant leur téléchargement ou capture non autorisée.

8.1 Sprint Backlog

Le sprint s'est concentré sur trois axes :

- L'intégration de Stripe Checkout,
- La **protection des médias avant paiement**,
- Et la **gestion automatisée du statut des paiements**.

Tableau 29: Sprint Backlog du sixième sprint

ID	User Story	Priorité
US6.1	En tant que marque, je veux payer un contenu accepté via Stripe pour débloquer le fichier.	Haute
US6.2	En tant que système, je veux afficher le contenu avec watermark avant paiement.	Haute
US6.3	En tant que marque, je veux noter le créateur avant le paiement.	Moyenne

US6.4	En tant que système, je veux retirer le filigrane une fois le paiement confirmé.	Haute
US6.5	En tant qu'administrateur, je veux pouvoir suivre l'état des paiements.	Moyenne

Ce sprint a permis de connecter le frontend Angular, le backend Spring Boot et le service Stripe dans un **parcours fluide et sécurisé**, du visionnage du contenu protégé jusqu'au téléchargement final après paiement.

8.2 Spécification fonctionnelle du sixième sprint

Les fonctionnalités développées sont les suivantes :

- Intégration de **Stripe Checkout** (mode test) pour les paiements en ligne.
- Application d'un **filigrane dynamique** "© Market Bridge" aux contenus avant règlement.
- **Déblocage automatique** du fichier original après validation Stripe.
- Synchronisation du **statut, de la note et du feedback** du contenu avec la base de données.
- Redirection vers le téléchargement sécurisé après paiement confirmé.

Technologies utilisées :

- **Frontend** : Angular 15 (intégration Stripe.js, redirection, gestion UI)
- **Backend** : Spring Boot 3 (API REST */create-checkout-session*)
- **Base de données** : MySQL
- **Sécurité** : JWT (authentification rôle MARQUE)
- **Service externe** : Stripe Checkout (mode test)

8.2.1 Diagramme de cas d'utilisation du sixième sprint

Ce diagramme illustre les interactions entre les trois entités principales :

- La **marque**,
- La **plateforme Market Bridge**,
- Et le service **Stripe**.

L'utilisateur :

- Consulte le contenu protégé,
- L'évalue et fournit un feedback,
- Effectue un **paiement sécurisé** via Stripe,
- Puis **accède automatiquement** au fichier original sans watermark.

Ce fonctionnement garantit à la fois la **sécurité**, la **traçabilité** des paiements, et une **expérience utilisateur fluide**.

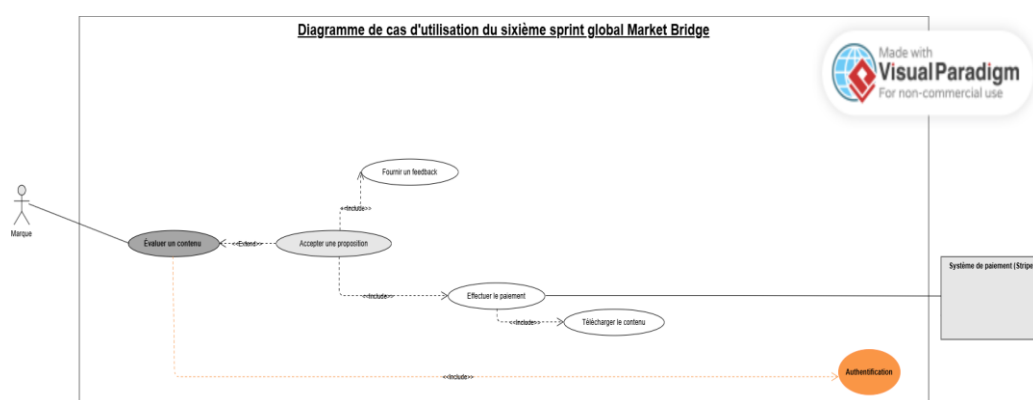


Figure 82: Diagramme de cas d'utilisation du sixième sprint

8.2.2 Description textuelle des cas d'utilisation

8.2.2.1 Cas d'utilisation : Payer un contenu validé via Stripe

Tableau 30: Description textuelle des cas d'utilisation - Payer un contenu validé via Stripe (Marque)

Élément	Description
Acteurs	Marque, Système Stripe, Système Market Bridge
Pré-condition	La marque a accepté la proposition de contenu et attribué une note au créateur.
Post-condition	Le paiement est validé, le contenu est débloqué et téléchargé automatiquement sans filigrane.

Scénario nominal	1. La marque clique sur le bouton “Payer ce pack” dans son espace “Mes propositions”. 2. Le système redirige vers Stripe Checkout. 3. La marque saisit les informations de sa carte (mode test) et valide le paiement. 4. Stripe confirme la transaction au backend Spring Boot. 5. Le système met à jour le statut de la notification en “VALIDÉE”. 6. Le fichier original (sans watermark) est automatiquement téléchargé. 7. Un message de confirmation s’affiche : “Paiement validé votre contenu est en cours de téléchargement...”.
Scénario d’exception	Si le paiement échoue ou est annulé, le contenu reste filigrané et le statut demeure “EN ATTENTE”.

8.3 Conception du sixième sprint

Ce sprint a porté sur la **mise en place du paiement sécurisé** via Stripe et la **protection des contenus** avec filigrane.

La conception s’est articulée autour de l’interaction entre quatre composants principaux :

- **Angular** : affichage des contenus avec watermark, saisie de la note, redirection vers Stripe Checkout.
- **Spring Boot** : création de la session de paiement, suivi du statut, mise à jour automatique après validation.
- **Stripe (API)** : gestion du paiement sécurisé (mode test).
- **MySQL** : stockage des transactions, notes et statuts associés.

8.3.1 Diagrammes de séquence

8.3.1.1 Paiement et téléchargement sécurisé

Ce diagramme décrit le parcours complet depuis la décision de paiement par la marque jusqu’à la réception du contenu final sans filigrane :

- La marque initie le paiement,
- Le backend contacte Stripe et crée une session,

- Une fois le paiement confirmé, la plateforme valide la transaction et déclenche **automatiquement** le téléchargement sécurisé.

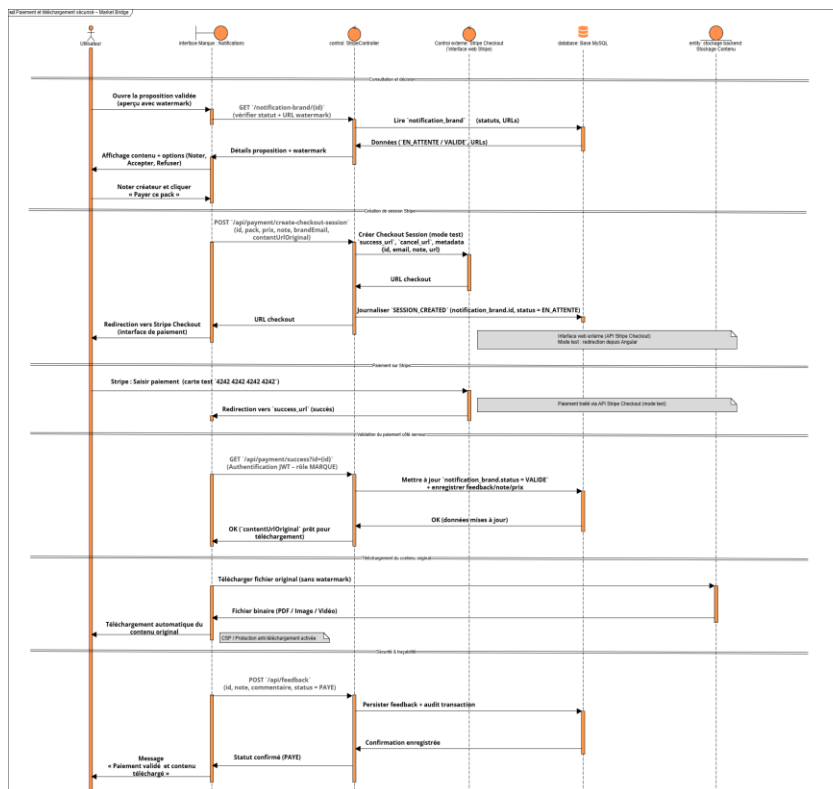


Figure 83: Diagramme de séquence – Paiement et téléchargement sécurisé (Market Bridge)

8.4 Réalisation du sprint

Une fois les objectifs définis, l'équipe a développé les modules nécessaires à la **sécurisation des contenus validés** et à l'**intégration de Stripe**.

Ce sprint visait à offrir une **expérience fluide**, tout en garantissant la **protection des créations** et la **traçabilité du paiement**.

Les principales actions réalisées :

- Permettre à la **marque de noter le créateur**,
- **Protéger** les contenus avec un watermark avant paiement,
- **Débloquer automatiquement** le fichier original après confirmation de paiement.

Technologies utilisées :

- **Frontend** : Angular (Stripe.js, interface utilisateur)
- **Backend** : Spring Boot (création de session Stripe, gestion du statut)

- **Base de données** : MySQL (transactions et feedbacks)
- **Sécurité** : JWT (authentification et rôles)

8.4.1 Interfaces réalisées

8.4.1.1 Interface de notation et paiement

La marque évalue le créateur, puis clique sur « **Payer ce pack** » pour accéder à Stripe Checkout.

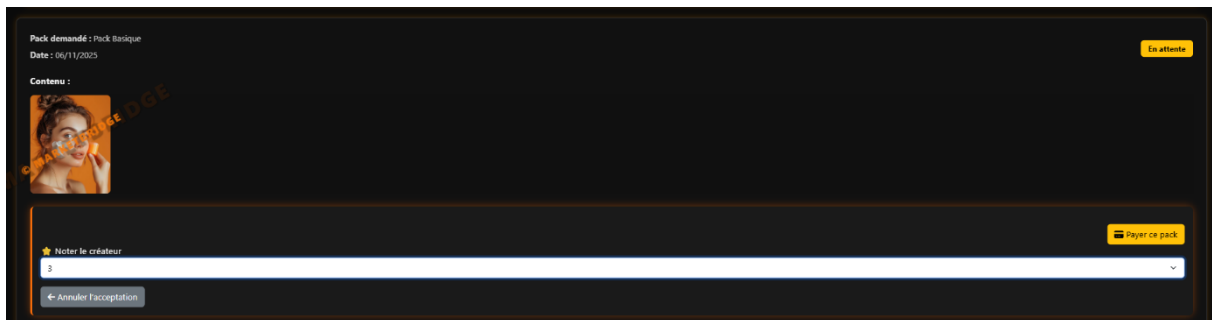


Figure 84: Interface de notation et paiement (Marque)

8.4.1.2 Interface de confirmation Stripe

Une fois la session créée, une page intermédiaire s'affiche avec un bouton menant à la passerelle de paiement.

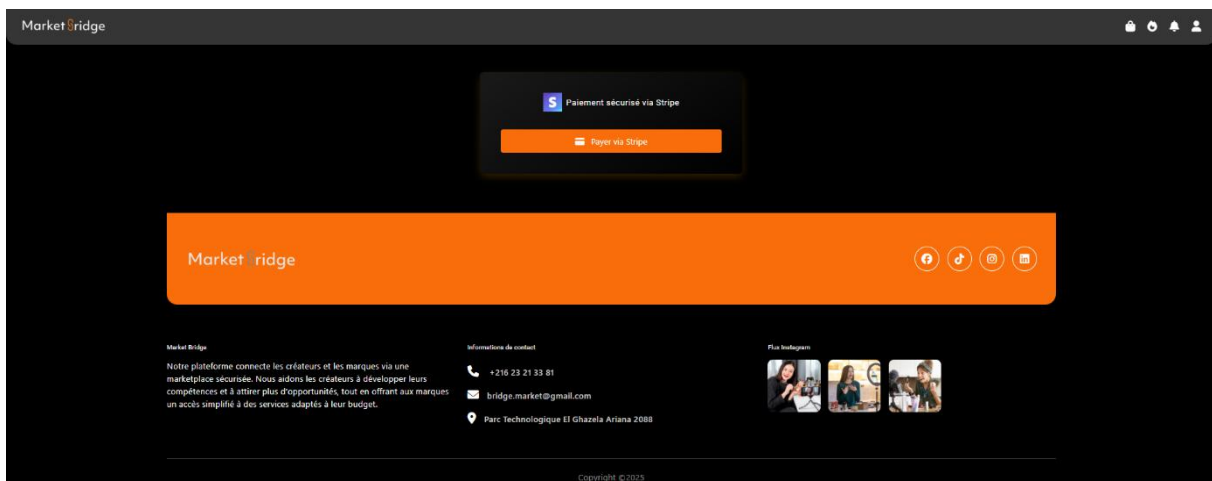


Figure 85: Interface de confirmation Stripe

8.4.1.3 Interface Stripe Checkout

Formulaire de paiement sécurisé affichant : nom du pack, prix, pays et email de la marque.
La carte virtuelle (4242...) peut être utilisée pour simuler la transaction.

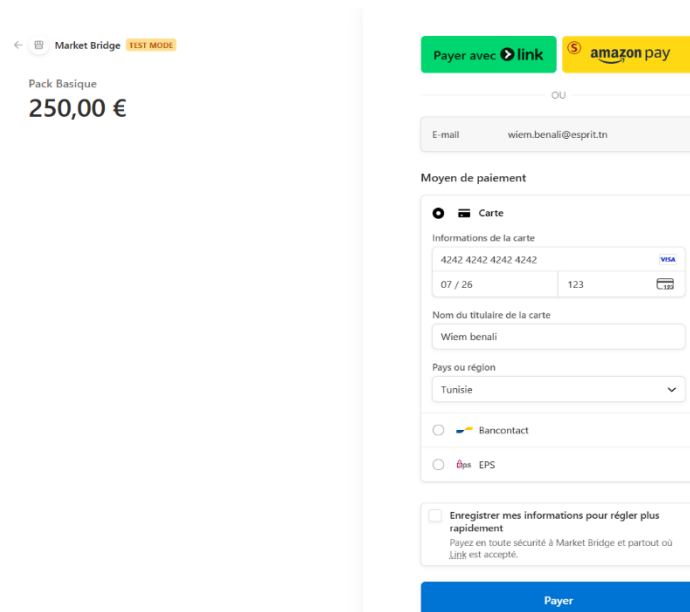


Figure 86: Interface Stripe Checkout (mode test)

8.4.1.4 Interface de téléchargement automatique

Après validation, un message s'affiche :

« **Paiement validé votre contenu est en cours de téléchargement...** »

Le fichier original (sans watermark) est alors téléchargé automatiquement.

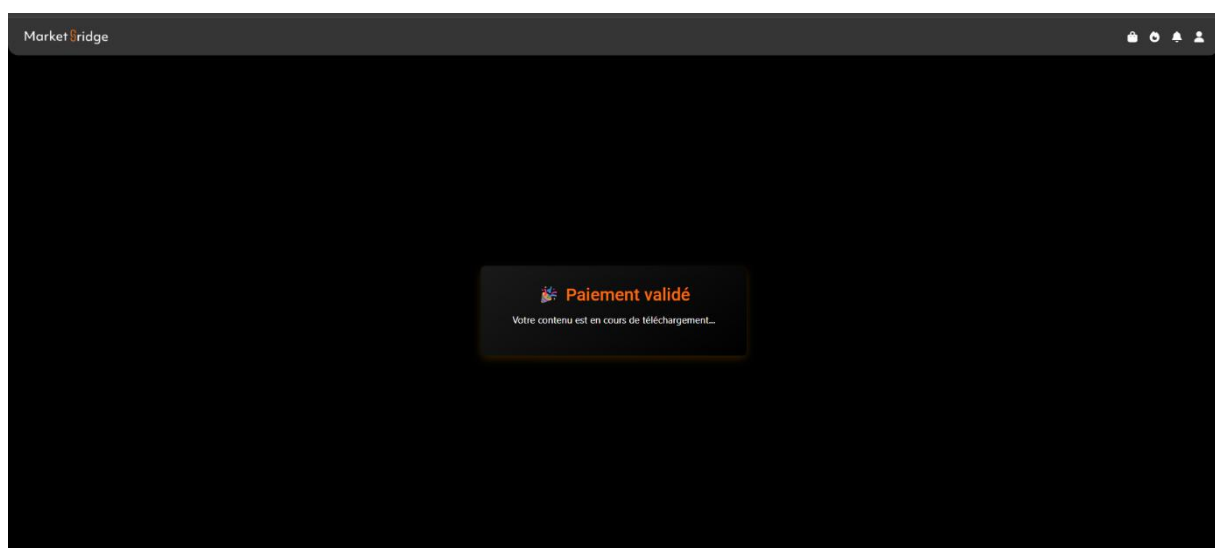


Figure 87: Interface de confirmation et téléchargement automatique

8.4.1.5 Interface de visualisation sécurisée avant paiement

Avant toute transaction, les contenus restent protégés par un filigrane « © **Market Bridge** », désactivant les tentatives de capture ou téléchargement.

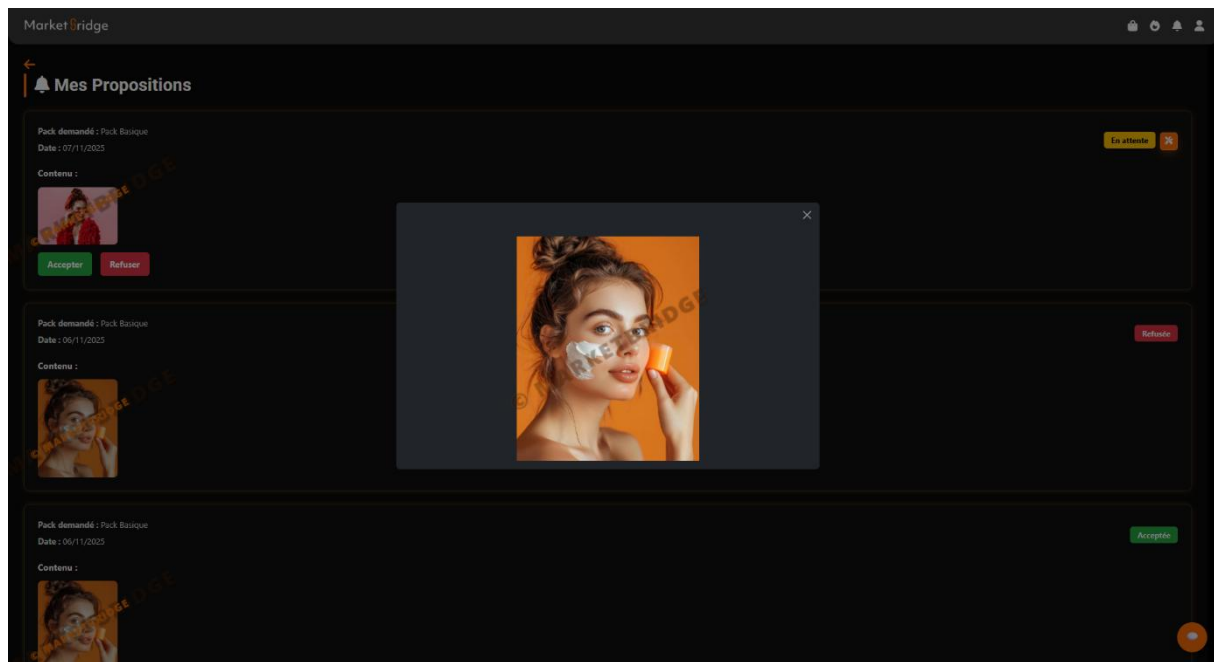


Figure 88: Interface de visualisation sécurisée avant paiement

8.5 Revue du sixième sprint

Une réunion de revue a été organisée avec le Scrum Master, le Product Owner et l'équipe de développement pour évaluer les livrables par rapport aux objectifs fixés.

Ce sprint visait principalement à intégrer un **système de paiement sécurisé via Stripe** et à **protéger les contenus validés** avec un filigrane dynamique.

Les fonctionnalités prévues ont été **implémentées, testées et validées avec succès**.

Les résultats obtenus :

- Intégration complète de **Stripe Checkout (mode test)**.
- Le **téléchargement automatique** du contenu original après validation du paiement.
- Protection efficace des fichiers par **watermark** avant paiement.
- Une **expérience utilisateur fluide, claire et sécurisée**, garantissant la confiance entre marque et créateur.

Le Product Owner s'est montré satisfait de la fiabilité du processus, de l'harmonie visuelle des interfaces et de l'intégration fluide entre Stripe, Spring Boot et Angular.

Ce sprint marque une **étape clé dans la monétisation de la plateforme**, en posant les bases d'un système de transaction sécurisé et transparent.

8.6 Rétrospective du sixième sprint

Le sprint a été mené dans les délais, avec une livraison stable du module Stripe.

La collaboration entre les équipes frontend et backend a permis une intégration sans friction, notamment pour la gestion du watermark et des statuts de paiement.

Défis rencontrés :

- Gestion des redirections Stripe sans webhook, en s'appuyant sur *success_url*.
- Synchronisation entre les statuts de contenu, de paiement et le token JWT.
- Automatisation du téléchargement post-paiement.

Améliorations apportées :

- Optimisation du déclenchement automatique du téléchargement.
- Interfaces plus réactives et claires pour la phase de paiement.
- Meilleure traçabilité des transactions côté base de données.

Les difficultés ont été surmontées grâce à une approche collaborative, des tests rigoureux et un suivi précis du flux de paiement.

Conclusion

Ce sprint a renforcé l'aspect **professionnel et sécurisé** de Market Bridge.

L'intégration de Stripe Checkout, combinée à un filigrane actif avant paiement, garantit désormais une **transaction fiable**, tout en protégeant les contenus jusqu'à leur déblocage automatique.

La plateforme offre ainsi un **parcours complet et sécurisé**, bénéfique pour les marques comme pour les créateurs, consolidant la crédibilité et la valeur ajoutée de Market Bridge dans un contexte de collaboration numérique.

CHAPITRE 9 : ÉTUDE ET RÉALISATION DU SEPTIÈME SPRINT

Introduction

Ce septième sprint marque une étape importante dans l'évolution de Market Bridge, avec la mise en place d'un tableau de bord intelligent dédié au Super Administrateur.

Après avoir sécurisé les paiements et protégé les contenus au sprint précédent, l'objectif ici est de **centraliser les données clés** de la plateforme pour en faciliter le suivi et la prise de décision.

Le module développé permet une **visualisation dynamique des performances** (KPI) via des graphiques interactifs et l'**exportation de rapports statistiques** en PDF, avec un accès sécurisé réservé au rôle SUPERADMIN.

Les principales fonctionnalités intégrées :

- Suivi en temps réel des statistiques : créateurs, marques, packs, formations, feedbacks.
- Visualisation graphique des données (histogrammes, camemberts).
- Export automatique des rapports au format PDF.
- Accès restreint via authentification JWT.

Ce sprint transforme Market Bridge en un outil de pilotage analytique, donnant au Super Administrateur une **vision claire et synthétique de l'activité globale** de la plateforme.

9.1 Sprint Backlog

Le travail de ce sprint s'est concentré sur l'intégration d'un **module de statistiques visuelles** au tableau de bord administratif.

Ce dernier permet de suivre l'activité des utilisateurs, d'analyser la répartition par genre ou par pack, et de générer des rapports PDF à des fins de reporting.

Tableau 31: Backlog du Septième Sprint

ID	User Story	Priorité
US7.1	En tant que Super Admin, je veux consulter les statistiques globales (marques, créateurs...).	Haute

US7.2	En tant que Super Admin, je veux visualiser ces données sous forme de graphiques dynamiques.	Haute
US7.3	En tant que Super Admin, je veux pouvoir exporter les statistiques en PDF.	Haute
US7.4	En tant que système, je veux actualiser automatiquement les graphiques depuis la base.	Moyenne
US7.5	En tant que système, je veux sécuriser l'accès au module statistique (rôle SUPERADMIN).	Haute

Le sprint a nécessité une **connexion fluide entre Angular (Chart.js, jsPDF), Spring Boot et la base MySQL**, avec des données mises à jour en temps réel.

9.2 Spécification fonctionnelle du septième sprint

Ce module analytique donne au Super Administrateur un **tableau de bord interactif** pour surveiller les performances de Market Bridge.

Fonctionnalités développées :

- Affichage des indicateurs clés : nombre de marques, créateurs, services, formations.
- Graphiques interactifs pour visualiser les répartitions (par genre, pack...).
- Exportation automatique de rapports PDF incluant logo et signature.
- Authentification sécurisée par JWT, accès réservé au rôle SUPERADMIN.
- Synchronisation automatique avec la base MySQL pour des données à jour.

9.2.1 Diagramme de cas d'utilisation du septième sprint

Le diagramme ci-dessous montre les interactions entre le Super Administrateur et le système.

Il illustre comment l'administrateur s'authentifie pour accéder au module analytique, consulte les statistiques, puis génère un rapport PDF.

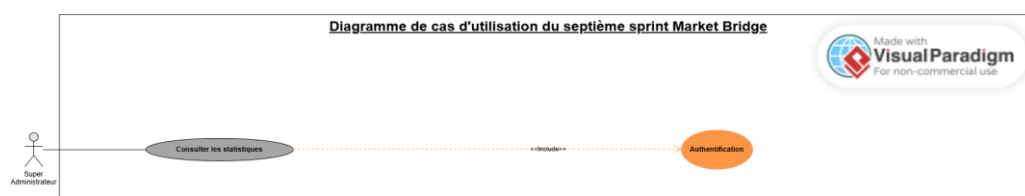


Figure 89: Diagramme de cas d'utilisation du septième sprint (Market Bridge)

9.2.2 Description textuelle des cas d'utilisation

9.2.2.1 Cas d'utilisation : Consulter les statistiques globales

Tableau 32: Description textuelle des cas d'utilisation - Consulter les statistiques globales

Élément	Description
Acteurs	Super Administrateur, Système Market Bridge
Pré-condition	Le Super Administrateur est connecté à la plateforme via authentification JWT (rôle SUPERADMIN).
Post-condition	Les statistiques globales (nombre de marques, créateurs, packs, formations) sont affichées sous forme de graphiques interactifs.
Scénario nominal	1. Le Super Administrateur accède à son tableau de bord analytique. 2. Le système interroge la base de données MySQL pour récupérer les indicateurs clés. 3. Les données sont traitées et renvoyées vers le frontend Angular. 4. Les graphiques sont générés dynamiquement (barres, secteurs, histogrammes). 5. Le Super Administrateur visualise les statistiques en temps réel sur son interface.
Scénario d'exception	Si la base de données est indisponible, le système affiche : “Erreur de chargement des statistiques veuillez réessayer plus tard.”

9.2.2.2 Cas d'utilisation : Visualiser la répartition par pack ou par genre

Tableau 33: Description textuelle des cas d'utilisation - Visualiser la répartition par pack ou par genre

Élément	Description
Acteurs	Super Administrateur, Système Market Bridge
Pré-condition	Le Super Administrateur est connecté à la plateforme et accède à la section “Statistiques détaillées”.
Post-condition	Les répartitions par pack, genre ou pays sont affichées sous forme de diagrammes circulaires et d'histogrammes.

Scénario nominal	1. Le Super Administrateur sélectionne un type de statistique (ex. : “Répartition des créateurs par pack”). 2. Le système interroge la base MySQL et calcule la répartition correspondante. 3. Les données sont transmises au composant Chart.js. 4. Le graphique s’affiche dynamiquement avec légende et couleurs distinctes. 5. Le Super Administrateur peut basculer entre différents types de graphiques (barres, camembert, lignes).
Scénario d’exception	Si aucune donnée n’est disponible, le message suivant s’affiche : “Aucune donnée à afficher pour cette catégorie.”

9.2.2.3 Cas d’utilisation : Exporter un rapport PDF

Tableau 34: Description textuelle des cas d’utilisation - Exporter un rapport PDF

Élément	Description
Acteurs	Super Administrateur, Système Market Bridge
Pré-condition	Les statistiques et graphiques sont déjà affichés sur le tableau de bord.
Post-condition	Un rapport PDF contenant les graphiques, le logo Market Bridge et la signature est généré et téléchargé automatiquement.
Scénario nominal	1. Le Super Administrateur clique sur Exporter en PDF. 2. Le système compile les données visibles à l’écran (graphiques, chiffres, titres). 3. jsPDF génère un rapport PDF incluant le logo Market Bridge, les graphiques et la date d’export. 4. Le fichier PDF est automatiquement téléchargé sur le poste de l’utilisateur. 5. Un message de confirmation s’affiche : “Rapport exporté avec succès.”
Scénario d’exception	Si la génération PDF échoue, le système affiche : “Erreur lors de la création du rapport, veuillez réessayer.”

9.3 Réalisation du sprint

Après validation du backlog, l'équipe a entamé la mise en œuvre des fonctionnalités prévues pour ce sprint.

L'objectif : offrir au Super Administrateur un **tableau de bord complet, interactif et exportable**, permettant le **suivi en temps réel** des indicateurs de la plateforme.

Les principales actions menées :

- Affichage des **KPI globaux** : marques, créateurs, packs, formations...
- Visualisation graphique des données (répartition par genre, pack, statut).
- **Export automatique en PDF**, intégrant les graphiques et une **signature électronique**.
- Sécurisation de l'accès via **authentification JWT** (rôle SUPERADMIN).

Technologies utilisées :

- **Frontend** : Angular 15 (Chart.js, jsPDF, Canvas pour la signature)
- **Backend** : Spring Boot 3 (API /stats et /export)
- **Base de données** : MySQL (agrégation des données)
- **Sécurité** : JWT – rôle SUPERADMIN

9.3.1 Interfaces réalisées

9.3.1.1 Tableau de bord analytique

Cette interface regroupe les indicateurs principaux :

total des utilisateurs, créateurs (pros/formés), marques, services et formations.

Les données sont affichées sous forme de **cartes dynamiques** et de **graphiques interactifs** (camemberts, histogrammes) permettant une vue d'ensemble rapide et claire.

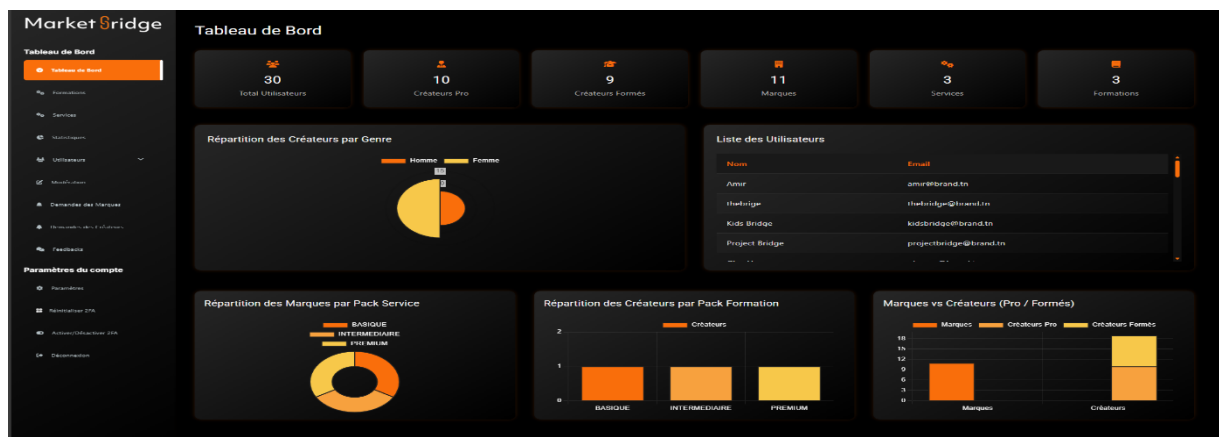


Figure 90: Interface Tableau de bord analytique - Super Administrateur

9.3.1.2 Statistiques détaillées et export PDF

Le Super Admin accède ici à une vue détaillée des répartitions (par pack, genre, statut), avec un bouton “**Exporter PDF**” générant un rapport complet, intégrant :
logo, graphiques et date d’export.



Figure 91: Interface des statistiques et export PDF

9.3.1.3 Signature numérique intégrée

Avant le téléchargement du rapport, une fenêtre modale s’ouvre pour **signer électroniquement** le document.

La signature est automatiquement ajoutée au PDF, garantissant la **validation officielle** de l’export.

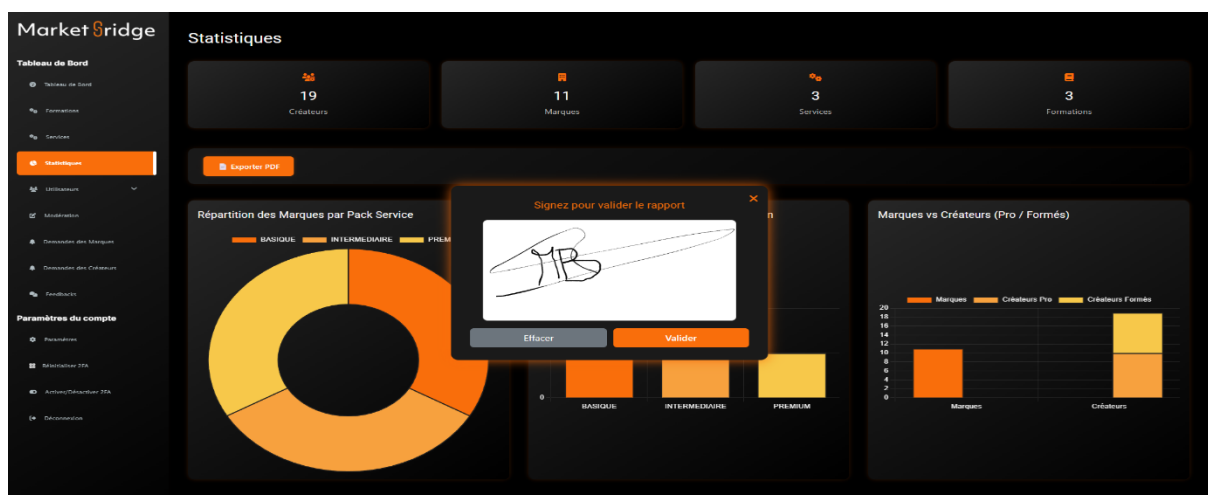


Figure 92: Signature du rapport avant export

9.3.1.4 Rapport PDF généré

Le rapport PDF contient :

- Le logo Market Bridge
- La date/heure de génération
- Un récapitulatif des KPI
- Les graphiques principaux
- La signature du Super Admin
- Une mention automatique : “Rapport généré automatiquement par MarketBridge”

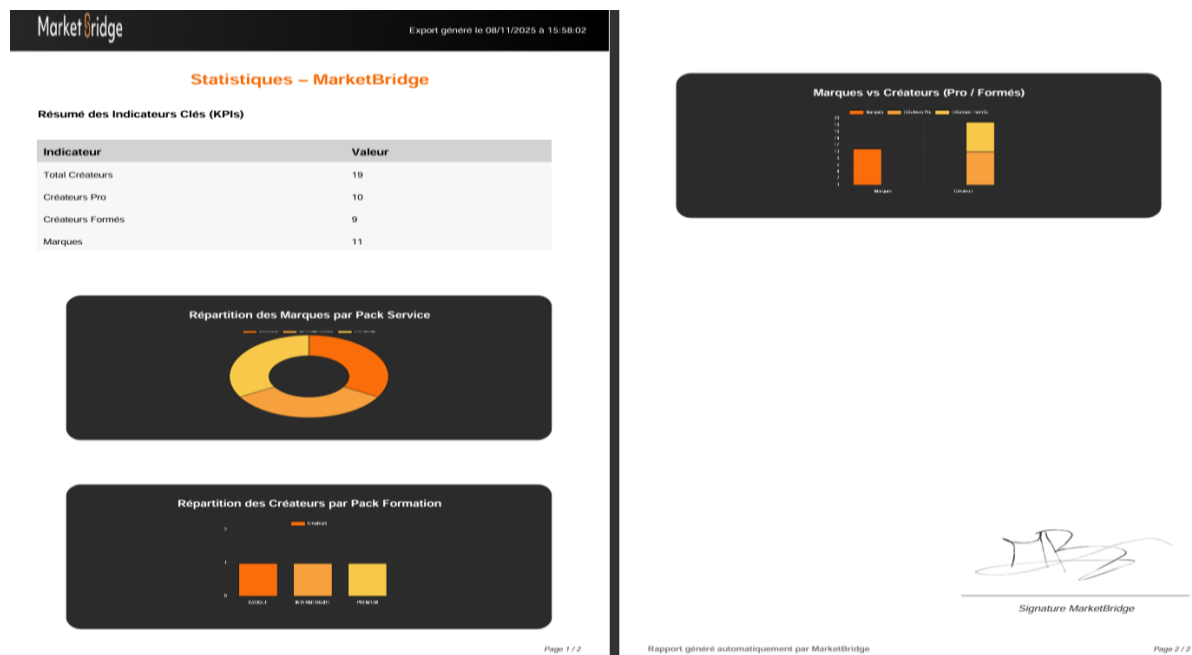


Figure 93: Rapport PDF final généré par la plateforme

9.4 Revue du septième sprint

Une réunion de revue a été organisée en présence du Scrum Master, du Product Owner et de l'équipe pour valider les résultats.

Tous les livrables ont été testés et validés avec succès, tant sur le plan fonctionnel que visuel.

Principaux résultats :

- Centralisation des indicateurs clés (KPI)
- Visualisation dynamique via **Chart.js**
- Répartition par pack, genre, statut utilisateur
- **Export PDF automatique** avec logo et signature numérique
- Cohérence graphique avec la charte du projet (noir/orange)

- Expérience fluide et intuitive pour le Super Admin

Le **Product Owner** a salué la **clarté des statistiques**, la qualité visuelle de l'interface et la robustesse de l'export PDF, soulignant la **valeur ajoutée professionnelle** de ce module.

9.5 Rétrospective du septième sprint

Le sprint s'est déroulé comme prévu et a abouti à la livraison complète et stable du module de statistiques et du tableau de bord analytique.

La collaboration entre les équipes frontend (Angular) et backend (Spring Boot) a été efficace, assurant une parfaite synchronisation entre la base MySQL et les visualisations interactives via Chart.js.

Défis rencontrés et solutions apportées :

- **Agrégation des données** issues des tables utilisateurs, packs, formations et feedbacks.
- **Intégration dynamique des graphiques** pour représenter les répartitions (genre, statut, pack).
- **Génération automatique d'un PDF** complet avec KPI, graphiques et logo.
- **Ajout d'une signature électronique** pour authentifier les rapports.
- **Respect de la charte visuelle** (noir/orange) avec un design responsive.

Ces défis ont été relevés grâce à :

- Des tests manuels réguliers (export PDF, lisibilité des graphiques),
- Des ajustements visuels progressifs pour une meilleure clarté,
- Une communication fluide entre développeurs frontend et backend pour l'harmonisation des données JSON.

Conclusion

Ce sprint a permis de doter le Super Administrateur d'un outil puissant pour le **suivi en temps réel** de l'activité sur la plateforme.

Il peut désormais :

- Consulter les **KPI essentiels** en un coup d'œil,
- Visualiser les **données sous forme graphique** claire et interactive,
- **Exporter un rapport PDF professionnel**, signé et horodaté.

Le rendu final allie **simplicité, fiabilité et professionnalisme**, renforçant la crédibilité de Market Bridge comme solution analytique complète.

CHAPITRE 10 : PHASE FINALE ET INDUSTRIALISATION (SPRINT DEVOPS)

Introduction

Ce dernier sprint marque la phase d'industrialisation du projet **Market Bridge**, avec pour objectif de garantir une mise en production fiable, stable et totalement automatisée.

Il s'inscrit dans une démarche **DevOps** moderne, basée sur la conteneurisation avec **Docker** et l'automatisation du cycle de vie applicatif via un pipeline **CI/CD** orchestré par **Jenkins**.

Les axes traités dans ce chapitre sont les suivants :

- L'automatisation du cycle de développement via l'intégration et le déploiement continus,
- Le choix stratégique de la conteneurisation face à la virtualisation,
- La mise en place d'une chaîne DevOps complète (Jenkins, SonarQube, Nexus, Prometheus, Grafana),
- Et le système de supervision pour le suivi en temps réel de la plateforme.

L'objectif global est d'obtenir un système robuste, sécurisé et capable de tester, publier et déployer automatiquement chaque nouvelle version sans intervention manuelle.

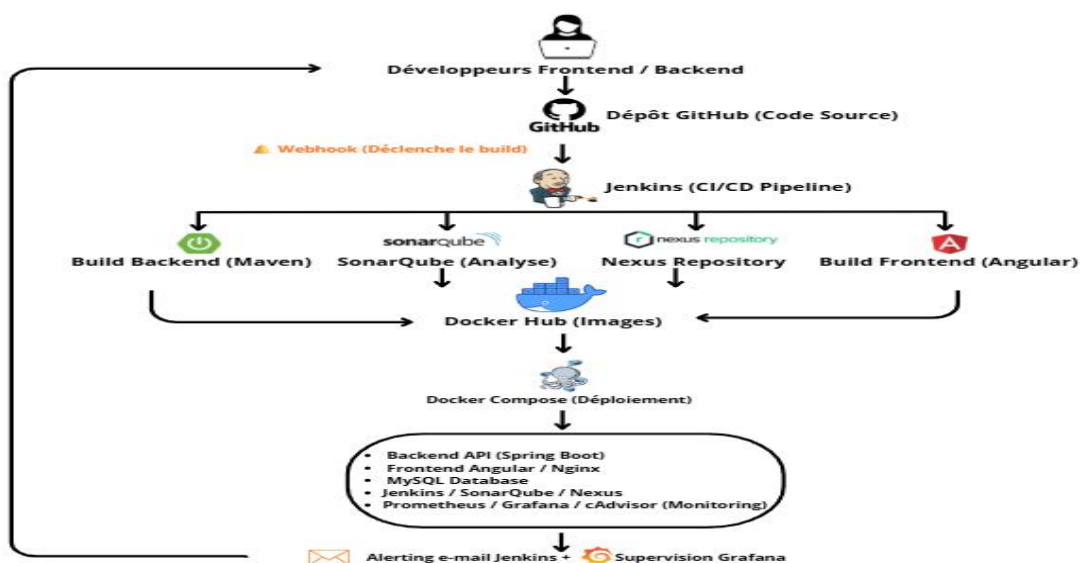


Figure 94: Architecture DevOps du projet Market Bridge

10 Méthodologie DevOps appliquée à Market Bridge

10.1.1 Intégration, livraison et déploiement continu

La méthode CI/CD (Continuous Integration / Continuous Delivery / Continuous Deployment) vise à automatiser l'ensemble des étapes, du commit jusqu'à la mise en production.

Dans le projet Market Bridge, cette automatisation est déclenchée dès chaque push sur GitHub.

Un webhook configuré sur les dépôts frontend et backend lance automatiquement les pipelines Jenkins, sans aucune intervention manuelle.

Ainsi, chaque modification du code source entraîne immédiatement un nouveau cycle de build, de tests, d'analyse de la qualité et de déploiement, garantissant une chaîne DevOps entièrement automatisée.

Dans Market Bridge, trois piliers ont été appliqués:

Intégration continue (CI) : chaque commit déclenche la compilation, l'exécution des tests et une analyse de la qualité du code ;

Livraison continue (CD) : les artefacts validés sont automatiquement publiés dans le dépôt Nexus;

- ❖ **Déploiement continu (CD)** : les images Docker sont déployées dans un environnement isolé via Docker Compose.

Cette chaîne permet de livrer des versions stables rapidement, de manière fiable et sans interruption.

10.1.2 Intégration continue (CI)

L'intégration continue repose sur **Jenkins**, connecté à **GitHub** et **SonarQube**. À chaque commit :

- Le code est récupéré,
- Compilé avec Maven (backend) et Angular CLI (frontend),
- Analysé par SonarQube (qualité, sécurité, duplications),
- Puis l'artefact *.jar* est publié dans **Nexus**.

Ce processus automatisé garantit la stabilité du code et la qualité logicielle avant chaque mise en production.

10.1.3 Livraison et déploiement continu (CD)

Le pipeline Jenkins est structuré en deux **jobs** :

- ❖ **Backend** : création de l'image Docker *wiembenali/marketbridge-backend* avec l'API Spring Boot,
- ❖ **Frontend** : build Angular, création de l'image *wiembenali/marketbridge-frontend* (Nginx) et déploiement automatique.

Les images sont ensuite poussées vers **Docker Hub**, puis déployées via **Docker Compose** dans un réseau isolé *marketbridge_net*.

Chaque exécution produit un rapport automatisé, et envoie une notification par mail en cas de succès ou d'erreur.

10.1.4 DevOps

Le DevOps désigne la synergie entre développement et exploitation, avec pour but d'automatiser, fiabiliser et accélérer le cycle de vie logiciel.

Dans le projet *Market Bridge*, cette démarche se traduit par :

- L'automatisation du build, des tests, de l'analyse de code et du déploiement ;
- Une collaboration fluide entre les pôles frontend, backend et infrastructure ;
- Un contrôle qualité continu via **SonarQube** et **Nexus** ;
- Un suivi en temps réel des performances grâce à **Prometheus** et **Grafana** ;
- Et un système **d'alerting par e-mail déclenché par Jenkins**, informant automatiquement l'équipe en cas d'échec d'un pipeline CI/CD.

Chaque pipeline Jenkins envoie ainsi une notification de succès ou d'échec par mail.

Ce mécanisme garantit une surveillance active sans avoir à consulter manuellement Jenkins ou les dashboards.

Ce dispositif global renforce la productivité, fiabilise les mises en production et garantit la traçabilité des versions livrées.

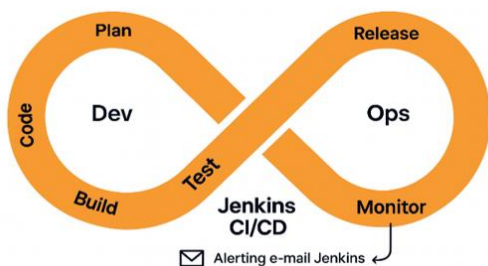


Figure 95: Chaîne DevOps du projet Market Bridge – Jenkins CI/CD et alertes e-mail

10.1.5 Virtualisation vs Conteneurisation

Virtualisation et conteneurisation sont deux méthodes d’isolation pour exécuter des applications dans des environnements indépendants.

- **La virtualisation** repose sur un hyperviseur (ex. VMware) créant plusieurs machines virtuelles, chacune avec son propre système d’exploitation. Cette approche est stable mais lourde et lente à démarrer.
- **La conteneurisation**, utilisée dans Market Bridge, s’appuie sur **Docker** pour isoler les processus tout en partageant le noyau du système hôte. Elle est plus légère, rapide et portable, ce qui en fait une solution idéale pour un environnement DevOps.

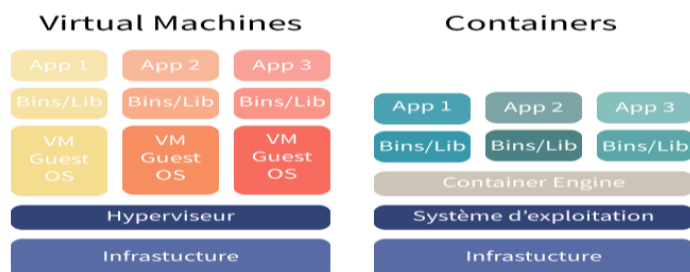


Figure 96: Différence entre machine virtuelle et conteneur Docker

10.1.6 La Virtualisation

La virtualisation permet d’exécuter plusieurs OS sur une seule machine physique via un hyperviseur.

Chaque machine virtuelle embarque un OS complet, ce qui la rend robuste mais gourmande en ressources.

Cette approche n’a pas été retenue dans Market Bridge, car elle manquait de flexibilité et de rapidité pour un projet en CI/CD.

L’équipe a opté pour **Docker**, plus adapté à un déploiement agile et portable.

10.1.7 La Conteneurisation

La conteneurisation est au cœur de l'architecture DevOps de Market Bridge.

Chaque conteneur embarque le code, les dépendances et sa configuration via un **Dockerfile**.

Les services exécutés via **Docker Compose** incluent :

- Le backend (**Spring Boot**),
- Le frontend (**Angular/Nginx**),
- La base de données **MySQL**,
- Les outils DevOps : **Jenkins**, **SonarQube**, **Nexus**,
- Ainsi que les outils de supervision : **Prometheus** et **Grafana**.

Tous les conteneurs fonctionnent dans un réseau sécurisé nommé *marketbridge_net*, assurant l'isolation, la reproductibilité et la montée en charge des déploiements.

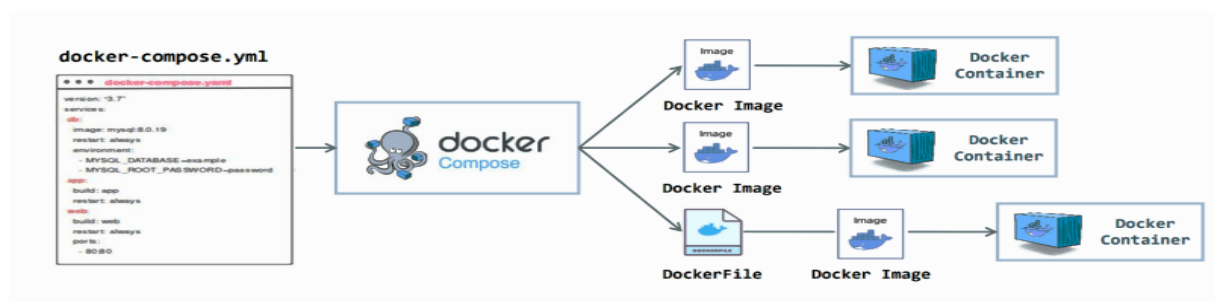


Figure 97: Architecture Docker Compose du projet Market Bridge

```
PS C:\Users\SAT\Desktop\Bridge Market> docker compose up -d
[+] Running 11/11
 ✓ Container cadvisor Running
 ✓ Container marketbridge_db Healthy
 ✓ Container sonarqube Running
 ✓ Container nexus Running
 ✓ Container prometheus Running
 ✓ Container jenkins Running
 ✓ Container flask-ai Running
 ✓ Container grafana Running
 ✓ Container chatbot-ai Running
 ✓ Container marketbridge_backend Running
 ✓ Container marketbridge_frontend Running
```

Figure 98: Exécution des conteneurs Docker du projet Market Bridge

10.1.8 Architecture et fonctionnement du pipeline CI/CD

Le pipeline CI/CD de *Market Bridge* s'appuie sur une chaîne d'outils open source interconnectés, chacun jouant un rôle clé dans l'automatisation du cycle de vie applicatif :

- **Jenkins** : pilote le build, les tests et le déploiement ;

- **SonarQube** : évalue la qualité du code et détecte les failles ;
- **Nexus** : stocke les artefacts Maven et assure la gestion des versions ;
- **Docker Hub** : héberge les images Docker du frontend et du backend ;
- **Prometheus** : collecte les métriques systèmes et applicatives ;
- **Grafana** : visualise les performances et génère des alertes.

Tous ces outils sont déployés dans un environnement Docker isolé (*marketbridge_net*), garantissant une interopérabilité fluide et une automatisation continue.

L’alerting repose sur un système de **notifications par e-mail déclenchées par Jenkins**, permettant au responsable DevOps d’être immédiatement informé en cas d’échec ou de surcharge.

10.1 Introduction

Ce dernier sprint clôt le projet par sa phase d’industrialisation. L’objectif est de garantir la stabilité, la sécurité et l’automatisation complète de la plateforme dans un cadre professionnel.

Reposant sur Docker et Jenkins CI/CD, cette démarche DevOps assure une livraison rapide et fiable sans intervention manuelle.

Ce chapitre aborde la méthode CI/CD appliquée dans le projet Market Bridge, la différence entre la virtualisation et la conteneurisation, l’ensemble des outils intégrés dans la chaîne DevOps tels que Jenkins, SonarQube, Nexus, Prometheus et Grafana, ainsi que le système de notifications automatisées mis en place via Jenkins.

10.2 Les outils DevOps du pipeline Market Bridge

La chaîne DevOps mise en place couvre l’ensemble des étapes : développement, intégration, déploiement, supervision.

Chaque service est déployé dans un réseau Docker isolé (*marketbridge_net*), assurant un environnement stable et reproductible.

10.2.1 GitHub

GitHub centralise la gestion du code source du projet.

GitHub centralise la gestion du code source du projet Market Bridge.

À chaque **push sur la branche principale**, un webhook déclenche **automatiquement**

l'exécution des pipelines Jenkins.

Cette intégration garantit que **chaque modification du code est immédiatement validée, testée et industrialisée**, assurant une intégration continue fiable et traçable.

Grâce à **Ngrok**, le webhook fonctionne en local comme en cloud, assurant une intégration continue 100 % automatisée et fiable.

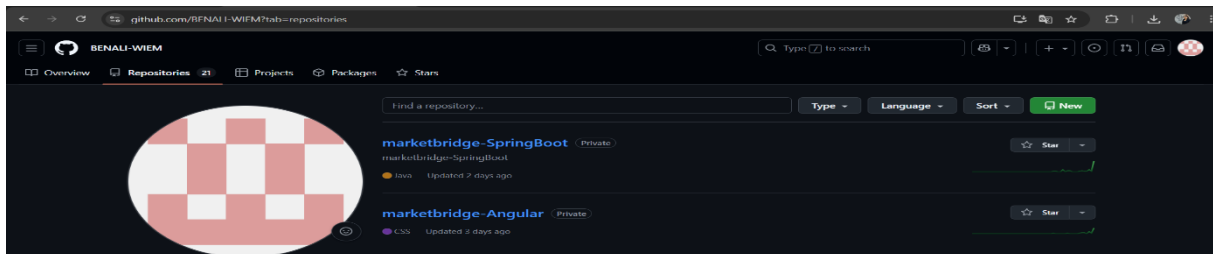


Figure 99: Vue d'ensemble des dépôts GitHub Market Bridge (Frontend et Backend)

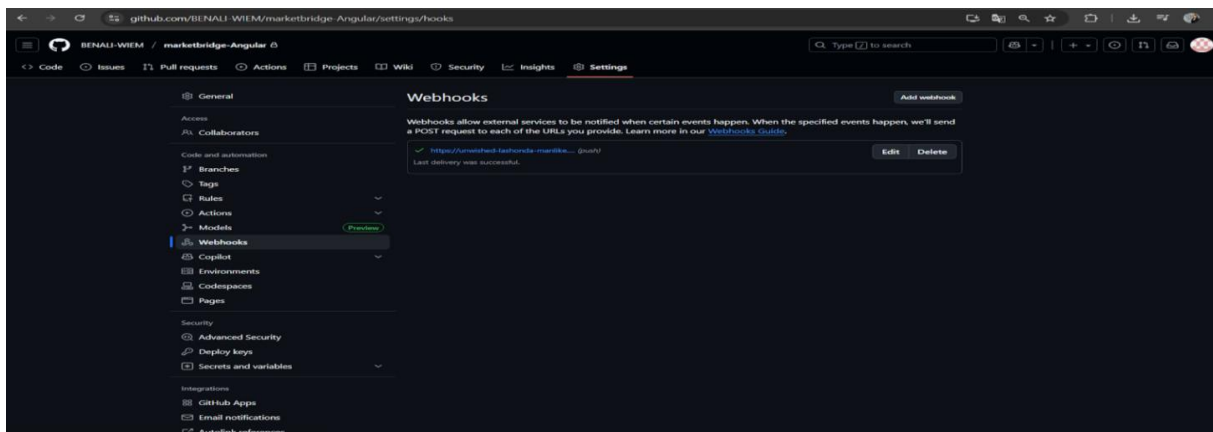


Figure 100: Dépôt GitHub Market Bridge Frontend synchronisé avec Jenkins

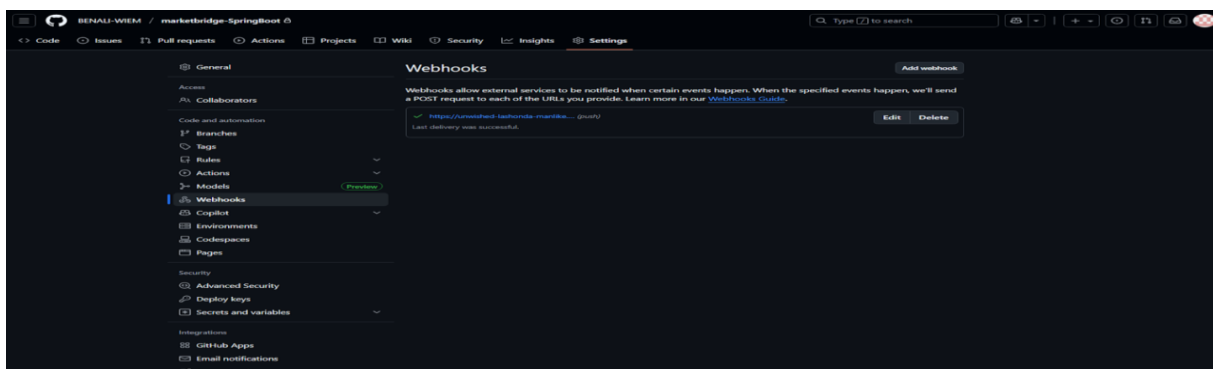


Figure 101: Dépôt GitHub Market Bridge Backend synchronisé avec Jenkins

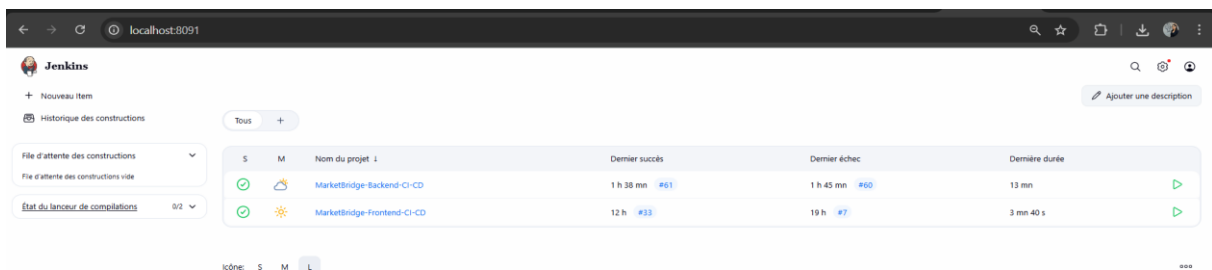
10.2.2 Jenkins

Pour le projet Market Bridge, deux pipelines Jenkins distincts ont été mis en place afin de séparer les workflows backend et frontend :

- ❖ **MarketBridge-Backend-CI-CD** : compile le backend avec Maven, exécute les tests unitaires, analyse le code avec SonarQube, publie les artefacts sur Nexus, et génère l'image Docker.
- ❖ **MarketBridge-Frontend-CI-CD** : construit le projet Angular, crée l'image Nginx, puis la pousse automatiquement sur Docker Hub.

Cette séparation améliore la résilience du système : un échec du pipeline frontend n'affecte pas le backend, et inversement. Cela garantit une meilleure disponibilité et une gestion plus souple des erreurs.

Chaque pipeline est déclenché automatiquement grâce aux webhooks configurés sur GitHub, assurant une intégration continue à chaque nouveau commit.



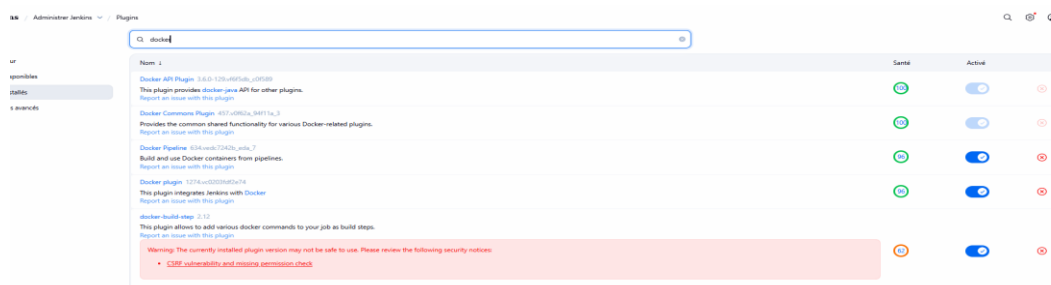
S	M	Nom du projet	Dernier succès	Dernier échec	Dernière durée
✓	☁	MarketBridge-Backend-CI-CD	1 h 38 mn #61	1 h 45 mn #60	13 mn
✓	☀	MarketBridge-Frontend-CI-CD	12 h #33	19 h #7	3 mn 40 s

Figure 102: Tableau Jenkins des jobs CI/CD

❖ Plugins installés

Pour prendre en charge les différents outils, Jenkins utilise les plugins suivants :

- Docker Pipeline, Docker API & Commons, Docker Compose Build Step
- NodeJS Plugin (pour Angular)
- SonarQube Scanner Plugin



Nom	Santé	Activé
Docker API Plugin 1.43.0-120a0f9d4c0000	✓	✓
Docker Commons Plugin 0.17.0-0f9d4c0000	✓	✓
Docker Pipeline 0.14.0-0f9d4c0000	✓	✓
Docker plugin 1.0.14-0f9d4c0000	✓	✓

Figure 103: Plugins installés dans Jenkins

❖ Connexion à Docker Hub

L'authentification avec Docker Hub est gérée via des identifiants sécurisés dans Jenkins (Manage Jenkins → Credentials) :

- **ID** : dockerhub-id
- **Utilisateur** : wiembenali
- **Mot de passe** : chiffré

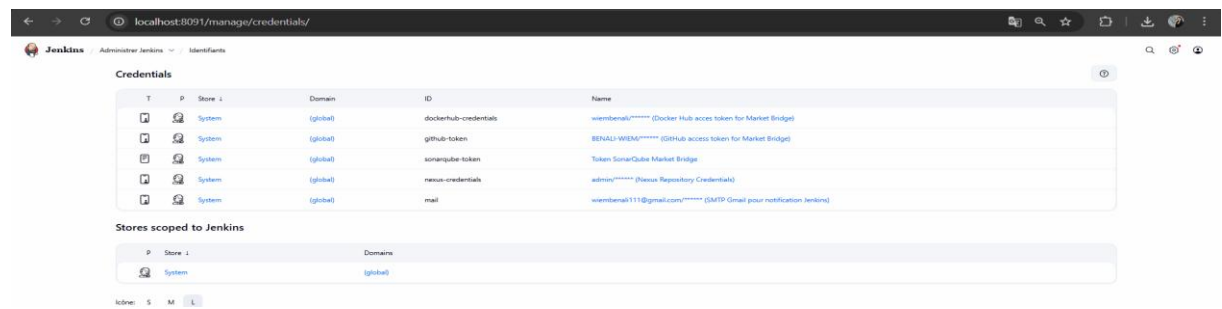


Figure 104: Configuration des identifiants Docker Hub

10.3 Intégration et déploiement continu du backend

10.3.1 Pipelines CI & CD

Le backend est construit et déployé à travers deux scripts :

- **Pipeline CI** : compile, teste et analyse le code (SonarQube)

```

1~ pipeline {
2~   agent any
3~
4~   environment {
5~     SONARQUBE = "SonarQubeLocal"
6~     GITHUB_CRED = "github-token"
7~     DOCKER_CRED = "dockerhub-credentials"
8~     NEXUS_CRED = "nexus-credentials"
9~     DOCKER_REPO = "wiembenali/marketbridge-backend"
10~    NEXUS_RELEASE_URL = "http://nexus:8081/repository/marketbridge-releases/"
11~    NEXUS_SNAPSHOT_URL = "http://nexus:8081/repository/marketbridge-snapshots/"
12~  }
13~
14~  options {
15~    disableConcurrentBuilds()
16~    timestamps()
17~  }
18~
19~  stages {
20~    stage("Checkout depuis Github") {
21~      steps {
22~        git branch: 'main',
23~          credentialsId: "${GITHUB_CRED}",
24~          url: 'https://github.com/BENALI-WIEM/marketbridge-SpringBoot.git'
25~      }
26~    }
27~
28~    stage("Détermination de la version Release") {
29~      steps {
30~        script {
31~          def raw = sh(returnStdout: true, script: """
32~            mvn -q -Dexec.executable="echo" -Dexec.args='${version}' \\
33~            --non-recursive org.codehaus.mojo:exec-maven-plugin:3.1.0:exec
34~          """).trim()
35~          def base = raw.replace("${SNAPSHOT}", "")
36~          env.RELEASE_VERSION = "${base}-${env.BUILD_NUMBER}"
37~          sh "mvn -B versions:set -DnewVersion=${env.RELEASE_VERSION} -DgenerateBackup"
38~        }
39~      }
40~    }
41~
42~    stage("Build Maven") {
43~      steps {
44~        sh 'mvn clean package -DskipTests'
45~      }
46~    }
47~  }
48~
49~  stage("Tests unitaires (JUnit)") {
50~    steps {
51~      sh 'mvn test'
52~    }
53~    post {
54~      always {
55~        junit 'target/surefire-reports/*.xml'
56~      }
57~    }
58~  }
59~
60~  stage("Analyse SonarQube") {
61~    steps {
62~      withSonarQubeEnv("${SONARQUBE}") {
63~        sh """
64~          mvn verify sonar:sonar \
65~            -Dsonar.projectKey=MarketBridge \
66~            -Dsonar.projectName=MarketBridge Backend \
67~            -Dsonar.host.url=http://sonarqube:9000 \
68~            -Dsonar.login=sonar_70813c36e91c62ac2f82aa875c24c394389
69~        """
70~      }
71~    }
72~  }
73~
74~  stage("Déploiement vers Nexus") {
75~    steps {
76~      echo "Déploiement du JAR vers Nexus..."
77~      withCredentials([usernamePassword(credentialsId: "${NEXUS_CRED}", usernameVariable: 'NEXUS_USER', passwordVariable: 'NEXUS_PASS')]) {
78~        sh """
79~          mvn deploy -DskipTests \
80~            -Dnexus.url=http://nexus:8081 \
81~            -Dnexus.repository=marketbridge-releases \
82~            -Dnexus.groupId=com.marketbridge \
83~            -Dnexus.artifactId=marketbridge-backend \
84~            -Dnexus.version=${RELEASE_VERSION} \
85~            -Dnexus.username=${NEXUS_USER} \
86~            -Dnexus.password=${NEXUS_PASS}
87~        """
88~      }
89~    }
90~  }
91~}

```

Figure 105: Script du pipeline CI

- **Pipeline CD** : génère l'image Docker et la pousse vers Docker Hub

```

91 ~ stage('Build Docker image') {
92 ~     steps {
93 ~         sh """
94 ~             docker build -t ${DOCKER_REPO}:${RELEASE_VERSION} -t ${DOCKER_REPO}:latest .
95 ~         """
96 ~     }
97 ~ }
98 ~
99 ~ stage('Push Docker image sur Docker Hub') {
100 ~     steps {
101 ~         withCredentials([usernamePassword(credentialsId: "${DOCKER_CRED}", usernameVariable: "DOCKER_USER", passwordVariable: "DOCKER_PASS")]) {
102 ~             sh """
103 ~                 echo "${DOCKER_PASS}" | docker login -u "${DOCKER_USER}" --password-stdin
104 ~                 docker push ${DOCKER_REPO}:${RELEASE_VERSION}
105 ~                 docker push ${DOCKER_REPO}:latest
106 ~             """
107 ~         }
108 ~     }
109 ~ }
110 ~
111 ~ stage('Check Monitoring (Prometheus / Grafana)') {
112 ~     steps {
113 ~         script {
114 ~             echo "🔍 Vérification de la disponibilité de Prometheus et Grafana..."
115 ~
116 ~             def prometheusStatus = sh(
117 ~                 script: "curl -s -o /dev/null -w '%(http_code)' http://prometheus:9090/-/healthy",
118 ~                 returnStdout: true
119 ~             ).trim()
120 ~
121 ~             if (prometheusStatus != '200') {
122 ~                 error("🚨 Prometheus n'est pas disponible (HTTP ${prometheusStatus}) !")
123 ~             } else {
124 ~                 echo "✅ Prometheus est UP."
125 ~             }
126 ~
127 ~             def grafanaStatus = sh(
128 ~                 script: "curl -s -o /dev/null -w '%(http_code)' http://grafana:3000/api/health",
129 ~                 returnStdout: true
130 ~             ).trim()
131 ~
132 ~             if (grafanaStatus != '200') {
133 ~                 error("🚨 Grafana n'est pas accessible (HTTP ${grafanaStatus}) - non bloquant.")
134 ~             } else {
135 ~                 echo "✅ Grafana est UP."
136 ~             }
137 ~         }
138 ~     }
139 ~ }
140 ~
141 ~ stage('Publier les métriques Prometheus') {
142 ~     steps {
143 ~         script {
144 ~             echo "📊 Publication des métriques Prometheus..."
145 ~             sh "curl -X POST http://jenkins:8080/prometheus"
146 ~         }
147 ~     }
148 ~ }
149 ~ }

```

Figure 106: Script du pipeline CD

10.3.2 Analyse de code avec SonarQube

À chaque build, une analyse statique est lancée pour détecter :

- Bugs
- Vulnérabilités
- Duplications de code

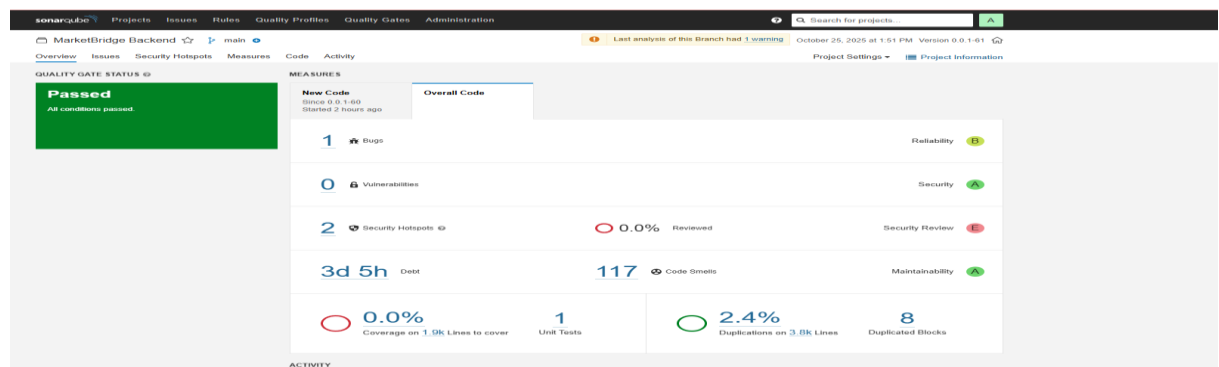


Figure 107: Tableau SonarQube du projet Market Bridge

10.3.3 Gestion des artefacts avec Nexus

Les fichiers .jar générés après chaque build sont stockés dans un dépôt Maven privé :

→ <http://localhost:8081/repository/marketbridge-releases/>

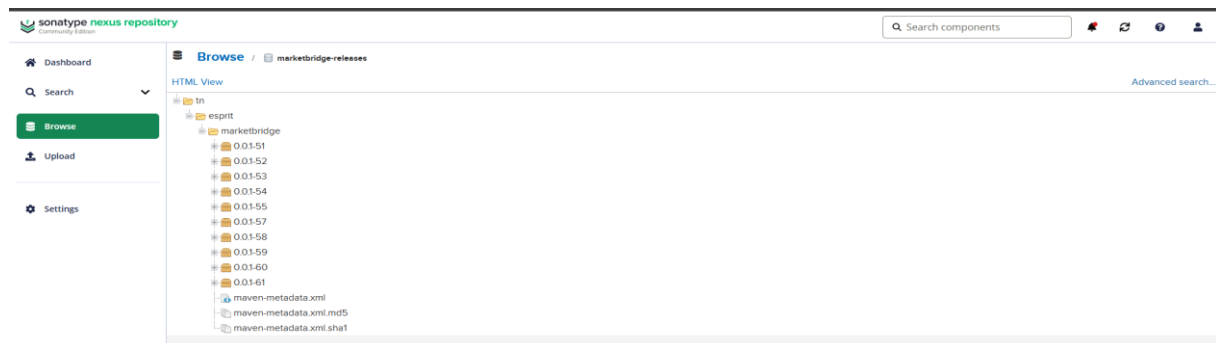


Figure 108: Dépôt Nexus Market Bridge

10.3.4 Conteneurisation et déploiement Docker

Tous les services de Market Bridge (backend, frontend, IA, Jenkins, SonarQube, Nexus, Prometheus, Grafana, MySQL...) sont conteneurisés avec Docker et orchestrés via Docker Compose.

Chaque composant fonctionne dans un conteneur isolé, ce qui garantit portabilité, indépendance et facilité de déploiement.

```
PS C:\Users\SAT> docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
bridgemarket-backend	latest	a31c7935e770	8 minutes ago	484MB
bridgemarket-frontend	latest	6e85feeba847	4 days ago	383MB
bridgemarket-chatbot-ai	latest	7213d467923a	4 days ago	1.44GB
bridgemarket-flask-ai	latest	38704a049a32	4 days ago	714MB
bridgemarket-jenkins	latest	af6d458350da	2 weeks ago	1.26GB
prom/prometheus	latest	f9b393a193e8	2 weeks ago	370MB
grafana/grafana	latest	bac4f177a0d5	2 weeks ago	730MB
sonatype/nexus3	latest	1fae659c16e2	4 weeks ago	684MB
mysql	8	69cf2b3f8428	6 weeks ago	785MB
sonarqube	lts-community	b2698b16c2b2	2 months ago	604MB
gcr.io/cadvisor/cadvisor	latest	c02cf39d3dba	20 months ago	80.8MB

```
PS C:\Users\SAT>
```

Figure 109: Liste des images Docker du projet

```
C:\Users\SAT\Desktop\Bridgemarket> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2530f9b30774	bridgemarket-jenkins	"/usr/bin/tini -- /u..."	16 hours ago	Up 23 seconds	0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp, 0.0.0.0:8091->8080/tcp	jenkins
519f2dab1e7f	gcr.io/cadvisor/cadvisor:latest	"/usr/bin/cadvisor -..."	36 hours ago	Up 23 seconds (health: starting)	0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp	cadvisor
12d0a1f35520	grafana/grafana	"/run.sh"	37 hours ago	Up 22 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp	grafana
f9f907020eab	prom/prometheus	"/bin/prometheus --c..."	37 hours ago	Up 23 seconds	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp	prometheus
b3659c46c2c8	bridgemarket-frontend	"/docker-entrypoint.s..."	37 hours ago	Up 11 seconds	0.0.0.0:4200->400/tcp, [::]:4200->400/tcp	marketbridge-frontend
2ab397b06970	bridgemarket-backend	"java -jar app.jar"	37 hours ago	Up 12 seconds (health: starting)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	marketbridge-backend
ed0090604082	mysql:8	"docker-entrypoint.s..."	37 hours ago	Up 23 seconds (healthy)	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp	marketbridge_db
15036ccf9465	sonatype/nexus3	"/opt/sonatype/nexus..."	37 hours ago	Up 23 seconds	0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp	nexus
4d5689c026e4	sonarqube/lts-community	"/opt/sonarqube/dech..."	37 hours ago	Up 23 seconds	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp	sonarqube

```
C:\Users\SAT\Desktop\Bridgemarket>
```

Figure 110: Liste des conteneurs Docker actifs

10.3.5 Supervision et monitoring

- **Prometheus** assure la collecte des métriques systèmes de chaque conteneur.

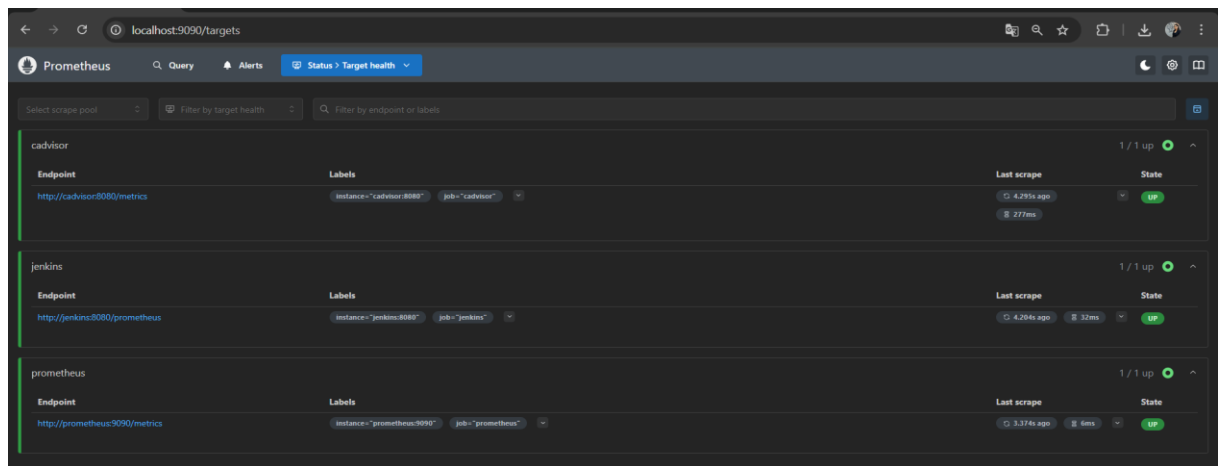


Figure 111: Interface Prometheus – état des services

- **Grafana** permet de suivre en temps réel les performances de la base de données, du pipeline CI/CD et des conteneurs Docker. Il aide à détecter rapidement les problèmes de charge ou les ralentissements.

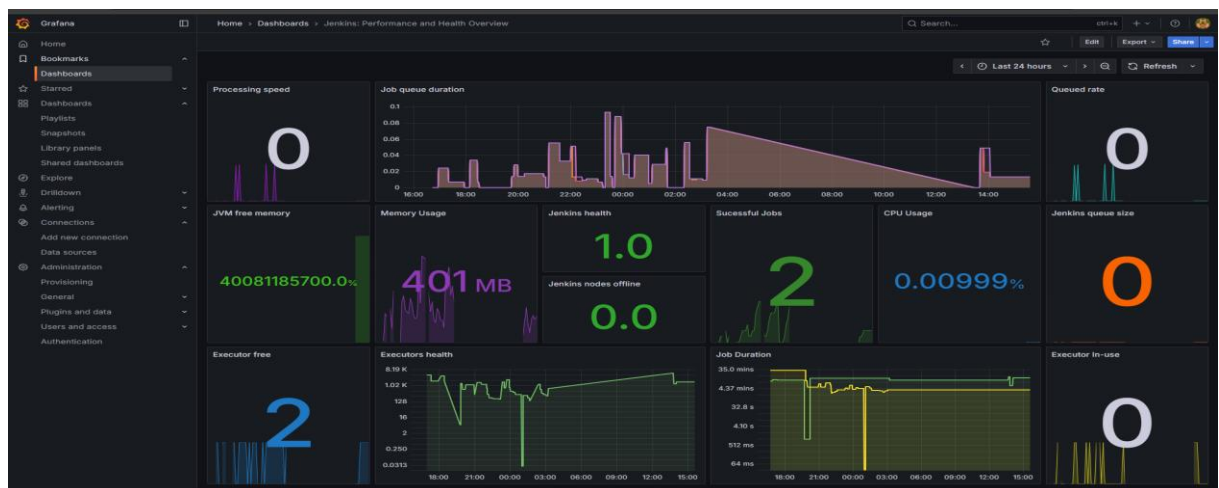


Figure 112: Dashboard Grafana Market Bridge

10.3.6 Alerting automatique (Backend)

À chaque exécution du pipeline backend, Jenkins envoie un mail de résumé :

☑ En cas de succès :

Le mail contient le statut du build, la version, la durée, un résumé des étapes (Git, Maven, Tests, SonarQube, Nexus, Docker, Monitoring), ainsi que des liens directs vers les outils utilisés.

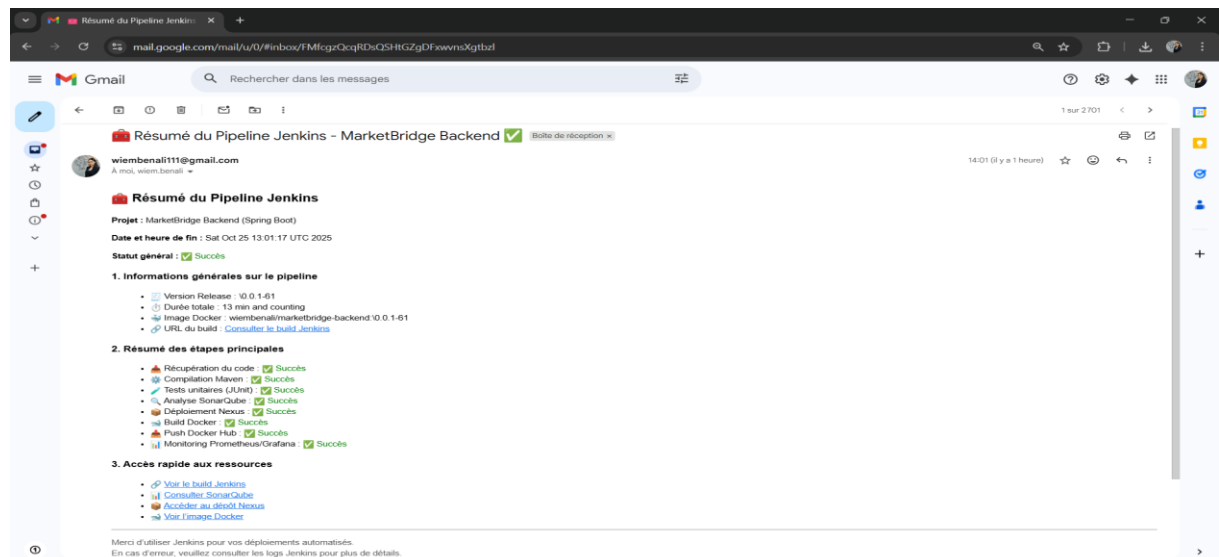


Figure 113: E-mail Jenkins Backend – Succès

☒ En cas d'échec :

Le message indique l'étape échouée, l'heure exacte du build, et fournit un lien vers les logs Jenkins pour analyse.

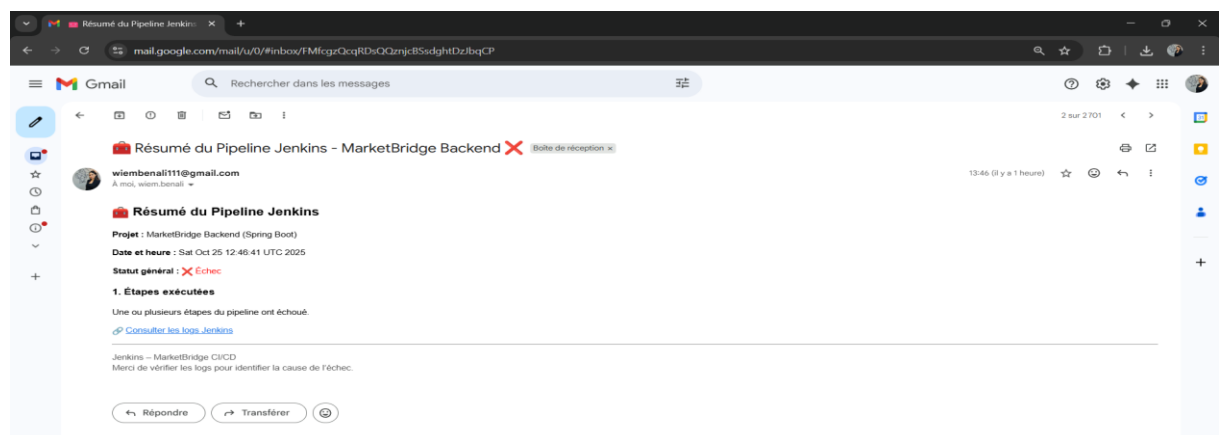


Figure 114: E-mail Jenkins – Échec

Ce système garantit une supervision réactive et une traçabilité complète de chaque exécution.

10.3.7 Résultat du pipeline Backend

Le pipeline **MarketBridge-Backend-CI-CD** s'exécute en moyenne en **13 minutes**, avec un **dernier build (#61)** marqué comme **réussi** ✓.

Toutes les étapes ont été validées sans erreur :

- Récupération du code (1s)

- Détermination de la version (55s)
- Build Maven (1 min 10 s)
- Tests unitaires JUnit (39 s)
- Analyse SonarQube (1 min 20 s)
- Déploiement Nexus (43 s)
- Construction de l'image Docker (2 min 26 s)
- Push vers Docker Hub (3 min 56 s)
- Monitoring Prometheus/Grafana (608 ms)
- Publication des métriques Prometheus (360 ms)
- Étapes post-déploiement (7 s)

L'analyse SonarQube a validé le **Quality Gate**, et l'image du backend a bien été publiée sur **Docker Hub**.

L'ensemble du processus est automatisé, traçable et conforme aux standards DevOps.

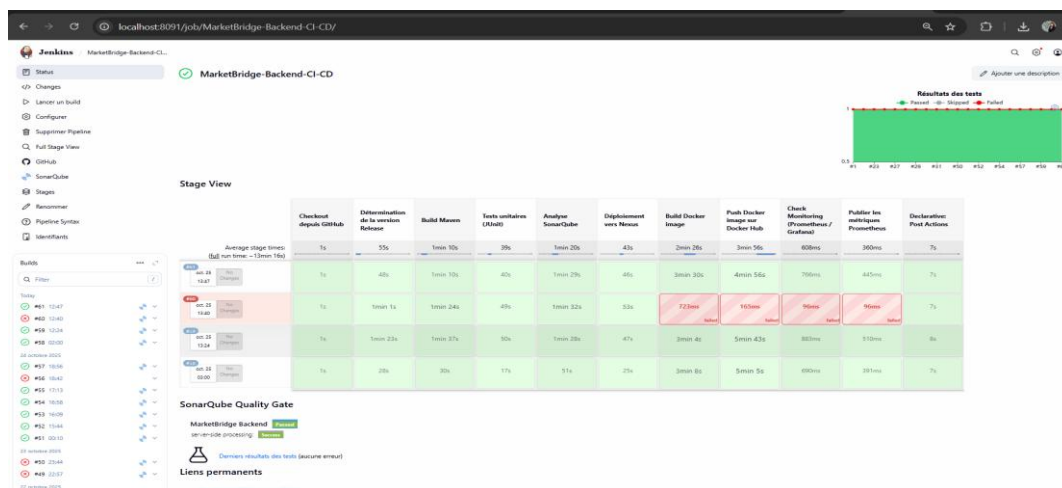


Figure 115: Durée des étapes du pipeline CI/CD – Backend

10.4 Intégration et déploiement du Frontend

Le pipeline **MarketBridge-Frontend-CI-CD** prend en charge l'automatisation complète du build Angular, jusqu'à la publication sur Docker Hub.

Chaque commit déclenche une mise à jour fluide de l'interface utilisateur via Jenkins.

10.4.1 Étapes principales

- Clonage depuis GitHub

```

1~ pipeline {
2~   agent any
3~
4~   environment {
5~     GITHUB_CRED = 'github-token'
6~     DOCKER_CRED = 'dockerhub-credentials'
7~     DOCKER_REPO = 'wiebbenali/marketbridge-frontend'
8~   }
9~
10~   stages {
11~     stage('Checkout depuis GitHub') {
12~       steps {
13~         echo '🔥 Clonage du dépôt MarketBridge Angular...'
14~         git branch: 'main',
15~             credentialsId: "${GITHUB_CRED}",
16~             url: 'https://github.com/BENALI-WIEM/marketbridge-Angular.git'
17~       }
18~     }
19~   }
20~ }

```

Figure 116: Clonage depuis GitHub

➤ Build Angular

```

19~
20~   stage('Build du projet Angular') {
21~     steps {
22~       echo '🔥 Construction du build de production Angular...'
23~       sh '''
24~         npm install --legacy-peer-deps
25~         npm run build -- --configuration production
26~       '''
27~     }
28~   }
29~ }

```

Figure 117: Build Angular

➤ Création de l'image Docker

```

30~
31~   stage('Build Docker image') {
32~     steps {
33~       echo '🔥 Construction de l'image Docker frontend...'
34~       sh '''
35~         docker build -t ${DOCKER_REPO}:latest .
36~       '''
37~     }
38~   }
39~ }

```

Figure 118: Création de l'image Docker

➤ Publication sur Docker Hub

```

stage('Push Docker image to Docker Hub') {
  steps {
    echo '🔥 Envoi de l'image Docker sur Docker Hub...'
    withCredentials([usernamePassword(credentialsId: "${DOCKER_CRED}", usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
      sh '''
        echo "${DOCKER_PASS}" | docker login -u "${DOCKER_USER}" --password-stdin
        docker push ${DOCKER_REPO}:latest
      '''
    }
  }
}

```

Figure 119: Publication sur Docker Hub

10.4.2 Alerting automatique (Frontend)

Jenkins envoie également un e-mail de synthèse après chaque pipeline frontend.

- ☒ En cas de succès :

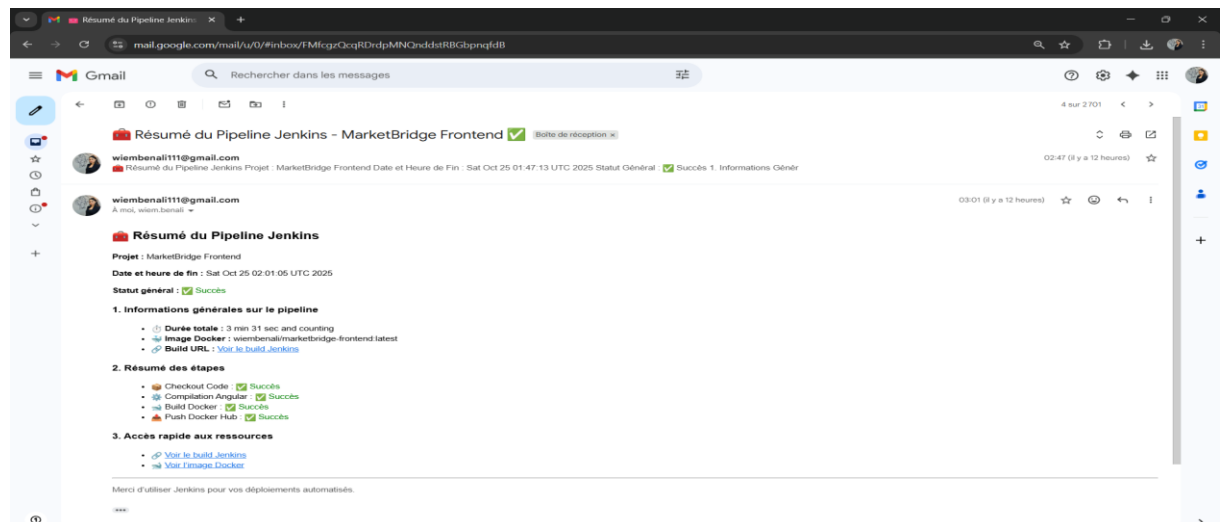


Figure 120: E-mail Jenkins Frontend – Succès

10.4.3 Résultat du pipeline Frontend

Le pipeline **MarketBridge-Frontend-CI-CD** s'exécute en moyenne en **3 minutes 36 secondes**, avec toutes les étapes validées (checkout, build Angular, build/push Docker).

Le **dernier build (#33)** s'est terminé avec succès ✓, confirmant le bon fonctionnement de la chaîne CI/CD.

L'image **Angular/Nginx** est automatiquement générée et publiée sur **Docker Hub** après chaque commit sur la branche principale GitHub.

Ce mécanisme assure une mise à jour fluide et continue de l'interface utilisateur, sans intervention manuelle.

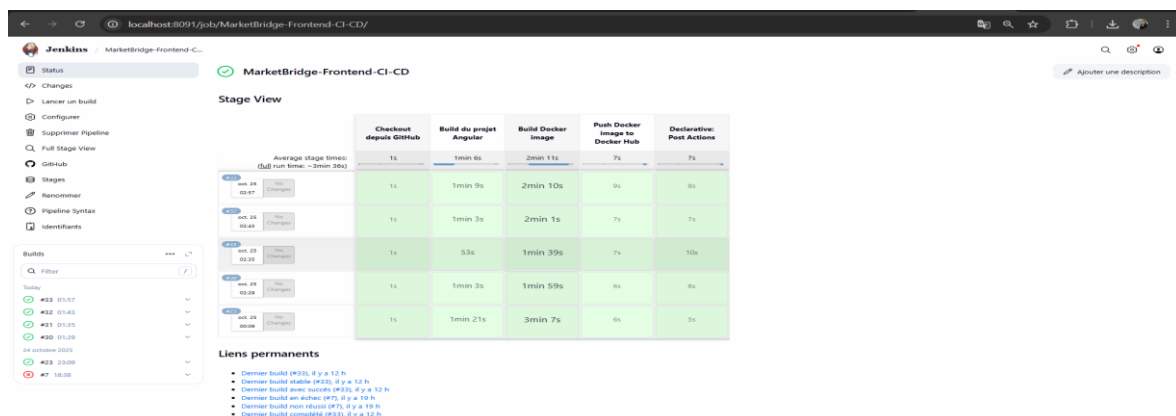


Figure 121: Résultat du pipeline Jenkins – Frontend (étapes du build #33)

Conclusion

Ce sprint DevOps a permis de professionnaliser le déploiement de Market Bridge grâce à une chaîne CI/CD complète et automatisée.

Les outils intégrés (Jenkins, Docker, SonarQube, Nexus, Prometheus, Grafana) assurent un haut niveau de qualité, de traçabilité et de supervision.

Les pipelines fonctionnent indépendamment : si l'un échoue, l'autre poursuit son exécution, assurant ainsi une continuité opérationnelle.

Grâce à cette architecture, les mises à jour du backend et du frontend sont désormais fiables, rapides et prêtes pour la production.

Grâce à cette approche, **aucun build n'est lancé manuellement** : chaque push GitHub déclenche automatiquement l'ensemble de la chaîne CI/CD, ce qui garantit une industrialisation conforme aux standards professionnels DevOps.

Conclusion Générale

Ce stage au sein de l'entreprise **Bee Coders** a été une expérience à la fois enrichissante et professionnalisante. Il m'a offert l'opportunité d'appliquer concrètement les compétences acquises à l'**École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)** à travers le développement de **Market Bridge**, une plateforme web innovante facilitant la mise en relation entre **marques** et **créateurs de contenu**.

Le projet visait à concevoir une solution complète intégrant un moteur de recommandation basé sur l'IA, un système de gestion et de modération des propositions, un module de paiement sécurisé via **Stripe**, ainsi qu'une infrastructure **DevOps** entièrement automatisée. Grâce à la méthodologie **Scrum**, le travail a été organisé en sprints successifs, assurant une progression continue et structurée.

J'ai eu la chance de participer à l'ensemble du cycle de développement : analyse des besoins, modélisation UML, développement **front-end** et **back-end**, jusqu'à la mise en production. Ce projet m'a permis de maîtriser des technologies modernes comme **Spring Boot**, **Angular**, **MySQL**, et divers outils **DevOps** tels que **Jenkins**, **Docker**, **SonarQube**, **Nexus**, **Prometheus** et **Grafana**, ainsi que des modules d'intelligence artificielle développés avec **Flask** et **Chatbot IA**.

Au-delà de l'aspect technique, cette expérience m'a appris à travailler dans un environnement collaboratif exigeant, à gérer les priorités et à m'adapter rapidement aux imprévus. J'y ai renforcé mes compétences en **architecture full stack**, en **intégration et déploiement continu (CI/CD)**, et en **qualité logicielle**, tout en développant un sens accru de la rigueur et de la communication d'équipe.

Je tiens à remercier **M. Ahmed Naffeti**, encadrant professionnel chez Bee Coders, pour son accompagnement et ses précieux conseils, ainsi que **Mme Sinda Belhajdaoud**, encadrante académique à ESPRIT, pour son suivi attentif. Ma gratitude s'étend également à toute l'équipe **Bee Coders** pour leur accueil, leur confiance et leur disponibilité.

Ce projet a confirmé ma passion pour le **développement logiciel**, le **DevOps** et l'**intelligence artificielle**. Il constitue une étape clé dans mon parcours d'ingénieur, marquant le début d'une carrière tournée vers l'innovation et l'excellence technologique.

BIBLIOGRAPHIE

- [1] **Spring Boot – Documentation officielle.** [En ligne]. Disponible sur : <https://spring.io/projects/spring-boot>. 2025.
- [2] **Angular – Documentation officielle.** [En ligne]. Disponible sur : <https://angular.io/docs>. 2025.
- [3] **MySQL – Documentation officielle.** [En ligne]. Disponible sur : <https://dev.mysql.com/doc/>. 2025.
- [4] **Docker – Documentation officielle.** [En ligne]. Disponible sur : <https://docs.docker.com/>. 2025.
- [5] **Jenkins – Documentation officielle.** [En ligne]. Disponible sur : <https://www.jenkins.io/doc/>. 2025.
- [6] **SonarQube – Documentation officielle.** [En ligne]. Disponible sur : <https://docs.sonarqube.org/latest/>. 2025.
- [7] **Nexus Repository Manager – Documentation officielle.** [En ligne]. Disponible sur : <https://help.sonatype.com/repomanager3>. 2025.
- [8] **Prometheus – Documentation officielle.** [En ligne]. Disponible sur : <https://prometheus.io/docs/>. 2025.
- [9] **Grafana – Documentation officielle.** [En ligne]. Disponible sur : <https://grafana.com/docs/>. 2025.
- [10] **Stripe API – Documentation officielle.** [En ligne]. Disponible sur : <https://stripe.com/docs/api>. 2025.



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454